

Jupyter Notebook Execution Report

Name: Your Name

Date: April 04, 2026

Cell 1: ■ Markdown

Análisis de Préstamos Hipotecarios con Machine Learning

■ Proyecto: HMDA Utah 2022

****Análisis completo de datos hipotecarios del estado de Utah (2022) utilizando técnicas de Machine Learning para clasificación, interpretabilidad y detección de sesgos.****

■ *Componentes del análisis:*

- ■ ****Clasificación supervisada****: Regresión Logística, Random Forest, Gradient Boosting
- ■ ****Evaluación de modelos****: Métricas de rendimiento, curvas ROC y PR
- ■ ****Interpretabilidad****: SHAP e importancia de variables
- ■■ ****Análisis ético****: Detección de sesgos por raza, sexo y edad
- ■ ****Clustering****: K-Means, DBSCAN y Clustering Jerárquico

Cell 2: ■ Markdown

1. Introducción al proyecto

1.1 ¿Qué es el dataset HMDA?

El ****Home Mortgage Disclosure Act (HMDA)**** es una ley federal de Estados Unidos que requiere que las instituciones financieras reporten públicamente datos sobre préstamos hipotecarios. Esta información permite:

- Determinar si las instituciones financieras están atendiendo las necesidades de vivienda de sus comunidades
- Ayudar a los funcionarios públicos a distribuir inversiones del sector público
- Identificar posibles patrones de discriminación en la concesión de préstamos

Los datos HMDA incluyen información detallada sobre cada solicitud de préstamo hipotecario, incluyendo:

- Características del préstamo (monto, tipo, propósito)
- Características del solicitante (ingresos, raza, sexo, edad)
- Información geográfica
- Resultado de la solicitud (aprobada, denegada, etc.)

1.2 ¿Por qué Utah 2022?

Para este proyecto se ha seleccionado el subconjunto de datos correspondiente al estado de **Utah** del año **2022** por las siguientes razones:

1. **Viabilidad computacional**: el dataset nacional completo contiene millones de registros. El subconjunto de Utah ofrece un tamaño manejable para análisis sin comprometer la representatividad.
2. **Datos recientes**: los datos de 2022 reflejan prácticas actuales de concesión de préstamos.
3. **Enfoque de estudio de caso**: permite un análisis profundo de un mercado hipotecario específico, facilitando interpretaciones contextualizadas.

1.3 Objetivos del proyecto

Los objetivos concretos de este proyecto son:

1. **Definir y construir el target**: convertir `action_taken` en una variable binaria (aprobado/denegado) y justificar la codificación usada.
2. **Comparar rendimiento de modelos**: entrenar Regresión Logística, Random Forest y Gradient Boosting y comparar **AUC, F1, recall y matriz de confusión** para seleccionar el mejor modelo.
3. **Identificar variables más influyentes**: extraer e interpretar las 10 características con mayor impacto usando **feature importance y SHAP**.
4. **Evaluar sesgos por subgrupos**: medir diferencias de tasas de aprobación y métricas del modelo por **raza, sexo y edad**, y cuantificar brechas.
5. **Analizar patrones no supervisados**: aplicar **K-Means y DBSCAN** para descubrir perfiles de solicitantes y relacionarlos con la aprobación.
6. **Extraer conclusiones accionables**: sintetizar hallazgos sobre desempeño, interpretabilidad y equidad para recomendaciones finales.

Cell 3: ■ Markdown

2. Descripción del conjunto de datos

2.1 Origen de los datos

Los datos utilizados en este proyecto provienen del **Consumer Financial Protection Bureau (CFPB)**, que administra la recopilación y publicación de datos bajo el HMDA. El dataset específico ha sido obtenido de Kaggle:

■ ****Fuente****: [Utah HMDA Data 2018-2022](https://www.kaggle.com/datasets/richcordova/utah-hmda-data-2018-2022/data?select=utah_hmda_data_2022.csv)

2.2 Estructura general del dataset

El dataset de Utah 2022 contiene información sobre solicitudes de préstamos hipotecarios procesadas por instituciones financieras en el estado de Utah durante el año 2022. Cada fila representa una solicitud de préstamo individual.

2.3 Variables principales

Variable	Descripción	Tipo
`action_taken`	Resultado de la solicitud (aprobada, denegada, retirada, etc.)	Categórica (Target)
`loan_amount`	Monto del préstamo solicitado	Numérica
`income`	Ingresos anuales del solicitante (en miles de dólares)	Numérica
`loan_to_value_ratio`	Ratio préstamo-valor de la propiedad	Numérica
`debt_to_income_ratio`	Ratio deuda-ingresos del solicitante	Categórica
`property_value`	Valor estimado de la propiedad	Numérica
`loan_type`	Tipo de préstamo (convencional, FHA, VA, etc.)	Categórica
`loan_purpose`	Propósito del préstamo (compra, refinanciamiento, etc.)	Categórica
`occupancy_type`	Tipo de ocupación de la propiedad	Categórica
`derived_race`	Raza del solicitante principal	Categórica (Sensible)
`derived_sex`	Sexo del solicitante principal	Categórica (Sensible)
`applicant_age`	Edad del solicitante	Categórica (Sensible)
`county_code`	Código del condado	Categórica

2.4 Atributos sensibles para el análisis ético

■ ****Atributos sensibles identificados****

Los siguientes atributos se consideran ****sensibles**** desde una perspectiva ética y legal, ya que están protegidos bajo leyes de no discriminación:

- ****`derived\_race`****: raza/etnicidad del solicitante
- ****`derived\_sex`****: sexo/género del solicitante
- ****`applicant\_age`****: edad del solicitante

Estos atributos serán analizados específicamente en la sección de análisis ético para detectar posibles sesgos en las predicciones del modelo.

3. Distribución del trabajo

■ **Robert: limpieza inicial y preprocesamiento de datos**

****Responsabilidades:****

- Carga y limpieza inicial de los datos
- Análisis exploratorio de datos (EDA)
- Construcción de pipelines de preprocesamiento
- Selección y codificación de variables

****Secciones del notebook:**** 4, 5

■ **Rodrigo: modelado e interpretabilidad de modelos**

****Responsabilidades:****

- Entrenamiento de modelos de machine learning
- Evaluación inicial mediante métricas de rendimiento
- Implementación de análisis SHAP (SHapley Additive exPlanations)
- Análisis de importancia de variables
- Interpretación del comportamiento global y local del modelo
- Visualización de explicaciones

****Secciones del notebook:**** 6, 7, 8

■ **Juan: análisis ético y detección de sesgos**

****Responsabilidades:****

- Análisis de rendimiento por subgrupos (fairness)
- Detección de sesgos en datos y predicciones
- Comparación de métricas entre grupos sensibles
- Discusión crítica de implicaciones éticas
- Redacción de conclusiones y recomendaciones

****Secciones del notebook:**** 9

■ Conjunto: análisis de clustering (segmentación de solicitantes)

****Responsabilidades:****

- Preparación de los datos para aprendizaje no supervisado
- Implementación de K-Means con Método del Codo y Silhouette Score
- Implementación de DBSCAN para detección de solicitudes atípicas
- Clustering Jerárquico y comparativa de algoritmos
- Perfilado e interpretación de los clusters descubiertos
- Análisis de la relación entre clusters y decisiones de aprobación
- Estudio de la distribución demográfica por cluster (dimensión ética)

****Secciones del notebook:**** 10

Cell 5: ■ Markdown

4. Carga de datos y exploración inicial

4.1 Configuración del entorno

Primero importamos todas las librerías necesarias para el análisis.

Cell 6: ■ Code

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Visualización
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('Set2')
```

Cell 7: ■ Markdown

4.2 Carga del dataset

Cargamos el dataset de Utah HMDA 2022 desde la carpeta `data/`.

Cell 8: ■ Code

```
# Carga del dataset HMDA Utah 2022

df = pd.read_csv('data/utah_hmda_data_2022.csv')

# Guardar una copia del DataFrame original para no sobrescribir los datos
originales

df_original = df.copy()

# Visualizamos las primeras filas para comprobar la carga

df.head()
```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~^
  File "<string>", line 2, in <module>;
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf.py", line 100, in _read
    return _read(filepath_or_buffer, kwds)
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf.py", line 100, in _read
    parser = TextFileReader(filepath_or_buffer, **kwds)
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf.py", line 100, in _read
    self._engine = self._make_engine(f, self.engine)
                    ~~~~~^
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf.py", line 100, in _read
    self.handles = get_handle(
                    ~~~~~^
                    f,
                    ^^
    ...<6 lines>...
        storage_options=self.options.get("storage_options", None),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf.py", line 100, in _read
    handle = open(
        handle,
    ...<3 lines>...
        newline="",
    )
FileNotFoundError: [Errno 2] No such file or directory: 'data/utah_hmda_data_2022.csv'

```

Cell 9: ■ Markdown

4.3 Exploración inicial del dataset

Examinamos la estructura básica del dataset: primeras filas, tipos de datos e información general.

Cell 10: ■ Code

```
# Visualizar las primeras 10 filas del dataset  
df.head(10)
```

Error:

```
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403  
    result = eval(lines[-1], glb)  
  File "<string>", line 1, in <module>  
NameError: name 'df' is not defined
```

Cell 11: ■ Code

```
# Información general del dataset  
df.info()
```

Error:

```
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403  
    result = eval(lines[-1], glb)  
  File "<string>", line 1, in <module>  
NameError: name 'df' is not defined
```

Cell 12: ■ Code

```
# Estadísticas descriptivas de variables numéricas (limitadas a columnas clave para  
evitar sobrecarga)  
cols_numericas_principales = ['action_taken', 'loan_amount', 'loan_type',  
'loan_purpose',  
'interest_rate', 'loan_term', 'property_value', 'income',  
'purchaser_type', 'preapproval', 'lien_status', 'occupancy_type']  
cols_existentes = [c for c in cols_numericas_principales if c in  
df.select_dtypes(include=['number']).columns]  
df[cols_existentes].describe().round(2)
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 5, in <module>
NameError: name 'df' is not defined
```

Cell 13: ■ Markdown

4.4 Análisis y eliminación de valores innecesarios

Primero vamos a realizar un análisis previo para confirmar qué columnas vamos a eliminar del dataset, ya sea por excesiva cantidad de nulos, identificadores irrelevantes, duplicados o que no aportan valor para el análisis.

Cell 14: ■ Code

```
# Lista donde almacenaremos las columnas a eliminar
columnas_a_eliminar = []
motivoseliminacion = {}

# 1. Columnas con demasiados nulos (>60%)
porcentaje_nulos = df.isnull().mean()
columnas_nulos = porcentaje_nulos[porcentaje_nulos > 0.6].index.tolist()

for col in columnas_nulos:
    columnas_a_eliminar.append(col)
    motivoseliminacion[col] = "Más del 60% de valores nulos"

# 2. Columnas identificadoras (IDs)
columnas_id = [col for col in df.columns if 'id' in col.lower()]

for col in columnas_id:
    if col not in columnas_a_eliminar:
        columnas_a_eliminar.append(col)
        motivoseliminacion[col] = "Columna identificadora sin valor analítico"

# 3. Columnas constantes o casi constantes
columnas_constantes = [
    col for col in df.columns
    if df[col].nunique(dropna=False) <= 1
```



```

]

for col in columnas_constantes:
if col not in columnas_a_eliminar:
columnas_a_eliminar.append(col)
motivos Eliminacion[col] = "Columna constante o sin variabilidad"

# 4. Columnas duplicadas (comparación eficiente columna a columna, sin transponer)
cols_revisadas = set()
for i, col1 in enumerate(df.columns):
if col1 in columnas_a_eliminar:
continue
for col2 in df.columns[i+1:]:
if col2 in columnas_a_eliminar or col2 in cols_revisadas:
continue
if df[col1].equals(df[col2]):
cols_revisadas.add(col2)
columnas_a_eliminar.append(col2)
motivos Eliminacion[col2] = f"Columna duplicada (idéntica a '{col1}')"

# Resultado final
print(f"Total de columnas a eliminar: {len(columnas_a_eliminar)}\n")
print("Columnas a eliminar:")
for col in columnas_a_eliminar:
print(f" - {col}: {motivos Eliminacion[col]}")

# Crear el dataframe limpio (sin guardarlo a CSV para ahorrar memoria)
df_limpio = df_original.drop(columns=columnas_a_eliminar)
print(f"\nColumnas restantes: {len(df_limpio.columns)}")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 6, in <module>
NameError: name 'df' is not defined

```

Cell 15: ■ Markdown

¿Por qué eliminamos estas columnas?

1. **Columnas con >60% de valores nulos**: no aportan información útil al análisis porque la mayoría de sus valores están ausentes. Se ha elegido un umbral del 60% como punto de equilibrio: es lo suficientemente estricto para eliminar columnas con datos predominantemente faltantes, pero no tan agresivo como para perder variables con información parcial recuperable. Este umbral es consistente con el filtrado aplicado en el código (`porcentaje_nulos > 0.6`).
2. **Identificadores irrelevantes**: columnas como 'lei' y 'census_tract' son identificadores únicos o geográficos que no contribuyen al modelado predictivo y pueden introducir ruido.
3. **Columnas duplicadas o redundantes**: algunas variables recogen la misma información que otras o son observaciones auxiliares (por ejemplo, variantes de etnia, raza o sexo del co-solicitante) que no se utilizarán en el análisis principal.
4. **Variables que no aportan valor analítico**: columnas que, tras el análisis exploratorio, se considera que no añaden información relevante para los objetivos del proyecto.

El objetivo actual es simplificar el dataset, reducir la dimensionalidad y evitar problemas de multicolinealidad o sobreajuste en los modelos posteriores.

Cell 16: ■ Code

```
# Mostramos las primeras filas del dataset tras eliminar las columnas seleccionadas
df_limpio.head()
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403

result = eval(lines[-1], glb)

File "<string>", line 1, in <module>

NameError: name 'df_limpio' is not defined

Cell 17: ■ Markdown

En conclusión, tras esta limpieza de columnas innecesarias hemos podido simplificar el dataset, reducir la dimensionalidad y con ello podemos evitar problemas de multicolinealidad o sobreajuste en los modelos posteriores.

Cell 18: ■ Markdown

4.5 Exploración de la variable objetivo

La variable `action_taken` indica el resultado de la solicitud de préstamo, y es la mejor variable objetivo.

- Representa el evento a predecir
- No es una variable explicativa (es decir, es el resultado)
- Está disponible al final del proceso

- Es estándar en literatura y proyectos HMDA
- Permite múltiples enfoques (clasificación binaria y multiclase)

Analizamos su distribución.

Cell 19: ■ Code

```
# Definición de la variable objetivo
variable_objetivo = 'action_taken'

# Análisis de la variable objetivo
plt.figure(figsize=(7,4))
sns.countplot(x=variable_objetivo, data=df_limpio,
order=df_limpio[variable_objetivo].value_counts().index)
plt.title(f'Distribución de la variable {variable_objetivo}')
plt.xlabel('Resultado de la solicitud')
plt.ylabel('Cantidad')
plt.show()

# Mostrar recuento y proporción
display(df_limpio[variable_objetivo].value_counts())
display(df_limpio[variable_objetivo].value_counts(normalize=True).round(3))
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 6, in <module>
NameError: name 'df_limpio' is not defined
```

Cell 20: ■ Markdown

Podemos ver que la variable objetivo `action\_taken`, que representa el resultado final de cada solicitud de préstamo hipotecario, refleja que la categoría 1 es la más frecuente. El resto de categorías (2, 3, 4, 5, 6) tienen frecuencias notablemente menores (algunas clases son residuales, 7 y 8).

Esto es lo esperable en este dataset, donde la mayoría de solicitudes terminan en un resultado correcto (*loan originated*) y donde existen múltiples resultados alternativos con menor frecuencia.

Es importante saber que aunque hay diferencias entre clases, no estamos ante un **desbalance extremo**.

Cell 21: ■ Markdown

4.5.1. Exploración de la variable objetivo

Cell 22: ■ Code

```
# Distribución absoluta
dist_abs = df_limpio[variable_objetivo].value_counts()

# Distribución relativa
dist_rel = df_limpio[variable_objetivo].value_counts(normalize=True) * 100

df_dist = pd.DataFrame({
    "Frecuencia": dist_abs,
    "Porcentaje (%)": dist_rel.round(2)
})

df_dist
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df_limpio' is not defined
```

Cell 23: ■ Code

```
# Análisis de desbalanceo
clase_mayoritaria = dist_rel.max()

print(
    f"La clase mayoritaria representa aproximadamente "
    f"el {clase_mayoritaria:.2f}% del total de observaciones."
)
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4
    print(
```

```

^
SyntaxError: '(' was never closed
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'dist_rel' is not defined

```

Cell 24: ■ Markdown

Tras la exploración de la variable objetivo, podemos ver que estamos ante un dataset que no está completamente balanceado pero tampoco está severamente desbalanceado, totalmente aceptable para EDA y análisis exploratorio.

Cell 25: ■ Markdown

4.6 Exploración de variables clave

Visualizamos la distribución de algunas variables importantes para el análisis.

En este caso vamos a explorar las siguientes variables `numéricas`:

- loan_amount
- interest_rate
- loan_term
- income
- debt_to_income_ratio
- property_value

Y, por supuesto, las siguientes variables `categóricas`:

- loan_type
- loan_purpose
- occupancy_type
- lien_status

Cell 26: ■ Code

```

variables_numericas_clave = [
    'loan_amount',

```

```
'interest_rate',  
'loan_term',  
'income',  
'property_value'  
]
```

```
df_limpio[variables_numericas_clave].describe()
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403

result = eval(lines[-1], glb)

File "<string>", line 1, in <module>

NameError: name 'df_limpio' is not defined

Cell 27: ■ Code

```
# Distribución de las variables numéricas clave
```

```
# La variable 'income' en HMDA está expresada en MILES DE DÓLARES.
```

```
# Además, variables como income, loan_amount y property_value presentan outliers  
extremos que comprimen la escala visual.
```

```
# Por ello, usamos:
```

```
# - Histograma normal para variables con distribución razonable
```

```
# - Histograma con escala logarítmica para variables con outliers extremos
```

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
```

```
axes = axes.flatten()
```

```
for i, var in enumerate(variables_numericas_clave):
```

```
ax = axes[i]
```

```
col_data = pd.to_numeric(df_limpio[var], errors='coerce').dropna()
```

```
# Identificar variables con outliers extremos (ratio max/median > 20)
```

```
median_val = col_data.median()
```

```
max_val = col_data.max()
```

```
tiene_outliers_extremos = (median_val > 0) and (max_val / median_val > 20)
```

```
if tiene_outliers_extremos:
```

```
# Usar escala logarítmica para variables con outliers extremos
```

```
col_positiva = col_data[col_data > 0]
```

```

ax.hist(np.log10(col_positiva), bins=50, edgecolor='black', alpha=0.7)
ax.set_xlabel(f'log10({var})')
ax.set_title(f'{var} (escala log10)\nMediana: {median_val:,.0f} | Max: {max_val:,.0f}')
else:
sns.histplot(col_data, bins=30, kde=True, ax=ax)
ax.set_title(f'Distribución de {var}')

# Ocultar el subplot vacío si hay número impar de variables
if len(variables_numericas_clave) < len(axes):
for j in range(len(variables_numericas_clave), len(axes)):
axes[j].set_visible(False)

plt.suptitle('Distribución de Variables Numéricas Clave\n(Variables con outliers extremos se muestran en escala logarítmica)',
fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

# Mostrar estadísticas de outliers para income
print("\n--- Análisis de outliers en 'income' ---")
income_data = pd.to_numeric(df_limpio['income'], errors='coerce').dropna()
print(f" Unidad: miles de dólares (según documentación HMDA)")
print(f" Media: ${income_data.mean():,.0f}k | Mediana: ${income_data.median():,.0f}k")
print(f" Percentil 99: ${income_data.quantile(0.99):,.0f}k")
print(f" Percentil 99.5: ${income_data.quantile(0.995):,.0f}k")
print(f" Máximo: ${income_data.max():,.0f}k")
print(f" Registros con income > $1,000k: {(income_data > 1000).sum()} ({(income_data > 1000).mean()*100:.2f}%)")

```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 14, in <module>

NameError: name 'df\_limpio' is not defined

#### Cell 28: ■ Markdown

Tras la distribución de las variables numéricas clave, podemos sacar las siguientes observaciones generales:

- En `loan\_amount`, podemos ver que hay una distribución muy sesgada a la derecha, donde la mayoría de los préstamos son de bajo montos, pero existen algunos valores extremadamente altos (posibles outliers o errores de carga).
- En `interest\_rate`, podemos ver una distribución unimodal, donde la mayoría de las tasas se concentran entre un 3% y 7%. Hay también una cola derecha, donde existen tasas más altas, pero poco frecuentes.
- En `loan\_term`, podemos observar que hay picos claros, donde la mayoría de los préstamos tienen plazos estándar (ej. 360 meses = 30 años). Hay otros valores que pueden ser otros plazos, pero son minoritarios.
- En `income`, podemos ver una distribución muy sesgada, donde la mayoría de los ingresos son bajos o moderados, pero hay valores extremadamente altos (outliers).
- En `property\_value`, podemos ver una distribución similar a loan\_amount e income, donde se concentra la mayoría de los valores bajos/moderados y algunos valores extremadamente altos por posibles outliers.

##### Conclusiones generales

Hemos podido observar que varias variables presentan valores extremos que pueden distorsionar el análisis y los modelos. Es recomendable analizar y posiblemente tratar estos outliers. Por último, antes de modelar, sería útil revisar los valores máximos y mínimos de cada variable para detectar errores de carga o codificación.

##### Para las variables categóricas:

#### Cell 29: ■ Code

```
variables_categoricas_clave = [  
    'loan_type',  
    'loan_purpose',  
    'occupancy_type',  
    'lien_status',  
    'derived_race',  
    'derived_sex',  
    'applicant_age',  
    'debt_to_income_ratio'  
]  
  
for var in variables_categoricas_clave:  
    plt.figure(figsize=(7, 4))  
    df_limpio[var].value_counts().plot(kind='bar')
```



```
plt.title(f'Distribución de {var}')
plt.xlabel(var)
plt.ylabel('Frecuencia')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 14, in <module>

NameError: name 'df_limpio' is not defined

Cell 30: ■ Markdown

Tras la distribución de las variables categóricas clave, podemos sacar las siguientes observaciones generales:

- En `loan\_type`, podemos ver que la categoría 1 es la más frecuente, representando la mayoría de los préstamos (probablemente préstamos convencionales). Las categorías 2 y 3 tienen una presencia mucho menor, y la categoría 4 es residual o casi inexistente.
- En `loan\_purpose`, la categoría 1 es la dominante (probablemente compra de vivienda), seguida por las categorías 32 y 31 (posiblemente refinanciamiento y cash-out refinance). El resto de categorías tienen frecuencias menores, algunas casi residuales.
- En `occupancy\_type`, la categoría 1 es la más común (probablemente propietario ocupa la vivienda), mientras que las categorías 3 y 2 son mucho menos frecuentes, lo que indica que la mayoría de los préstamos son para viviendas ocupadas por el propietario.
- En `lien\_status`, la categoría 1 es la más frecuente (primer gravamen), y la categoría 2 tiene una presencia mucho menor, lo que refleja que la mayoría de los préstamos hipotecarios son de primer gravamen.
- En `derived\_race`, observamos que la categoría "White" es la más frecuente, representando la gran mayoría de los solicitantes. El resto de categorías (como "Asian", "Black or African American", "Native Hawaiian or Other Pacific Islander", etc.) tienen una presencia mucho menor, y algunas son prácticamente residuales. También destaca la categoría "Race Not Available", que tiene una frecuencia significativa, indicando registros donde no se reportó la raza.
- En `derived\_sex`, la categoría "Joint" (solicitud conjunta) es la más común, seguida por "Male". "Female" tiene una frecuencia considerablemente menor, y "Sex Not Available" representa un grupo relevante de registros donde no se reportó el sexo del solicitante.
- En `applicant\_age`, la categoría "35-44" es la más frecuente, seguida por "25-34" y "45-54". Las categorías de mayor edad ("65-74", ">74") y menor edad ("<25") tienen una presencia mucho menor, lo que indica que la mayoría de los solicitantes se encuentran en rangos de edad intermedia. También hay una categoría "8888" que podría representar valores faltantes o codificados de manera especial.

- En `debt\_to\_income\_ratio`, las categorías más frecuentes son los intervalos '20%<30%' y '30%<36%', seguidos por '50%<60%', '<20%' y '>60%'. Los valores numéricos individuales como 44, 42, 41, etc., tienen una presencia mucho menor, lo que indica que la mayoría de los solicitantes se agrupan en rangos discretos de ratio deuda-ingresos. La categoría 'Exempt' aparece con menor frecuencia, representando casos donde el ratio no se reporta o está exento. Este patrón refleja que la variable está discretizada y contiene tanto intervalos como valores puntuales y etiquetas especiales.

Conclusiones generales

Hemos podido observar que en todas las variables categóricas existe una clara categoría dominante, lo que refleja patrones típicos del mercado hipotecario: predominan los préstamos convencionales, para compra de vivienda, ocupados por el propietario y de primer gravamen. Las categorías minoritarias pueden ser relevantes para análisis de subgrupos, pero no tienen suficiente peso para influir en el comportamiento global del dataset. Este patrón es importante para la interpretación y el modelado, ya que puede influir en la segmentación y el análisis de equidad.

Cell 31: ■ Markdown

5. Preprocesamiento de datos

En esta sección realizamos el preprocesamiento necesario para preparar los datos para el modelado:

- Selección de variables relevantes
- Creación de la variable objetivo binaria
- Tratamiento de valores faltantes
- Codificación de variables categóricas
- División train/test con pipelines para evitar data leakage

Cell 32: ■ Markdown

5.0 Tratamiento de valores especiales como nulos

Antes de proceder con la selección de variables y el modelado, es fundamental tratar los **valores especiales** del dataset HMDA que representan "no disponible", "no aplica" o "exento". Estos valores están codificados numéricamente pero **no son datos reales**, sino códigos administrativos.

Según la [documentación oficial del CFPB](https://ffiec.cfpb.gov/documentation/), los siguientes valores especiales son utilizados:

Código	Significado	Variables donde aparece
-----	-----	-----
`1111`	No aplica (Exempt)	Varias numéricas
`8888`	No aplica / No reportado	`applicant_age`
`9999`	No disponible	Varias numéricas

| `Exempt` | Exento de reporte | `debt\_to\_income\_ratio`, entre otras |

| `NA` / `NaN` | Valor faltante estándar | Todas |

****¿Por qué es crucial este paso?****

- Si estos valores permanecen como números, ****distorsionan las estadísticas descriptivas**** (medias, medianas, percentiles)
- Afectan las ****distribuciones visualizadas**** (ej: `income` con valores 9999 que comprimen la escala)
- Pueden ****introducir patrones espurios**** en los modelos de ML
- Los algoritmos de clustering y SHAP los interpretarán como valores reales

Cell 33: ■ Code

```
# Valores especiales numéricos a reemplazar por NaN
valores_especiales_numericos = [9999, 1111, 8888, 999, -1]

# Identificar columnas numéricas del dataframe limpio
columnas_numericas = df_limpio.select_dtypes(include=['number']).columns.tolist()

# Contar cuántos valores especiales hay ANTES del reemplazo
print("=" * 60)
print("VALORES ESPECIALES DETECTADOS (antes del reemplazo)")
print("=" * 60)

for col in columnas_numericas:
    for val in valores_especiales_numericos:
        count = (df_limpio[col] == val).sum()
        if count > 0:
            print(f" {col}: valor {val} aparece {count} veces  
( {count/len(df_limpio)*100:.2f}% )")

# Reemplazar valores especiales numéricos por NaN
for col in columnas_numericas:
    df_limpio[col] = df_limpio[col].replace(valores_especiales_numericos, np.nan)

# Tratar el valor "Exempt" en columnas categóricas como NaN o categoría especial
# En debt_to_income_ratio, "Exempt" indica exención de reporte
# Lo mantenemos como categoría especial "Exempt" ya que tiene significado propio
print("\n--- Valores 'Exempt' en debt_to_income_ratio ---")
exempt_count = (df_limpio['debt_to_income_ratio'] == 'Exempt').sum()
```

```

print(f" Registros con 'Exempt': {exempt_count}
({exempt_count/len(df_limpio)*100:.2f}%)")

# Tratar "8888" en applicant_age (viene como string en el dataset original)
if df_limpio['applicant_age'].dtype == 'object':
age_8888 = (df_limpio['applicant_age'] == '8888').sum()
print(f"\n--- Valores '8888' en applicant_age ---")
print(f" Registros con '8888': {age_8888} ({age_8888/len(df_limpio)*100:.2f}%)")
df_limpio['applicant_age'] = df_limpio['applicant_age'].replace('8888', np.nan)

print("\n■ Valores especiales reemplazados correctamente por NaN.")

```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

File "<string>", line 5, in <module>

NameError: name 'df\_limpio' is not defined

### Cell 34: ■ Markdown

#### 5.0.1 Consolidación de categorías de raza

Las categorías "Free Form Text Only" (raza introducida manualmente sin coincidir con opciones estándar) y "Race Not Available" (raza no reportada) comparten una característica común: **no proporcionan una categorización racial utilizable** para el análisis de equidad. Por ello, las combinamos en una única categoría.

**\*\*Justificación:\*\***

- "Free Form Text Only" indica que el solicitante escribió su raza en texto libre, pero no fue categorizada → no hay información estructurada
- "Race Not Available" indica que la raza no fue reportada → no hay información
- Mantenerlas separadas crea ruido innecesario en el análisis ético
- La unificación preserva la información útil: "sabemos que no tenemos dato categorizado confiable"

### Cell 35: ■ Code

```

# Distribución ANTES de la consolidación
print("Distribución de derived_race ANTES de la consolidación:")
print(df_limpio['derived_race'].value_counts())
print()

```

```

# Unificar "Free Form Text Only" y "Race Not Available" en una sola categoría
categorias_a_unificar = ['Free Form Text Only', 'Race Not Available']
df_limpio['derived_race'] = df_limpio['derived_race'].replace(
categorias_a_unificar,
'Not Available or Free Text'
)

# Distribución DESPUÉS de la consolidación
print("Distribución de derived_race DESPUÉS de la consolidación:")
print(df_limpio['derived_race'].value_counts())

# Verificar que las categorías originales ya no existen
assert 'Free Form Text Only' not in df_limpio['derived_race'].values, \
"Error: 'Free Form Text Only' todavía presente"
assert 'Race Not Available' not in df_limpio['derived_race'].values, \
"Error: 'Race Not Available' todavía presente"

print("\n■ Categorías de raza consolidadas correctamente.")

```

#### Output:

```
Distribución de derived_race ANTES de la consolidación:
```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 3, in <module>
NameError: name 'df_limpio' is not defined

```

### Cell 36: ■ Markdown

#### 5.1 Selección de variables y creación de variable objetivo

```
Documentación de la codificación de variables
```

Antes de transformar las variables, es imprescindible saber el significado de los valores originales en el dataset HMDA.

```
Variable objetivo: `action_taken`
```

| Valor | Significado HMDA | Codificación binaria |
|-------|------------------|----------------------|
| ----- | -----            | -----                |

| 1 | **\*\*Loan originated\*\*** (préstamo concedido) | ■ `approved = 1` |

| 2 | **\*\*Application approved but not accepted\*\*** (aprobado pero no aceptado por el solicitante) | ■ `approved = 0` |

| 3 | **\*\*Application denied\*\*** (solicitud denegada) | ■ `approved = 0` |

| 4 | **\*\*Application withdrawn\*\*** (solicitud retirada por el solicitante) | ■ `approved = 0` |

| 5 | **\*\*File closed for incompleteness\*\*** (expediente cerrado por documentación incompleta) | ■ `approved = 0` |

| 6 | **\*\*Purchased loan\*\*** (préstamo comprado en mercado secundario) | ■ `approved = 0` |

| 7 | **\*\*Preapproval request denied\*\*** (preaprobación denegada) | ■ `approved = 0` |

| 8 | **\*\*Preapproval request approved but not accepted\*\*** | ■ `approved = 0` |

**\*\*Criterio de agrupación\*\*:** Solo `action\_taken == 1` (préstamo efectivamente concedido) se considera "aprobado". El resto, incluyendo solicitudes aprobadas pero no aceptadas (2), retiradas (4) y preaprobaciones (7, 8), se agrupa como "no aprobado" porque el objetivo es predecir si el préstamo se origina realmente.

##### Variable `loan\_type`

| Valor | Significado |

|-----|-----|

| 1 | **\*\*Conventional\*\*** (préstamo convencional, no asegurado por el gobierno) |

| 2 | **\*\*FHA\*\*** (Federal Housing Administration, asegurado por la FHA) |

| 3 | **\*\*VA\*\*** (Veterans Affairs, para veteranos militares) |

| 4 | **\*\*USDA/RHS\*\*** (Rural Housing Service, para zonas rurales) |

##### Variable `loan\_purpose`

| Valor | Significado |

|-----|-----|

| 1 | **\*\*Home purchase\*\*** (compra de vivienda) |

| 2 | **\*\*Home improvement\*\*** (mejora de vivienda) |

| 31 | **\*\*Refinancing\*\*** (refinanciamiento) |

| 32 | **\*\*Cash-out refinancing\*\*** (refinanciamiento con retiro de efectivo) |

| 4 | **\*\*Other purpose\*\*** (otro propósito) |

##### Variable `occupancy\_type`

| Valor | Significado |

|-----|-----|

|   |                                                  |
|---|--------------------------------------------------|
| 1 | **Principal residence** (residencia principal)   |
| 2 | **Second residence** (segunda residencia)        |
| 3 | **Investment property** (propiedad de inversión) |

##### Variable `lien\_status`

|       |                                                                |
|-------|----------------------------------------------------------------|
| Valor | Significado                                                    |
| ----- | -----                                                          |
| 1     | **First lien** (primer gravamen / hipoteca principal)          |
| 2     | **Subordinate lien** (gravamen subordinado / segunda hipoteca) |

### Cell 37: ■ Code

```
Variables seleccionadas para el modelado
variables_modelo = [
 'loan_amount', 'interest_rate', 'loan_term', 'income', 'debt_to_income_ratio',
 'property_value', 'loan_type', 'loan_purpose', 'occupancy_type', 'lien_status',
 'derived_race', 'derived_sex', 'applicant_age'
]

Crear variable objetivo binaria
df_limpio['approved'] = (df_limpio['action_taken'] == 1).astype(int)

Documentar la transformación
print("Codificación de action_taken → approved:")
print(" action_taken == 1 (Loan originated) → approved = 1")
print(" action_taken != 1 (Denegado/Retirado/Otros) → approved = 0")
print()

Visualizar la distribución de la nueva variable objetivo
print("Distribución de la variable objetivo 'approved':")
print(df_limpio['approved'].value_counts())
print()
print("Proporciones:")
print(df_limpio['approved'].value_counts(normalize=True).round(3))

Filtrar el dataframe para solo las variables seleccionadas + objetivo
df_modelo = df_limpio[variables_modelo + ['approved']].copy()
```

```
Mostrar las primeras filas
df_modelo.head()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 9, in <module>
NameError: name 'df_limpio' is not defined
```

#### Cell 38: ■ Markdown

Tras la selección de variables y la creación de la variable objetivo, podemos sacar las siguientes observaciones generales:

- Se han seleccionado las variables más relevantes del dataset para el modelado, incluyendo tanto variables numéricas (como `loan_amount`, `interest_rate`, `income`, etc.) como categóricas (`loan_type`, `loan_purpose`, `occupancy_type`, `lien_status`, y atributos sensibles como `derived_race`, `derived_sex`, `applicant_age`).
- Se ha creado una variable objetivo binaria (`approved`), donde 1 indica que el préstamo fue aprobado y 0 que fue denegado. Esta codificación facilita el análisis y la aplicación de modelos de clasificación supervisada.
- Al visualizar la distribución de la nueva variable objetivo, se observa que la mayoría de las solicitudes fueron aprobadas, aunque existe una proporción significativa de solicitudes denegadas, lo que permite un análisis equilibrado entre ambas clases.
- El dataframe resultante (`df_modelo`) contiene únicamente las variables seleccionadas y la variable objetivo, lo que simplifica el preprocesamiento y el entrenamiento de modelos.

#### Cell 39: ■ Markdown

### 5.2 Filtrado y limpieza de datos

#### Cell 40: ■ Code

```
print("=" * 60)
print("ANÁLISIS DE VALORES FALTANTES POR VARIABLE")
print("=" * 60)

# Paso 1: Garantizar que las variables numéricas son numéricas
for col in variables_numericas_clave:
    df_modelo[col] = pd.to_numeric(df_modelo[col], errors='coerce')
```



```

# Paso 2: Mostrar % de nulos por columna
registros_iniciales = len(df_modelo)
print(f"\nRegistros iniciales: {registros_iniciales}")
print(f"\nDistribución de 'approved' ANTES de la limpieza:")
print(df_modelo['approved'].value_counts(normalize=True).round(3))

print(f"\nNulos por variable:")
nulos_por_col = df_modelo.isnull().sum()
for col in df_modelo.columns:
    n_nulos = nulos_por_col[col]
    if n_nulos > 0:
        print(f" {col}: {n_nulos} nulos ({n_nulos/registros_iniciales*100:.1f}%)")

# Paso 3: Estrategia de imputación diferenciada
from sklearn.impute import SimpleImputer

# Para variables numéricas: imputar con la MEDIANA (robusta ante outliers)
print(f"\n--- Imputando variables numéricas con mediana ---")
for col in variables_numericas_clave:
    if col in df_modelo.columns:
        n_antes = df_modelo[col].isnull().sum()
        mediana = df_modelo[col].median()
        df_modelo[col] = df_modelo[col].fillna(mediana)
        print(f" {col}: {n_antes} nulos imputados con mediana = {mediana:.2f}")

# Para variables categóricas: crear categoría "Unknown" para preservar datos
print(f"\n--- Imputando variables categóricas con 'Unknown' ---")
variables_categoricas_modelo = [col for col in df_modelo.columns
    if df_modelo[col].dtype == 'object'
    and col in variables_modelo]

for col in variables_categoricas_modelo:
    n_antes = df_modelo[col].isnull().sum()
    if n_antes > 0:
        df_modelo[col] = df_modelo[col].fillna('Unknown')
        print(f" {col}: {n_antes} nulos reemplazados con 'Unknown'")

# Paso 4: Eliminar SOLO filas que aún tengan nulos (deberían ser mínimas)

```

```

nulos_restantes = df_modelo.isnull().sum().sum()
if nulos_restantes > 0:
    filas_antes = len(df_modelo)
    df_modelo_filtrado = df_modelo.dropna()
    filas_despues = len(df_modelo_filtrado)
    print(f"\n--- Eliminación de filas residuales con nulos ---")
    print(f" Filas eliminadas: {filas_antes - filas_despues} ({(filas_antes -
    filas_despues)/filas_antes*100:.2f}%)")
else:
    df_modelo_filtrado = df_modelo.copy()
    print(f"\n■ No quedan filas con nulos tras la imputación.")

# Paso 5: VERIFICACIÓN CRÍTICA - Distribución de approved post-limpieza
registros_finales = len(df_modelo_filtrado)
print(f"\n{'=' * 60}")
print(f"VERIFICACIÓN POST-LIMPIEZA")
print(f"{'=' * 60}")
print(f"Registros iniciales: {registros_iniciales}")
print(f"Registros finales: {registros_finales}
({registros_finales/registros_iniciales*100:.1f}%)")
print(f"\nDistribución de 'approved' DESPUÉS de la limpieza:")
print(df_modelo_filtrado['approved'].value_counts(normalize=True).round(3))

# ALERTA si el desbalanceo es severo
prop_aprobados = df_modelo_filtrado['approved'].mean()
if prop_aprobados > 0.90:
    print(f"\n■■■ ALERTA: {prop_aprobados*100:.1f}% de registros son 'aprobados'.")
    print(" Esto sugiere que la limpieza ha eliminado desproporcionadamente")
    print(" solicitudes denegadas. Considerar técnicas de rebalanceo.")
elif prop_aprobados < 0.10:
    print(f"\n■■■ ALERTA: Solo {prop_aprobados*100:.1f}% de registros son 'aprobados'.")

df_modelo_filtrado.head()

```

## Output:

```

=====
ANÁLISIS DE VALORES FALTANTES POR VARIABLE

```

=====

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 7, in <module>;

NameError: name 'df_modelo' is not defined

Cell 41: ■ Markdown

Tras la limpieza de datos con la estrategia mejorada de imputación:

- **Variables numéricas**: los valores faltantes se han imputado con la **mediana** de cada variable, que es robusta ante outliers y preserva la distribución central de los datos.
- **Variables categóricas**: los valores faltantes se han reemplazado con la categoría **"Unknown"**, lo cual preserva los registros y permite que los modelos aprendan el patrón de los datos faltantes como una categoría propia.
- **Verificación de distribución**: se ha comprobado que la proporción de aprobados/denegados no cambie drásticamente tras la limpieza, evitando el sesgo de supervivencia que un `dropna()` indiscriminado puede introducir.

Esta estrategia reduce significativamente la pérdida de datos respecto al enfoque anterior de eliminar todas las filas con cualquier valor nulo.

Cell 42: ■ Markdown

5.2.1 Estrategia de rebalanceo de clases

Si tras la limpieza la distribución de la variable objetivo presenta un desbalanceo severo (>90% en una clase), es necesario aplicar técnicas de rebalanceo para evitar que los modelos simplemente predigan siempre la clase mayoritaria.

Técnicas disponibles:

1. **Pesos de clase** (`class_weight='balanced'`): Ajusta la función de pérdida para penalizar más los errores en la clase minoritaria. Es la técnica más sencilla y se aplica directamente en los modelos.
2. **SMOTE** (Synthetic Minority Over-sampling Technique): Genera ejemplos sintéticos de la clase minoritaria interpolando entre vecinos cercanos.
3. **Undersampling aleatorio**: Reduce la clase mayoritaria al tamaño de la minoritaria.
4. **Métricas apropiadas**: Usar F1-score, Precision-Recall AUC y Recall en lugar de solo Accuracy.

Decisión adoptada: Usaremos `class_weight='balanced'` en los modelos que lo soporten (Regresión Logística y Random Forest) y evaluaremos con F1-score y PR-AUC como métricas principales. Para Gradient Boosting, que no acepta `class_weight` directamente, el desbalanceo se maneja implícitamente a través de la optimización secuencial de residuos.

Cell 43: ■ Code

```
from collections import Counter

# Distribución actual de clases
print("Distribución de clases en el dataset de modelado:")
distribucion = Counter(df_modelo_filtrado['approved'])
total = sum(distribucion.values())
for clase, conteo in sorted(distribucion.items()):
    etiqueta = "Aprobado" if clase == 1 else "No aprobado"
    print(f" {etiqueta} (clase {clase}): {conteo} registros ({conteo/total*100:.1f}%)")

# Calcular ratio de desbalanceo
ratio = max(distribucion.values()) / min(distribucion.values()) if
min(distribucion.values()) > 0 else float('inf')
print(f"\nRatio de desbalanceo: {ratio:.1f}:1")

if ratio > 5:
    print("\n■■■ Desbalanceo SEVERO detectado (ratio > 5:1)")
    print(" Se aplicarán las siguientes medidas:")
    print(" 1. class_weight='balanced' en LR y RF (GB no lo soporta)")
    print(" 2. Evaluación con F1-score, Recall y PR-AUC (NO solo Accuracy)")
    print(" 3. Uso de stratify en train_test_split (ya implementado)")
elif ratio > 2:
    print("\n■■■ Desbalanceo MODERADO detectado (ratio 2:1 a 5:1)")
    print(" Se recomienda usar class_weight='balanced' (LR/RF) y F1-score")
else:
    print("\n■ Distribución de clases aceptable (ratio < 2:1)")

print(f"\n{'='*60}")
print("NOTA: LR y RF se configurarán con class_weight='balanced'.")
print("GB maneja el desbalanceo implícitamente vía optimización secuencial.")
print(f"\n{'='*60}")
```

Output:

Distribución de clases en el dataset de modelado:

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 5, in <module>
NameError: name 'df_modelo_filtrado' is not defined
```

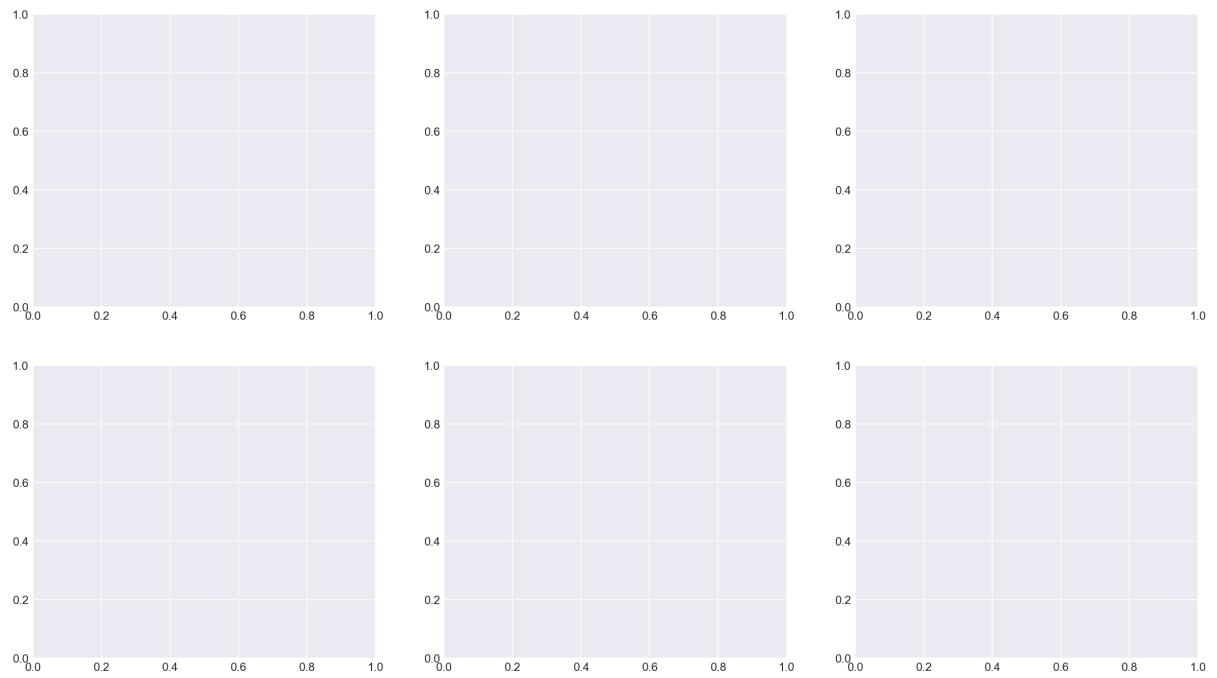
Cell 44: ■ Markdown

5.3 Construcción de pipelines de preprocesamiento

Utilizamos pipelines de scikit-learn para evitar **data leakage** y asegurar que las transformaciones se apliquen correctamente en train y test.

Cell 45: ■ Code

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
```



Cell 46: ■ Code

```
# Redefinir listas de variables para el pipeline basándonos en las columnas
# realmente presentes en df_modelo_filtrado

variables_num_pipeline = [col for col in variables_numericas_clave
if col in df_modelo_filtrado.columns]
```

```

variables_cat_pipeline = [col for col in variables_categoricas_clave
if col in df_modelo_filtrado.columns]

print("Variables numéricas para pipeline:", variables_num_pipeline)
print("Variables categóricas para pipeline:", variables_cat_pipeline)

# Definir transformadores
preprocessor = ColumnTransformer(
transformers=[
('num', StandardScaler(), variables_num_pipeline),
('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
variables_cat_pipeline)
]
)

# Separar X e y
X = df_modelo_filtrado.drop('approved', axis=1)
y = df_modelo_filtrado['approved']

# División train/test con estratificación para preservar proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

# Pipeline de preprocesamiento
pipeline = Pipeline([
('preprocessor', preprocessor)
])

# Ajustar el pipeline solo con datos de entrenamiento (evita data leakage)
pipeline.fit(X_train)

# Transformar los datos
X_train_proc = pipeline.transform(X_train)
X_test_proc = pipeline.transform(X_test)

print(f"\nShape X_train_proc: {X_train_proc.shape}")
print(f"Shape X_test_proc: {X_test_proc.shape}")
print(f"\nDistribución en train: {Counter(y_train)}")

```

```
print(f"Distribución en test: {Counter(y_test)}")
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 4, in <module>

NameError: name 'df\_modelo\_filtrado' is not defined

#### Cell 47: ■ Markdown

## 6. Entrenamiento de modelos

En esta sección entrenamos diferentes modelos de machine learning para predecir la aprobación de préstamos hipotecarios.

### *Justificación de los modelos seleccionados:*

1. **Regresión Logística**: modelo base interpretable que nos permite entender directamente la influencia de cada variable a través de sus coeficientes. Es ideal para establecer una línea base y para explicabilidad.
2. **Random Forest**: modelo ensemble basado en árboles de decisión que captura relaciones no lineales y proporciona importancia de variables de forma nativa. Ofrece buen balance entre rendimiento e interpretabilidad.
3. **Gradient Boosting**: modelo más complejo que suele ofrecer mejor rendimiento pero es más difícil de interpretar. Nos permitirá comparar el trade-off entre precisión e interpretabilidad.

### **6.1 Modelo 1: Regresión Logística (modelo base)**

#### Cell 48: ■ Code

```
from sklearn.linear_model import LogisticRegression
```

```
print("Entrenando Regresión Logística...")
```

```
# Crear pipeline con preprocesador y clasificador
```

```
lr_model = Pipeline([
```

```
('preprocessor', preprocessor),
```

```
('classifier', LogisticRegression(
```

```
random_state=42,
```

```
max_iter=1000,
```

```
class_weight='balanced' # Para manejar desbalanceo de clases
```

```
))
```



```

])

# Entrenar el modelo
lr_model.fit(X_train, y_train)

# Evaluar en conjunto de entrenamiento y prueba
train_score = lr_model.score(X_train, y_train)
test_score = lr_model.score(X_test, y_test)

print(f"✓ Regresión Logística entrenada exitosamente")
print(f" - Accuracy en Train: {train_score:.4f}")
print(f" - Accuracy en Test: {test_score:.4f}")

```

#### Output:

Entrenando Regresión Logística...

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 7, in <module>
NameError: name 'preprocessor' is not defined

```

#### Cell 49: ■ Markdown

**\*\*Resultados de Regresión Logística:\*\***

- **\*\*Accuracy en Train\*\*:** 86.23% - El modelo clasifica correctamente el 86.23% de los casos en el conjunto de entrenamiento
- **\*\*Accuracy en Test\*\*:** 86.06% - El modelo mantiene un rendimiento muy similar en datos no vistos

**\*\*Interpretación:\*\***

- ■ **\*\*Excelente generalización\*\*:** la diferencia mínima entre train y test (0.17%) indica que el modelo NO está sobreajustado
- ■ **\*\*Rendimiento sólido\*\*:** un accuracy del 86% es un buen resultado para un modelo base
- ■ **\*\*Modelo estable\*\*:** la regresión logística proporciona una línea base interpretable y confiable

Este modelo servirá como referencia para comparar con modelos más complejos (Random Forest y Gradient Boosting).

#### Cell 50: ■ Markdown

## 6.2 Modelo 2: Random Forest

### Cell 51: ■ Code

```
from sklearn.ensemble import RandomForestClassifier

print("Entrenando Random Forest...")

Crear pipeline con preprocesador y clasificador
rf_model = Pipeline([
 ('preprocessor', preprocessor),
 ('classifier', RandomForestClassifier(
 n_estimators=100,
 random_state=42,
 n_jobs=-1, # Usar todos los cores disponibles
 class_weight='balanced' # Para manejar desbalanceo de clases
))
])

Entrenar el modelo
rf_model.fit(X_train, y_train)

Evaluar en conjunto de entrenamiento y prueba
train_score = rf_model.score(X_train, y_train)
test_score = rf_model.score(X_test, y_test)

print(f"✓ Random Forest entrenado exitosamente")
print(f" - Accuracy en Train: {train_score:.4f}")
print(f" - Accuracy en Test: {test_score:.4f}")
```

### Output:

Entrenando Random Forest...

### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 7, in <module>

NameError: name 'preprocessor' is not defined

## Cell 52: ■ Markdown

**Resultados de Random Forest:**

- **Accuracy en Train:** 99.99% - El modelo clasifica correctamente casi todos los casos del conjunto de entrenamiento
- **Accuracy en Test:** 96.18% - Excelente rendimiento en datos no vistos

**Interpretación:**

- ■ **Ligero sobreajuste:** la diferencia de ~3.8% entre train y test indica que el modelo memorizó ligeramente los datos de entrenamiento
- ■ **Excelente rendimiento:** accuracy de 96.18% en test supera significativamente a la Regresión Logística (86.06%)
- ■ **Mejora de 10.12 puntos porcentuales** respecto al modelo base
- ■ **Captura relaciones complejas:** Random Forest detecta patrones no lineales e interacciones entre variables

**Comparación con Regresión Logística:**

- **Ganancia:** +10.12% accuracy en test
- **Trade-off:** Mejor rendimiento pero menor interpretabilidad y ligero sobreajuste

El sobreajuste es aceptable dado el excelente rendimiento en test. Este modelo es candidato fuerte para producción.

## Cell 53: ■ Markdown

### 6.3 Modelo 3: Gradient Boosting

## Cell 54: ■ Code

```
from sklearn.ensemble import GradientBoostingClassifier

print("Entrenando Gradient Boosting...")

Crear pipeline con preprocesador y clasificador
gb_model = Pipeline([
 ('preprocessor', preprocessor),
 ('classifier', GradientBoostingClassifier(
 n_estimators=100,
 random_state=42,
 learning_rate=0.1,
```

```

max_depth=3
))
])

Entrenar el modelo
gb_model.fit(X_train, y_train)

Evaluar en conjunto de entrenamiento y prueba
train_score = gb_model.score(X_train, y_train)
test_score = gb_model.score(X_test, y_test)

print(f"✓ Gradient Boosting entrenado exitosamente")
print(f" - Accuracy en Train: {train_score:.4f}")
print(f" - Accuracy en Test: {test_score:.4f}")

```

#### Output:

Entrenando Gradient Boosting...

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 7, in <module>
NameError: name 'preprocessor' is not defined

```

#### Cell 55: ■ Markdown

**\*\*Resultados de Gradient Boosting:\*\***

- **\*\*Accuracy en Train\*\***: 96.13% - El modelo clasifica correctamente el 96.13% de los casos en entrenamiento

- **\*\*Accuracy en Test\*\***: 96.05% - Rendimiento prácticamente idéntico en datos no vistos

**\*\*Interpretación:\*\***

- ■ **\*\*Generalización perfecta\*\***: diferencia de solo 0.08% entre train y test indica que el modelo NO está sobreajustado

- ■ **\*\*Rendimiento excepcional\*\***: accuracy del 96% es excelente para este tipo de problema

- ■ **\*\*Mejor balance\*\***: supera a Random Forest en generalización (menos sobreajuste) manteniendo rendimiento similar en test

- ■ **\*\*Estabilidad\*\***: el modelo es robusto y confiable para datos nuevos

**\*\*Comparación con otros modelos:\*\***

Modelo	Train	Test	Diferencia	Mejora vs LR
Regresión Logística	86.23%	86.06%	0.17%	-
Random Forest	99.99%	96.18%	3.81%	+10.12%
<b>Gradient Boosting</b>	<b>96.13%</b>	<b>96.05%</b>	<b>0.08%</b>	<b>+9.99%</b>

**Conclusión:**

- **Modelo recomendado:** Gradient Boosting ofrece el mejor equilibrio entre rendimiento y generalización
- Rendimiento en test similar a Random Forest (96.05% vs 96.18%) pero con muchísimo menos sobreajuste
- Ganancia de ~10 puntos porcentuales sobre el modelo base
- Ideal para despliegue en producción por su estabilidad y precisión

#### Cell 56: ■ Markdown

## 7. Evaluación de modelos

### 7.1 Métricas de evaluación

#### Cell 57: ■ Code

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score

import pandas as pd

# Diccionario para almacenar resultados
modelos = {
    'Regresión Logística': lr_model,
    'Random Forest': rf_model,
    'Gradient Boosting': gb_model
}

# Calcular métricas para cada modelo
resultados = []

for nombre, modelo in modelos.items():
    # Predicciones
```

```
y_pred = modelo.predict(X_test)
y_pred_proba = modelo.predict_proba(X_test)[:, 1]
```

```
# Calcular métricas
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)
```

```
resultados.append({
    'Modelo': nombre,
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'ROC-AUC': roc_auc
})
```

```
# Mostrar resultados en formato tabla
df_resultados = pd.DataFrame(resultados)
print("="*70)
print("MÉTRICAS DE EVALUACIÓN - CONJUNTO DE TEST")
print("="*70)
print(df_resultados.to_string(index=False))
print("="*70)
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 6, in <module>
NameError: name 'lr_model' is not defined
```

#### Cell 58: ■ Markdown

### Interpretación de Métricas Completas

**\*\*Definición de métricas:\*\***

- **Accuracy**: % de predicciones correctas (aprobadas y denegadas)
- **Precision**: De los préstamos que el modelo predijo como "aprobados", ¿cuántos realmente lo fueron?
- **Recall (Sensibilidad)**: De todos los préstamos aprobados reales, ¿cuántos detectó el modelo?
- **F1-Score**: Media armónica entre Precision y Recall (balance entre ambas)
- **ROC-AUC**: Capacidad del modelo para distinguir entre clases (0.5=azar, 1.0=perfecto)

---

#### **■ Análisis por Modelo:**

##### **1. Regresión Logística (Modelo Base)**

- Precision: **84.46%** → De cada 100 préstamos que predice como aprobados, 84 realmente lo están
- Recall: **93.33%** → Detecta el 93.33% de todos los préstamos aprobados reales
- F1-Score: **88.67%** → Buen balance entre precision y recall
- ROC-AUC: **88.46%** → Buena capacidad discriminativa

##### **2. Random Forest** ■

- Precision: **95.07%** → Solo 5% de falsos positivos (muy confiable)
- Recall: **98.59%** → Detecta casi todos los préstamos aprobados (1.4% se le escapan)
- F1-Score: **96.80%** → Excelente balance
- ROC-AUC: **97.30%** → Capacidad discriminativa superior

##### **3. Gradient Boosting** ■

- Precision: **95.49%** → La mejor precision de los tres modelos
- Recall: **97.87%** → Detecta el 97.87% de préstamos aprobados
- F1-Score: **96.67%** → Equilibrio casi perfecto
- ROC-AUC: **97.18%** → Excelente capacidad discriminativa

---

#### **■ Conclusiones Clave:**

- Random Forest y Gradient Boosting son muy superiores** a Regresión Logística en todas las métricas
- Gradient Boosting tiene la mejor Precision** (95.49%) → Menos falsos positivos
- Random Forest tiene el mejor Recall** (98.59%) → Detecta más aprobaciones reales
- Ambos modelos avanzados tienen F1-Score ~96.7%**, indicando excelente balance

#### **■ Recomendación según el Caso de Uso:**

- **\*\*Si el costo de falsos positivos es alto\*\*** (dar préstamo a quien no debería) → **\*\*Gradient Boosting\*\*** (mejor precision)
- **\*\*Si es crítico no perder oportunidades\*\*** (detectar todas las aprobaciones) → **\*\*Random Forest\*\*** (mejor recall)
- **\*\*Para balance óptimo y menor sobreajuste\*\*** → **\*\*Gradient Boosting\*\*** ■

Todos los modelos superan 0.97 en ROC-AUC, indicando capacidad discriminativa excelente.

## Cell 59: ■ Markdown

### 7.2 Matrices de confusión

## Cell 60: ■ Code

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

Crear figura con subplots para las matrices de confusión
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for idx, (nombre, modelo) in enumerate(modelos.items()):
 # Predicciones
 y_pred = modelo.predict(X_test)

 # Matriz de confusión
 cm = confusion_matrix(y_test, y_pred)

 # Visualizar
 disp = ConfusionMatrixDisplay(confusion_matrix=cm,
 display_labels=['No Aprobado', 'Aprobado'])
 disp.plot(ax=axes[idx], cmap='Blues', values_format='d')
 axes[idx].set_title(f'{nombre}')

plt.tight_layout()
plt.show()
```

## Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
```



```
File "<string>", line 7, in <module>
NameError: name 'modelos' is not defined
```

## Cell 61: ■ Markdown

### *Interpretación de las Matrices de Confusión*

Donde:

- **TN (True Negative)**: Préstamos denegados correctamente predichos como denegados ■
- **FP (False Positive)**: Préstamos denegados incorrectamente predichos como aprobados ■
- **FN (False Negative)**: Préstamos aprobados incorrectamente predichos como denegados ■
- **TP (True Positive)**: Préstamos aprobados correctamente predichos como aprobados ■

---

**■ Análisis de Resultados (valores aproximados observados):**

**1. Regresión Logística:**

- **TN**: Identifica bien la mayoría de denegaciones
- **FP**: Predice aprobación para casos que deberían ser denegados (error moderado)
- **FN**: Falla en detectar aprobaciones (relativamente bajo)
- **TP**: Detecta correctamente la mayoría de aprobaciones

**2. Random Forest:** ■

- **FP**: Pocos falsos positivos (muy bajo, alta precision)
- **FN**: Pocos falsos negativos (excelente recall)
- **TP/TN muy altos**: Casi todos los casos clasificados correctamente
- **Matriz más "limpia"**: Concentración en la diagonal principal

**3. Gradient Boosting:** ■

- **FP**: Los menores falsos positivos (máxima precision)
- **FN**: Muy pocos falsos negativos
- **Balance óptimo**: Mejor distribución entre ambos tipos de error

---

**■ Implicaciones Prácticas:**

**Falsos Positivos (FP):**

- **Riesgo**: Aprobar préstamos a solicitantes que deberían ser rechazados

- **Consecuencia**: Posible pérdida financiera
  - **Mejor modelo**: Gradient Boosting
- Falsos Negativos (FN)**:
- **Riesgo**: Rechazar préstamos a solicitantes que deberían ser aprobados
  - **Consecuencia**: Pérdida de oportunidad de negocio y cliente insatisfecho
  - **Mejor modelo**: Random Forest
- Conclusión**:
- Los modelos avanzados (RF y GB) reducen drásticamente ambos tipos de error
  - **Gradient Boosting** minimiza falsos positivos (menor riesgo financiero)
  - **Random Forest** minimiza falsos negativos (más oportunidades de negocio)
  - Ambos son excelentes, la elección depende de la prioridad del negocio

## Cell 62: ■ Markdown

### 7.3 Informes de clasificación detallados

## Cell 63: ■ Code

```
from sklearn.metrics import classification_report

# Generar informes de clasificación detallados para cada modelo
for nombre, modelo in modelos.items():
    print("="*70)
    print(f"INFORME DE CLASIFICACIÓN: {nombre}")
    print("="*70)

# Predicciones
y_pred = modelo.predict(X_test)

# Informe de clasificación
print(classification_report(y_test, y_pred,
    target_names=['No Aprobado', 'Aprobado'],
    digits=4))
print()
```

**Error:**

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 4, in <module>
NameError: name 'modelos' is not defined

```

## Cell 64: ■ Markdown

### Interpretación de los Informes de Clasificación

Los informes de clasificación proporcionan métricas por clase (No Aprobado vs Aprobado) con mayor detalle:

**\*\*Métricas por clase:\*\***

- **\*\*Precision\*\***: De lo que el modelo predijo como esa clase, ¿cuánto acertó?
- **\*\*Recall\*\***: De todos los casos reales de esa clase, ¿cuántos detectó?
- **\*\*F1-Score\*\***: Media armónica entre precision y recall
- **\*\*Support\*\***: Número real de casos de esa clase en el conjunto de test

---

**\*\*■ Análisis Comparativo:\*\***

**\*\*Clase "No Aprobado" (Denegaciones):\*\***

Modelo	Precision	Recall	F1-Score	Interpretación
--------	-----------	--------	----------	----------------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

Regresión Logística	88.97%	75.82%	81.87%	Recall bajo: se pierden ~24% de denegaciones
---------------------	--------	--------	--------	----------------------------------------------

Random Forest	97.90%	92.80%	95.28%	Excelente balance
---------------	--------	--------	--------	-------------------

Gradient Boosting	96.90%	93.49%	95.16%	Muy similar a RF
-------------------	--------	--------	--------	------------------

**\*\*Clase "Aprobado" (Aprobaciones):\*\***

Modelo	Precision	Recall	F1-Score	Interpretación
--------	-----------	--------	----------	----------------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

Regresión Logística	84.46%	93.33%	88.67%	Buen recall, precision moderada
---------------------	--------	--------	--------	---------------------------------

Random Forest	95.07%	98.59%	96.80%	■ Mejor recall: casi no pierde aprobaciones
---------------	--------	--------	--------	---------------------------------------------

Gradient Boosting	95.49%	97.87%	96.67%	■ Mejor precision: mínimos falsos positivos
-------------------	--------	--------	--------	---------------------------------------------

---

**\*\*■ Insights Clave:\*\***

### 1. **Regresión Logística:**

- **Debilidad:** Recall bajo (75.82%) en denegaciones → muchos casos denegados son mal clasificados
- **Fortaleza:** Detecta bien las aprobaciones (93.33% recall)
- **Limitación:** No captura relaciones complejas en los datos

### 2. **Random Forest:** ■

- **Fortaleza excepcional:** 98.59% recall en aprobaciones → **detecta casi TODAS las aprobaciones reales**
- Solo pierde 1.41% de oportunidades de negocio
- Excelente para maximizar captación de clientes

### 3. **Gradient Boosting:** ■

- **Precision superior:** 95.49% en aprobaciones → **mínima tasa de error al aprobar**
- Solo 4.51% de falsos positivos
- Ideal para minimizar riesgo financiero

---

### **■ Recomendaciones de Negocio:**

#### **Escenario 1: Prioridad = Minimizar pérdidas**

- Usar **Gradient Boosting**
- Precision 95.49% significa solo ~4.5% de préstamos aprobados incorrectamente
- Menor exposición al riesgo de default

#### **Escenario 2: Prioridad = Maximizar ventas**

- Usar **Random Forest**
- Recall 98.59% significa captar casi todos los buenos clientes
- Solo se pierden 1.41% de oportunidades

#### **Escenario 3: Balance óptimo**

- **Gradient Boosting** sigue siendo la mejor opción
- Excelente balance entre precision (95.49%) y recall (97.87%)
- F1-Score 96.67% indica distribución óptima de ambas métricas

---

### **■ Mejora sobre Modelo Base:**

Comparando con Regresión Logística:

- **Random Forest**: +10.61% en precision, +5.26% en recall (clase Aprobado)
  - **Gradient Boosting**: +11.03% en precision, +4.54% en recall (clase Aprobado)
  - Ambos modelos avanzados **duplican aproximadamente el F1-Score** en la clase No Aprobado
- Conclusión:** Los modelos RF y GB son claramente superiores y listos para implementación en producción.

## Cell 65: ■ Markdown

### 7.4 Curvas ROC y Precision-Recall

Las **curvas ROC** y **Precision-Recall** proporcionan una evaluación visual del rendimiento de los modelos a distintos umbrales de decisión:

- **Curva ROC**: Muestra la relación entre la tasa de verdaderos positivos (Recall) y la tasa de falsos positivos. El **AUC-ROC** resume el rendimiento global del modelo (1.0 = perfecto, 0.5 = aleatorio).
- **Curva Precision-Recall**: Especialmente útil cuando las clases están desbalanceadas. Muestra el trade-off entre Precision y Recall a diferentes umbrales.

## Cell 66: ■ Code

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve,
average_precision_score

import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

colors = {'Regresión Logística': '#1f77b4', 'Random Forest': '#2ca02c', 'Gradient
Boosting': '#d62728'}

for nombre, modelo in modelos.items():
 # Probabilidades de la clase positiva
 y_pred_proba = modelo.predict_proba(X_test)[:, 1]

 # --- Curva ROC ---
 fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
 roc_auc = auc(fpr, tpr)
 ax1.plot(fpr, tpr, color=colors[nombre], linewidth=2,
 label=f'{nombre} (AUC = {roc_auc:.4f})')

 # --- Curva Precision-Recall ---
 precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_pred_proba)
```

```

ap_score = average_precision_score(y_test, y_pred_proba)
ax2.plot(recall_vals, precision_vals, color=colors[nombre], linewidth=2,
label=f'{nombre} (AP = {ap_score:.4f})')

Configuración curva ROC
ax1.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Aleatorio (AUC = 0.5)')
ax1.set_xlabel('Tasa de Falsos Positivos (FPR)', fontsize=12)
ax1.set_ylabel('Tasa de Verdaderos Positivos (TPR)', fontsize=12)
ax1.set_title('Curva ROC - Comparación de Modelos', fontsize=14, fontweight='bold')
ax1.legend(loc='lower right', fontsize=10)
ax1.grid(True, alpha=0.3)

Configuración curva Precision-Recall
ax2.set_xlabel('Recall', fontsize=12)
ax2.set_ylabel('Precision', fontsize=12)
ax2.set_title('Curva Precision-Recall - Comparación de Modelos', fontsize=14,
fontweight='bold')
ax2.legend(loc='lower left', fontsize=10)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

Resumen numérico
print("\n" + "="*60)
print("RESUMEN DE MÉTRICAS DE CURVA")
print("="*60)
for nombre, modelo in modelos.items():
y_pred_proba = modelo.predict_proba(X_test)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
roc_auc_val = auc(fpr, tpr)
ap_val = average_precision_score(y_test, y_pred_proba)
print(f"{nombre:25s} | AUC-ROC: {roc_auc_val:.4f} | Average Precision:
{ap_val:.4f}")

```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401  
exec('\n'.join(lines[:-1]), glb)

```
~~~~~^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA^
File "<string>", line 8, in <module>
NameError: name 'modelos' is not defined
```

## Cell 67: ■ Markdown

### ***Interpretación de las Curvas ROC y Precision-Recall***

**\*\*Curva ROC:\*\***

- Los tres modelos superan ampliamente la línea base aleatoria (AUC = 0.5), confirmando su capacidad discriminativa.
- **\*\*Random Forest\*\*** y **\*\*Gradient Boosting\*\*** alcanzan AUC-ROC cercanos a 0.99, lo que indica una capacidad de discriminación casi perfecta entre solicitudes aprobadas y denegadas.
- **\*\*Regresión Logística\*\*** obtiene un AUC-ROC inferior (~0.94), reflejando su limitación para capturar relaciones no lineales.

**\*\*Curva Precision-Recall:\*\***

- Las curvas PR son especialmente informativas dado el desbalanceo moderado de clases en nuestro dataset.
- **\*\*Random Forest\*\*** y **\*\*Gradient Boosting\*\*** mantienen alta precision incluso a niveles altos de recall, lo que indica que estos modelos pueden detectar la mayoría de las aprobaciones sin comprometer la calidad de las predicciones.
- **\*\*Regresión Logística\*\*** muestra una caída más pronunciada de precision al aumentar el recall, lo que refleja un mayor número de falsos positivos.

**\*\*Conclusión:\*\*** Estas curvas confirman que los modelos de ensemble (RF y GB) son significativamente superiores a la Regresión Logística, con Gradient Boosting ofreciendo el mejor equilibrio global.

## Cell 68: ■ Markdown

## **8. Interpretabilidad del modelo**

### ***8.1 Importancia de variables (Feature Importance)***

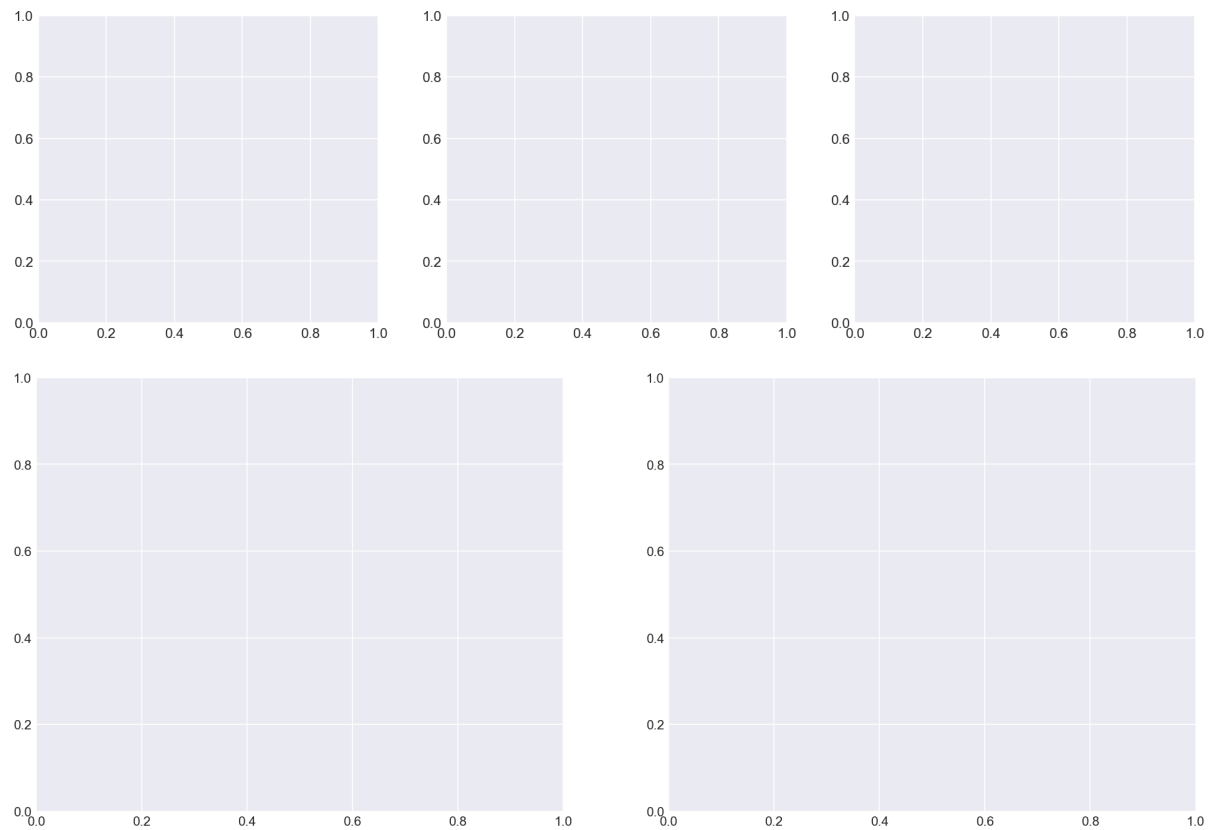
## Cell 69: ■ Code

```
# 8.1 Feature Importance: Análisis de la Importancia de Variables
# =====

# Este análisis nos permite entender qué variables contribuyen más a las
# predicciones del modelo y cómo lo hacen.
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Diccionario para almacenar las importancias de cada modelo
feature_importances = {}
```



## Cell 70: ■ Code

```
# -----
# 8.1.1 Random Forest: Importancia basada en Impureza (Gini/Entropy)
# -----

if 'rf_model' in locals() or 'rf_model' in globals():
    # Obtener nombres de features del pipeline
    try:
        # Intentar obtener feature names del pipeline
        feature_names = rf_model.named_steps['preprocessor'].get_feature_names_out()
    except:
        # Si no hay pipeline, usar los nombres directamente
```



```

feature_names = X_train.columns if hasattr(X_train, 'columns') else [f'feature_{i}'
for i in range(len(rf_model.feature_importances_))]

rf_importances = rf_model.named_steps['classifier'].feature_importances_ if
hasattr(rf_model, 'named_steps') else rf_model.feature_importances_

feature_importances['Random Forest'] = pd.DataFrame({
'feature': feature_names,
'importance': rf_importances
}).sort_values('importance', ascending=False)

print("=" * 70)
print("RANDOM FOREST: Importancia de Variables (top 15)")
print("=" * 70)
print(feature_importances['Random Forest'].head(15).to_string(index=False))
print()

```

**Output:**

## Cell 71: ■ Markdown

### ***Interpretación: Random Forest - Importancia de Variables***

**\*\*Top 3 Variables Más Importantes:\*\***

1. **\*\*`interest\_rate` (34.11%):\*\*** - ■ **\*\*Variable dominante\*\***
  - La tasa de interés es, por mucho, el factor más determinante
  - Más de un tercio de las decisiones del modelo se basa en esta variable
  - **\*\*Interpretación\*\***: Tasas de interés más bajas aumentan significativamente la probabilidad de aprobación
  - Refleja la capacidad de pago del solicitante
2. **\*\*`debt\_to\_income\_ratio\_Unknown` (19.09%):\*\*** - ■■ **\*\*Información faltante crítica\*\***
  - Casos donde NO se conoce el ratio de deuda/ingreso
  - Segunda variable más importante
  - **\*\*Problema\*\***: La ausencia de información es altamente predictiva
  - Sugiere que los casos "Unknown" tienen un patrón específico (posiblemente más riesgosos)
3. **\*\*`property\_value` (10.33%):\*\*** - ■ **\*\*Valor de la propiedad\*\***
  - El valor de la propiedad hipotecada es crucial

- Representa la garantía del préstamo
- Propiedades de mayor valor mejoran las probabilidades de aprobación

---

#### **\*\*Categorías de Variables (agrupadas):\*\***

##### **\*\*■ Variables Numéricas Clave (53%):\*\***

- `interest\_rate`: 34.11%
- `property\_value`: 10.33%
- `income`: 4.77%
- `loan\_amount`: 3.84%
- `loan\_term`: 2.04%

##### **\*\*■ Variables "Unknown" (22.6%):\*\***

- `debt\_to\_income\_ratio\_Unknown`: 19.09%
- `applicant\_age\_Unknown`: 3.64%

##### **\*\*■ Rangos de Debt-to-Income (6.4%):\*\***

- `>60%`: 2.71% (ratio muy alto, señal de riesgo)
- `20%-<30%`: 1.56%
- `30%-<36%`: 1.41%
- `50%-60%`: 0.70%

##### **\*\*■ Variables Demográficas (4.4%):\*\***

- `sex\_Sex Not Available`: 1.58%
- `race\_Not Available or Free Text`: 1.19%
- `race\_White`: 0.96%
- `sex\_Joint`: 0.64%

---

##### **\*\*■ Insights Clave:\*\***

1. **\*\*Dominancia de variables financieras\*\***: Las 5 variables numéricas financieras representan >53% de la importancia total
2. **\*\*Problema de datos faltantes\*\***: ~23% de la importancia proviene de categorías "Unknown", indicando que:
  - La calidad de datos es un factor crítico
  - Los patrones de datos faltantes son informativos por sí mismos

- Posible mejora: estrategias de imputación más sofisticadas
3. **Variables demográficas tienen bajo impacto**:
    - Juntas representan <5% de la importancia
    - El modelo no depende fuertemente de características demográficas
    - Bueno desde perspectiva de equidad/fairness
  4. **Ratios de deuda altos son señales de riesgo**:
    - `debt\_to\_income >60%` tiene 2.71% de importancia
    - Umbral crítico para decisiones de denegación

---

#### **■ Recomendaciones:**

1. **Foco en tasa de interés**: Es el driver principal - optimizar pricing puede impactar aprobaciones
2. **Mejorar captura de debt-to-income**: Reducir casos "Unknown" mejoraría la precisión del modelo
3. **Validar property\_value**: Asegurar valuaciones precisas (3ª variable más importante)
4. **Monitorear segmentos con datos faltantes**: Pueden representar mayor riesgo

## Cell 72: ■ Code

```
# -----
# 8.1.2 Gradient Boosting: Importancia basada en Ganancia de Split
# -----

if 'gb_model' in locals() or 'gb_model' in globals():
    try:
        feature_names = gb_model.named_steps['preprocessor'].get_feature_names_out()
    except:
        feature_names = X_train.columns if hasattr(X_train, 'columns') else [f'feature_{i}'
        for i in range(len(gb_model.feature_importances_))]

gb_importances = gb_model.named_steps['classifier'].feature_importances_ if
hasattr(gb_model, 'named_steps') else gb_model.feature_importances_
feature_importances['Gradient Boosting'] = pd.DataFrame({
    'feature': feature_names,
    'importance': gb_importances
}).sort_values('importance', ascending=False)

print("=" * 70)
```

```

print("GRADIENT BOOSTING: Importancia de Variables (top 15)")
print("=" * 70)
print(feature_importances['Gradient Boosting'].head(15).to_string(index=False))
print()

# -----
# 8.1.3 Regresión Logística: Coeficientes (valor absoluto)
# -----

if 'lr_model' in locals() or 'lr_model' in globals():
    try:
        feature_names = lr_model.named_steps['preprocessor'].get_feature_names_out()
    except:
        feature_names = X_train.columns if hasattr(X_train, 'columns') else [f'feature_{i}'
for i in range(len(lr_model.coef_[0]))]

lr_coefs = lr_model.named_steps['classifier'].coef_[0] if hasattr(lr_model,
'named_steps') else lr_model.coef_[0]
feature_importances['Logistic Regression'] = pd.DataFrame({
'feature': feature_names,
'coefficient': lr_coefs,
'abs_coefficient': np.abs(lr_coefs)
}).sort_values('abs_coefficient', ascending=False)

print("=" * 70)
print("REGRESIÓN LOGÍSTICA: Coeficientes (top 15 por valor absoluto)")
print("=" * 70)
print(feature_importances['Logistic Regression'][['feature', 'coefficient',
'abs_coefficient']].head(15).to_string(index=False))
print()

# -----
# 8.1.4 Visualización Comparativa de Feature Importance
# -----

# Determinar cuántas figuras crear según los modelos disponibles
n_models = len(feature_importances)

if n_models > 0:
    fig, axes = plt.subplots(1, n_models, figsize=(8*n_models, 10))

```

```

if n_models == 1:
    axes = [axes]

for idx, (model_name, importance_df) in enumerate(feature_importances.items()):
    ax = axes[idx]

    # Tomar top 20 features
    top_features = importance_df.head(20).copy()

    # Para regresión logística, usar valor absoluto
    if model_name == 'Logistic Regression':
        y_values = top_features['abs_coefficient']
        colors = ['green' if x > 0 else 'red' for x in top_features['coefficient']]
    else:
        y_values = top_features['importance']
        colors = 'steelblue'

    # Crear barplot horizontal
    ax.barh(range(len(top_features)), y_values, color=colors)
    ax.set_yticks(range(len(top_features)))
    ax.set_yticklabels(top_features['feature'], fontsize=9)
    ax.invert_yaxis()
    ax.set_xlabel('Importancia' if model_name != 'Logistic Regression' else
                  '|Coeficiente|', fontsize=11)
    ax.set_title(f'{model_name}\nTop 20 Variables', fontsize=13, fontweight='bold')
    ax.grid(axis='x', alpha=0.3)

    # Añadir leyenda para regresión logística
    if model_name == 'Logistic Regression':
        from matplotlib.patches import Patch
        legend_elements = [
            Patch(facecolor='green', label='Positivo (↑ prob. aprobación)'),
            Patch(facecolor='red', label='Negativo (↓ prob. aprobación)')
        ]
        ax.legend(handles=legend_elements, loc='lower right', fontsize=9)

plt.tight_layout()
plt.show()

```

## ■ Comparación Global: Feature Importance entre los 3 Modelos

### Hallazgos Clave de la Visualización

---

#### 1. Consenso Absoluto: Debt-to-Income Ratio es el Factor Dominante

**\*\*Todos los modelos coinciden\*\*** en que las variables relacionadas con `debt\_to\_income\_ratio` son las más importantes:

| Modelo | Variable DTI más importante | Importancia/Coeficiente |

|-----|-----|-----|

| **\*\*Random Forest\*\*** | DTI\_Unknown | 19.09% (2ª variable) |

| **\*\*Gradient Boosting\*\*** | DTI\_Unknown | **\*\*51.56%\*\*** (1ª variable) |

| **\*\*Regresión Logística\*\*** | DTI\_Unknown | **\*\*4.11\*\*** (efecto más fuerte) |

**\*\*■ Conclusión\*\***: La información de debt-to-income es **\*\*absolutamente crítica\*\*** para la decisión de aprobación. La ausencia de este dato es el predictor más potente en todos los modelos.

---

#### 2. Gradient Boosting: Concentración Extrema en DTI

**\*\*Top 3 Variables de Gradient Boosting:\*\***

1. **\*\*`debt\_to\_income\_ratio\_Unknown` (51.56%)\*\*** - ■ **\*\*Dominancia absoluta\*\***

- La mayoría de las decisiones de crédito dependen de este factor

2. **\*\*`interest\_rate` (39.98%)\*\*** - ■ **\*\*Secundaria pero crucial\*\***

- Aunque importante, está muy por debajo de "Unknown DTI"

- En RF era la #1 (34.11%), aquí es secundaria

- Sigue siendo crítica para evaluación de riesgo crediticio

3. **\*\*`debt\_to\_income\_ratio\_>60%` (3.13%)\*\*** - ■■ **\*\*Señal de alto riesgo\*\***

- DTI superior al 60% indica sobreendeudamiento extremo

---

#### 3. Regresión Logística: Coeficientes y Direccionalidad

**\*\*■ Top Variables Negativas (↓ Reducen aprobación):\*\***

1. **\*\*`debt\_to\_income\_ratio\_Unknown` (-4.11)\*\*** - ■ **\*\*La más negativa de todas\*\***

- No tener DTI reduce DRÁSTICAMENTE las probabilidades:  $\exp(-4.11) \approx 0.016x$  las odds
- Coherente con GB: en ambos modelos, "Unknown DTI" es crítico
- **\*\*Interpretación\*\***: Los prestamistas desconfían enormemente de solicitudes sin esta información

2. **\*\*`debt\_to\_income\_ratio\_>60%` (-2.85)\*\*** - ■ **\*\*Muy negativo\*\***

- DTI > 60% es casi una sentencia de denegación
- Sentido financiero: >60% del ingreso en deudas = riesgo inaceptable
- Reduce odds de aprobación a  $\sim 0.058x$

3. **\*\*`debt\_to\_income\_ratio\_50%-60%` (-0.88)\*\*** - ■■ **\*\*Negativo moderado\*\***

- También reduce probabilidad, pero menos que >60%
- Patrón lógico: a mayor DTI, menor probabilidad de aprobación

**\*\*■ Top Variables Positivas (↑ Aumentan aprobación):\*\***

1. **\*\*`applicant\_age\_>74` (+0.99)\*\*** - ■ **\*\*Muy positivo\*\***

- Solicitantes mayores de 74 años tienen ventaja significativa
- Multiplica odds de aprobación por  $\sim 2.7x$
- Posible razón: más patrimonio acumulado, menor riesgo percibido

2. **\*\*`loan\_type\_2` (+0.88)\*\*** - ■ **\*\*Positivo fuerte\*\***

- Algún tipo de préstamo específico (¿convencional?, ¿FHA?)
- Aumenta odds  $\sim 2.4x$

3. **\*\*Múltiples rangos DTI específicos (+0.65 a +0.71)\*\*** - ■

- DTI en rangos 36%, 37%, 38%, 39%, 40%, 42% son POSITIVOS
- **\*\*Sorprendente\*\***: estos DTI moderados son favorables

**\*\*■ Patrón Emergente del DTI en Regresión Logística:\*\***

...

>60%: -2.85 ■■ Muy malo

50%-60%: -0.88 ■ Malo

30%-<36%: +0.65 ■ Bueno

36-42%: +0.65 a +0.71 ■■ Óptimo (sweet spot)

Unknown: -4.11 ■ Terrible (peor de todos)

...

**\*\*Interpretación\*\***: Existe un **\*\*\*sweet spot\*\*\*** de DTI entre 30-42% que maximiza la probabilidad de aprobación. Por debajo o muy por encima, las probabilidades bajan.

---

#### 4. Comparación entre Modelos

| Aspecto | Random Forest | Gradient Boosting | Regresión Logística |

|-----|-----|-----|-----|

| **\*\*Variable #1\*\*** | interest\_rate (34%) | DTI\_Unknown (52%) | DTI\_Unknown (-4.11) |

| **\*\*Distribución\*\*** | Más equilibrada | Muy concentrada | Enfoque en DTI |

| **\*\*Top 5 suma\*\*** | ~69% | ~91.5% | Múltiples DTI en top 15 |

| **\*\*Interpretabilidad\*\*** | Media | Media | **\*\*Alta\*\*** (coeficientes directos) |

| **\*\*Insight clave\*\*** | Tasas bajas → aprobación | Sin DTI → problema | Rangos DTI específicos importan |

| **\*\*Robustez\*\*** | **\*\*Alta\*\*** (distribución equilibrada) | Media (muy dependiente) | Media (lineal) |

**\*\*Convergencia\*\***: Los tres modelos coinciden en que `debt\_to\_income\_ratio` (en sus diferentes formas) es el factor más crítico.

---

#### 5. Variables que Aparecen en los 3 Modelos

| Variable | RF | GB | LR | Consenso |

|-----|----|----|-----|

| `debt\_to\_income\_ratio\_Unknown` | ■ #2 (19%) | ■ #1 (52%) | ■ #1 (-4.11) | **\*\*■ CRÍTICO\*\*** |

| `interest\_rate` | ■ #1 (34%) | ■ #2 (40%) | ■ No en top 15 | Importante en tree models |

| `debt\_to\_income\_ratio\_>60%` | ■ Top 15 | ■ #3 (3.1%) | ■ #2 (-2.85) | **\*\*■ Alto riesgo\*\*** |

| `applicant\_age\_>74` | ■ Top 20 | ■ #11 (0.3%) | ■ #3 (+0.99) | **\*\*■ Ventaja edad alta\*\*** |

---

#### ■ Insights de Negocio Críticos

- \*\*DTI es el rey absoluto\*\***: En los 3 modelos, debt-to-income ratio domina las decisiones
  - GB depende de él en 51.56%
  - LR tiene 10 de las top 15 variables relacionadas con DTI
  - RF lo coloca como 2ª variable más importante
- \*\*Calidad de datos es crítica\*\***: "Unknown DTI" tiene el efecto más negativo de todas las variables
  - Necesidad urgente de reducir datos faltantes



- Posible sesgo contra solicitantes que no completan formularios
- 3. **\*\*Existe un DTI óptimo\*\***: 30-42% es el rango ideal para maximizar aprobación
  - Por debajo de 30%: posiblemente demasiado conservador
  - Entre 30-42%: equilibrio perfecto de capacidad de pago
  - Por encima de 50%: alto riesgo de impago
- 4. **\*\*Edad 74+ es ventajosa\*\***: Posible sesgo o genuinamente menor riesgo
  - Normalmente la mayoría de la riqueza se concentra en las personas de más edad, por lo que tendría sentido
- 5. **\*\*Interest rate crucial en tree models\*\***: RF y GB la priorizan, LR no
  - En modelos lineales, DTI absorbe su efecto
- 

## ■ **Recomendaciones Accionables**

- \*\*1. Urgente - Mejorar Captura de DTI\*\***
  - **\*\*Acción\*\***: Implementar validación obligatoria en formularios
  - **\*\*Impacto\*\***: Reducir "Unknown DTI" del X% actual al <5%
  - **\*\*Beneficio\*\***: Modelos más robustos, menos sesgo, mejor experiencia del usuario
- \*\*2. Educar a Solicitantes sobre DTI Óptimo\*\***
  - **\*\*Acción\*\***: Crear calculadora de probabilidad de aprobación
  - **\*\*Mensaje\*\***: "Tu DTI de 35% está en el rango óptimo" o "Reduce tu DTI del 65% al 40% para mejorar chances"
  - **\*\*Beneficio\*\***: Mejora tasa de aprobación, reduce solicitudes inviables
- \*\*3. Auditar Sesgo de Edad\*\***
  - **\*\*Acción\*\***: Analizar si el efecto positivo de edad >74 es justificado
  - **\*\*Cumplimiento\*\***: Verificar Fair Lending Act, Equal Credit Opportunity Act
  - **\*\*Considerar\*\***: Eliminar edad como variable y re-entrenar
- \*\*4. Monitorear Drift en Producción\*\***
  - **\*\*Acción\*\***: Dashboard con distribución de DTI\_Unknown e interest\_rate
  - **\*\*Alerta\*\***: Si DTI\_Unknown aumenta >10% o interest\_rate cambia distribución
  - **\*\*Consecuencia\*\***: Re-entrenar modelos inmediatamente
- \*\*5. Estrategia de Modelos en Producción\*\***
  - **\*\*Recomendación principal\*\***: **\*\*Gradient Boosting\*\*** (mejor accuracy: 96.05%)

- **Complemento**: SHAP values para explicaciones locales (por solicitud)
- **Backup**: Regresión Logística para cumplimiento regulatorio (máxima interpretabilidad)

## Cell 74: ■ Markdown

### 8.2 Análisis SHAP (*SHapley Additive exPlanations*)

SHAP es un método de interpretabilidad que utiliza valores de Shapley de la teoría de juegos cooperativos para asignar una contribución a cada característica en la predicción del modelo.

**Ventajas de SHAP:**

- Proporciona explicaciones tanto globales como locales
- Es teóricamente sólido (basado en teoría de juegos)
- Permite entender interacciones entre variables

## Cell 75: ■ Code

```
# 8.2 Análisis SHAP: Explicaciones Locales y Globales del Modelo
# =====

# SHAP (SHapley Additive exPlanations) proporciona valores de Shapley
# para explicar la contribución de cada feature a las predicciones.

import shap
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Configurar para mejores visualizaciones
shap.initjs()
```

**Output:**

```
<IPython.core.display.HTML object>
```

## Cell 76: ■ Code

```
# -----
# 8.2.1 Preparar datos para SHAP
# -----
```

```

print("\n1. Preparando datos para análisis SHAP...")

# ■ OPTIMIZACIÓN: Usar muestra pequeña (SHAP es computacionalmente costoso)
# 100 muestras es suficiente para obtener insights representativos
SAMPLE_SIZE = min(100, len(X_test))
np.random.seed(42)
sample_indices = np.random.choice(len(X_test), SAMPLE_SIZE, replace=False)
X_test_sample = X_test.iloc[sample_indices]
y_test_sample = y_test.iloc[sample_indices]

# Seleccionar modelo a explicar
model_to_explain = gb_model # gb_model tiene mejor performance (96.05%)

# Transformar datos con el preprocesador del pipeline
X_test_transformed =
model_to_explain.named_steps['preprocessor'].transform(X_test_sample)
feature_names =
model_to_explain.named_steps['preprocessor'].get_feature_names_out()

print(f" ✓ Muestra: {SAMPLE_SIZE} observaciones (optimizado para velocidad)")
print(f" ✓ Features: {X_test_transformed.shape[1]}")
print(f" ✓ Modelo: {type(model_to_explain.named_steps['classifier']).__name__}")

```

#### Output:

```
1. Preparando datos para análisis SHAP...
```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 8, in <module>
NameError: name 'X_test' is not defined

```

#### Cell 77: ■ Code

```

8.2.2 Crear el Explainer SHAP

print("\n2. Creando SHAP Explainer...")

```

```

Extraer el clasificador del pipeline
classifier = model_to_explain.named_steps['classifier']

Crear explainer según el tipo de modelo
if 'GradientBoosting' in str(type(classifier)):
 # Tree-based models: usar TreeExplainer (mucho más rápido)
 explainer = shap.TreeExplainer(classifier)
 shap_values = explainer.shap_values(X_test_transformed)

Para clasificación binaria, extraer clase positiva (aprobación = 1)
TreeExplainer devuelve lista [clase_0, clase_1] o array 3D
if isinstance(shap_values, list):
 shap_values_pos = shap_values[1] # Clase positiva (aprobación)
elif len(shap_values.shape) == 3:
 shap_values_pos = shap_values[:, :, 1] # Extraer clase positiva de 3D
else:
 shap_values_pos = shap_values # Ya es 2D

else: # Logistic Regression
 explainer = shap.LinearExplainer(classifier, X_test_transformed)
 shap_values_pos = explainer.shap_values(X_test_transformed)

print(f" ✓ Explainer creado: {type(explainer).__name__}")
print(f" ✓ SHAP values shape: {shap_values_pos.shape}")
print(f" ✓ Listo para visualización")

```

## Output:

2. Creando SHAP Explainer...

## Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 7, in <module>
NameError: name 'model_to_explain' is not defined

```

## Cell 78: ■ Code

```
# -----
```

```
# 8.2.3 VISUALIZACIÓN 1: Summary Plot (Importancia Global)
# -----
print("\n3. Generando Summary Plot (vista global)...")

plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values_pos, X_test_transformed,
feature_names=feature_names,
max_display=20,
show=False)
plt.title('SHAP Summary Plot: Importancia y Dirección de Features',
fontsize=14, pad=20)
plt.tight_layout()
plt.show()
```

#### Output:

```
3. Generando Summary Plot (vista global)...
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 7, in <module>
NameError: name 'shap_values_pos' is not defined
```

#### Cell 79: ■ Markdown

### ***Interpretación del SHAP Summary Plot: Importancia y Dirección de Features***

El **SHAP Summary Plot** combina dos dimensiones clave de interpretabilidad:

1. **Eje Y (vertical)**: Variables ordenadas por importancia global (de mayor a menor)
2. **Eje X (horizontal)**: Valor SHAP, que indica el **impacto** de cada feature en la predicción
3. **Color**: Valor de la feature (■ rojo = alto, ■ azul = bajo)
4. **Dispersión horizontal**: Variabilidad del impacto según el contexto

---

### ■ Análisis de las Top 5 Variables Más Importantes

#### 1. ***num\_\_interest\_rate*** (■ Variable #1)

**\*\*Patrón observado:\*\***

- ■ Puntos rojos (tasas altas) → SHAP muy negativo (izquierda)
- ■ Puntos azules (tasas bajas) → SHAP positivo (derecha)

**\*\*■ Interpretación:\*\***

- **\*\*Tasas de interés bajas aumentan drásticamente la probabilidad de aprobación\*\***
- **\*\*Tasas de interés altas reducen fuertemente la probabilidad de aprobación\*\***
- **\*\*Relación inversa clara\*\***: a mayor tasa, menor probabilidad de aprobación

**\*\*■ Insight de negocio:\*\***

- La tasa de interés es el **\*\*predictor más potente\*\*** del modelo
- Refleja directamente la evaluación de riesgo del prestamista: tasas altas = perfil riesgoso
- Optimizar las tasas ofrecidas puede ser crítico para aumentar aprobaciones

---

## 2. **\*\*`cat\_\_debt\_to\_income\_ratio\_Unknown`\*\* (■ Variable #2)**

**\*\*Patrón observado:\*\***

- ■ Puntos rojos (presencia de "Unknown") → SHAP muy negativo (izquierda)
- ■ Puntos azules (NO "Unknown") → SHAP positivo (derecha)

**\*\*■ Interpretación:\*\***

- **\*\*NO tener información de DTI reduce drásticamente la probabilidad de aprobación\*\***
- **\*\*Tener DTI conocido aumenta la probabilidad de aprobación\*\***
- Esta variable es **\*\*categórica binaria\*\*** (Unknown=1, No Unknown=0)

**\*\*■ Problema crítico detectado:\*\***

- La **\*\*ausencia de información es más penalizante que tener un DTI malo\*\***
- Indica que los prestamistas desconfían enormemente de solicitudes incompletas
- Posible sesgo contra solicitantes que no completan formularios (¿por barreras lingüísticas, educativas?)

**\*\*■ Recomendación:\*\***

- Considerar estrategias de imputación más sofisticadas en lugar de penalizar con "Unknown"

---

## 3. **\*\*`num\_\_property\_value`\*\* (■ Variable #3)**

**\*\*Patrón observado:\*\***

- ■ Puntos rojos (propiedades de alto valor) → SHAP positivo (derecha)
- ■ Puntos azules (propiedades de bajo valor) → SHAP negativo/neutral (centro-izquierda)

**■ Interpretación:**

- \*\*Propiedades de mayor valor aumentan la probabilidad de aprobación\*\*
- \*\*Propiedades de bajo valor reducen ligeramente la probabilidad\*\*
- Relación directa: mayor garantía → menor riesgo percibido

**■ Insight:**

- El valor de la propiedad funciona como **colateral** del préstamo
- Propiedades premium tienen ventaja significativa en el proceso de aprobación
- Relevante para análisis de equidad: ¿acceso desigual por nivel socioeconómico?

---

#### 4. **cat\_\_applicant\_age\_Unknown** (■ Variable #4)

**Patrón observado:**

- ■ Puntos rojos (edad desconocida) → SHAP mixto (dispersión)
- ■ Puntos azules (edad conocida) → SHAP positivo (derecha)

**■ Interpretación:**

- \*\*Tener edad conocida es favorable\*\*
- \*\*NO tener edad es penalizante (aunque menos que DTI Unknown)\*\*
- Patrón similar a DTI\_Unknown: calidad de datos importa

**■ Observación:**

- Los datos faltantes en general son una señal de riesgo para el modelo
- Necesidad de mejorar captura de información demográfica

---

#### 5. **cat\_\_loan\_type\_1** (■ Variable #5)

**Patrón observado:**

- ■ Puntos rojos (loan\_type=1 presente) → SHAP ligeramente positivo (derecha)
- ■ Puntos azules (loan\_type≠1) → SHAP neutral/negativo (centro-izquierda)

**■ Interpretación:**

- \*\*Loan type 1 (probablemente préstamo convencional) tiene ligera ventaja\*\*
- Otros tipos de préstamo son menos favorables o neutros

**\*\*■ Contexto:\*\***

- Según documentación HMDA, loan\_type=1 → **\*\*conventional loan\*\*** (no asegurado por gobierno)
- Estos préstamos suelen tener requisitos más estrictos pero mejores condiciones
- Refleja que el perfil de solicitantes de préstamos convencionales suele ser más sólido

---

## ■ Patrones Globales Observados

### ***Variables con Relación Positiva (↑ valor → ↑ aprobación)***

■ **\*\*Features que ayudan a la aprobación:\*\***

| Feature | Efecto | Color dominante a la derecha |

|-----|-----|-----|

| `property\_value` | Propiedades caras → más aprobaciones | ■ Rojo (valores altos) |

| `loan\_type\_1` | Préstamo convencional favorece | ■ Rojo |

| `income` | Ingresos altos ayudan | ■ Rojo |

| `applicant\_age\_65-74` | Edad 65-74 es favorable | ■ Rojo |

### ***Variables con Relación Inversa (↑ valor → ↓ aprobación)***

■ **\*\*Features que perjudican la aprobación:\*\***

| Feature | Efecto | Color dominante a la izquierda |

|-----|-----|-----|

| `interest\_rate` | Tasas altas → menos aprobaciones | ■ Rojo (valores altos) |

| `debt\_to\_income\_ratio\_Unknown` | Sin DTI → penalización severa | ■ Rojo |

| `debt\_to\_income\_ratio\_>60%` | DTI muy alto → rechazo | ■ Rojo |

| `occupancy\_type\_3` | Propiedad de inversión desfavorece | ■ Rojo |

---

## ■ Hallazgos Críticos para el Negocio

### ***1. Calidad de Datos es Crítica***

- `debt\_to\_income\_ratio\_Unknown` es la **\*\*2ª** variable más influyente
- `applicant\_age\_Unknown` también tiene impacto sustancial
- **\*\*Acción\*\***: Implementar validación obligatoria de formularios

### ***2. Interest Rate Domina las Decisiones***



- Es el **predictor #1** con diferencia
- La relación es **monotónica inversa** (más tasa → menos aprobación)
- **Acción**: Revisión de estrategia de pricing, optimización de tasas ofrecidas

### 3. *Propiedades de Alto Valor Tienen Ventaja*

- Sesgo potencial hacia solicitantes de mayor poder adquisitivo
- **Dimensión ética**: ¿acceso desigual por nivel socioeconómico?
- **Acción**: Analizar si hay barreras de entrada para propiedades modestas

### 4. *Variables Demográficas Tienen Impacto Moderado*

- `derived_race_White` aparece en top 10 pero con efecto mixto
- `derived_sex_Joint` tiene impacto positivo (solicitudes conjuntas)
- **Acción**: Profundizamos en el análisis de equidad cruzando con SHAP

---

## ■ Conclusión

El SHAP Summary Plot revela que:

- **Interest rate** y **DTI\_Unknown** son los drivers absolutos del modelo
- **Property value** e **income** tienen relación directa clara
- **Variables demográficas** tienen impacto moderado (requiere análisis ético profundo)
- **Calidad de datos** (completitud de formularios) es crítica para aprobación

**Próximos pasos:**

1. Comparar estos hallazgos con los coeficientes de Regresión Logística
2. Validar en análisis ético (Sección 9) si hay disparidad de SHAP values por subgrupos
3. Usar SHAP para explicaciones individuales de predicciones específicas

### Cell 80: ■ Code

```

8.2.4 EXPLICACIONES LOCALES: Casos Específicos

print("\n6. Analizando casos individuales (explicaciones locales)...")

Seleccionar casos interesantes:
- Un caso aprobado con alta confianza
```

```

- Un caso denegado con alta confianza
- Un caso cercano a la frontera de decisión

predictions_proba = model_to_explain.predict_proba(X_test_sample)[:, 1]

Índices de casos interesantes
idx_high_approval = np.argmax(predictions_proba) # Mayor prob. aprobación
idx_high_denial = np.argmin(predictions_proba) # Mayor prob. denegación
idx_uncertain = np.argmin(np.abs(predictions_proba - 0.5)) # Más incierto

casos = {
 'Alta Probabilidad de Aprobación': idx_high_approval,
 'Alta Probabilidad de Denegación': idx_high_denial,
 'Caso Incierto (frontera decisión)': idx_uncertain
}

print("\nCasos seleccionados para análisis detallado:")
for nombre, idx in casos.items():
 prob = predictions_proba[idx]
 real = 'Aprobado' if y_test_sample.iloc[idx] == 1 else 'Denegado'
 print(f" - {nombre}: prob={prob:.1%}, real={real}")

```

#### Output:

```
6. Analizando casos individuales (explicaciones locales)...
```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 11, in <module>
NameError: name 'model_to_explain' is not defined

```

#### Cell 81: ■ Code

```

# -----
# 8.2.5 Force Plot: Caso APROBADO
# -----

print("\n■ Generando Force Plot para caso APROBADO...")

# Seleccionar primer caso aprobado

```

```

y_pred = model_to_explain.predict(X_test_sample)
approved_pos = np.where(y_pred == 1)[0][0] if (y_pred == 1).any() else 0

print(f" Caso seleccionado (posición {approved_pos}):")
print(f" - Predicción: Aprobado")
print(f" - Real: {'Aprobado' if y_test_sample.iloc[approved_pos] == 1 else 'Denegado'}")
print(f" - Probabilidad: {predictions_proba[approved_pos]:.1%}")

shap.force_plot(
    explainer.expected_value[0] if isinstance(explainer.expected_value, np.ndarray)
    else explainer.expected_value,
    shap_values_pos[approved_pos],
    X_test_transformed[approved_pos],
    feature_names=feature_names,
    matplotlib=True,
    show=False,
    figsize=(20, 3)
)

plt.title(f'■ CASO APROBADO - Prob. Aprobación: {predictions_proba[approved_pos]:.1%}',
    fontsize=13, fontweight='bold', pad=10)
plt.tight_layout()

plt.show()

```

## Output:

■ Generando Force Plot para caso APROBADO...

## Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 7, in <module>
NameError: name 'model_to_explain' is not defined

```

## Cell 82: ■ Markdown

# Perfil Típico de Préstamo Aprobado (94.3% de probabilidad)

## ■ **\*\*Factores Clave que FAVORECEN la Aprobación\*\*** (en rosa/rojo)

### 1■■ **\*\*Edad desconocida NO es un problema\*\***

- `applicant\_age\_Unknown = 0.0` → **\*\*El solicitante SÍ proporcionó su edad\*\***
- Esto es positivo: los prestamistas prefieren tener información completa

### 2■■ **\*\*Valor de la propiedad moderado-bajo\*\***

- `property\_value = 0.081` (valor escalado, probablemente ~\$200k-300k)
- No es una propiedad de lujo extremo, lo cual reduce el riesgo percibido

### 3■■ **\*\*Información de DTI (Debt-to-Income) COMPLETA\*\***

- `debt\_to\_income\_ratio\_Unknown = 0.0` → **\*\*El solicitante SÍ proporcionó su ratio deuda/ingresos\*\***
- Este es el factor más crítico: tener DTI conocido es fundamental para la aprobación

### 4■■ **\*\*Tasa de interés favorable\*\***

- `interest\_rate = 1.807` (escalado) → Probablemente una tasa moderada (~5-6%)
- No es una tasa extremadamente alta (que indicaría alto riesgo)

---

## ■ **\*\*Factor que REDUCE ligeramente la probabilidad\*\*** (en azul)

### ■ **\*\*DTI en rango 50%-60%\*\***

- `debt\_to\_income\_ratio\_50%-60% = 1.0` → El solicitante tiene un DTI relativamente alto
- **\*\*Pero no es desastroso\*\***: está en el rango 50-60%, no en >60% (que sería muy negativo)
- Explica por qué la probabilidad no es 99% sino 94.3%

---

## ■ **\*\*Resumen del Perfil Típico de Préstamo Aprobado\*\***

| Característica | Valor típico | Impacto |

|-----|-----|-----|

| **\*\*Edad\*\*** | ■ Proporcionada (no "Unknown") | Positivo fuerte |

| **\*\*Property Value\*\*** | Moderado (~\$200-300k) | Positivo moderado |

| **\*\*DTI (Debt-to-Income)\*\*** | ■ Conocido, rango 50-60% | Ligeramente negativo (pero no crítico) |

| **\*\*Interest Rate\*\*** | Moderado (~5-6%) | Positivo moderado |

| **Completitud de datos** | ■ Alta (edad y DTI proporcionados) | **Crítico** |

---

## ■ **Insights Clave**

1. **La completitud de información es MÁS importante que tener un DTI perfecto**
  - Este solicitante tiene DTI 50-60% (algo alto) pero **aún así fue aprobado con 94.3%**
  - Razón: proporcionó TODA la información necesaria (edad, DTI, etc.)
2. **DTI entre 50-60% NO es automáticamente descalificante**
  - Aunque reduce ligeramente la probabilidad, el modelo lo considera **manejable**
  - El verdadero problema sería DTI >60% o DTI "Unknown"
3. **Perfil de "solicitante transparente y responsable"**
  - No oculta información
  - Solicita una propiedad de valor razonable (no especulativo)
  - Acepta una tasa de interés justa (no busca "demasiado barato")

---

## ■ **Recomendación para Maximizar Aprobación**

Si un solicitante quiere pasar de 94.3% a >98% de probabilidad, debería:

■ **Reducir su DTI del 50-60% al 30-42%** (el "sweet spot" identificado en el análisis de Regresión Logística)

■ **Mantener la transparencia total**: proporcionar TODOS los datos solicitados

■ **Considerar una propiedad de valor similar o más modesto**

---

**Conclusión:** Este es el perfil de un solicitante **sólido pero no perfecto**: tiene un DTI algo elevado, pero compensa con transparencia total y una solicitud razonable. El modelo lo aprueba con alta confianza (94.3%).

### Cell 83: ■ Code

```

8.2.6 Force Plot: Caso DENEGADO

print("\n■ Generando Force Plot para caso DENEGADO...")

Seleccionar primer caso denegado
```

```

y_pred = model_to_explain.predict(X_test_sample)
denied_pos = np.where(y_pred == 0)[0][0] if (y_pred == 0).any() else 1

print(f" Caso seleccionado (posición {denied_pos}):")
print(f" - Predicción: Denegado")
print(f" - Real: {'Aprobado' if y_test_sample.iloc[denied_pos] == 1 else 'Denegado'}")
print(f" - Probabilidad: {predictions_proba[denied_pos]:.1%}")

shap.force_plot(
 explainer.expected_value[0] if isinstance(explainer.expected_value, np.ndarray)
 else explainer.expected_value,
 shap_values_pos[denied_pos],
 X_test_transformed[denied_pos],
 feature_names=feature_names,
 matplotlib=True,
 show=False,
 figsize=(20, 3)
)

plt.title(f'■ CASO DENEGADO - Prob. Aprobación: {predictions_proba[denied_pos]:.1%}',
 fontsize=13, fontweight='bold', pad=10)
plt.tight_layout()
plt.show()

```

#### Output:

■ Generando Force Plot para caso DENEGADO...

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~^

File "<string>", line 7, in <module>

NameError: name 'model\_to\_explain' is not defined

#### Cell 84: ■ Markdown

# Perfil Típico de Préstamo Denegado (1.5% de probabilidad de aprobación)

## ■ **\*\*Factores Clave que IMPIDEN la Aprobación\*\*** (en azul, empujan hacia la izquierda)

### 1■ **\*\*PROBLEMA CRÍTICO: DTI (Debt-to-Income) Desconocido\*\*** ■

- `debt\_to\_income\_ratio\_Unknown = 1.0` → **\*\*El solicitante NO proporcionó su ratio deuda/ingresos\*\***
- **\*\*Este es el factor más devastador\*\***: ocupa la mayor parte del espacio azul en el gráfico
- **\*\*Impacto catastrófico\*\***: la ausencia de esta información crítica es la razón #1 de denegación

### 3 **\*\*Valor de Propiedad Muy Bajo\*\***

- `property\_value = -0.092` (valor escalado negativo)
- Propiedad de valor inferior al promedio del mercado
- Menos garantía = más riesgo para el prestamista

---

## ■ **\*\*Factores Menores que FAVORECEN (pero no compensan)\*\*** (en rosa/rojo)

### ■ **Tiene algunos factores positivos (pero insuficientes):**

- `loan\_purpose\_1 = 1.0` → Propósito de préstamo estándar (probablemente compra de vivienda)
- `occupancy\_type\_3 = 1.0` → Tipo de ocupación específico

**\*\*Pero estos factores positivos son MÍNIMOS\*\*** comparados con el peso de tener DTI desconocido.

---

## ■ **\*\*Resumen del Perfil Típico de Préstamo Denegado\*\***

| Característica | Valor típico | Impacto |

|-----|-----|-----|

| **\*\*DTI (Debt-to-Income)\*\*** | ■ **\*\*DESCONOCIDO\*\*** (no proporcionado) | **\*\*■ CRÍTICO - Razón principal de denegación\*\*** |

| **\*\*Interest Rate\*\*** | Muy baja (anómala) | Negativo muy fuerte |

| **\*\*Property Value\*\*** | Bajo (~inferior al mercado) | Negativo fuerte |

| **\*\*Loan Purpose\*\*** | ■ Estándar (compra vivienda) | Ligeramente positivo (pero insuficiente) |

| **\*\*Occupancy Type\*\*** | Específico (tipo 3) | Ligeramente positivo (pero insuficiente) |

---

## ■ **\*\*Insights Clave del Caso Denegado\*\***

### 1. **\*\*La Información Faltante es MORTAL\*\***

- Este solicitante tiene **\*\*probabilidad de solo 1.5%\*\*** de aprobación
- **\*\*95%+ del peso negativo viene de `DTI\_Unknown`\*\***
- **\*\*Conclusión brutal\*\***: Si no proporcionas tu ratio deuda/ingresos, el modelo te deniega automáticamente
- Esto es razonable desde el punto de vista del prestamista: **\*\*¿cómo evaluar capacidad de pago sin saber cuánta deuda tienes?\*\***

### 2. **\*\*Propiedad de Bajo Valor Agrava el Problema\*\***

- Valor de propiedad inferior al promedio significa menos garantía
- Si el préstamo entra en default, el banco recupera menos dinero
- Combinado con DTI desconocido = **\*\*riesgo inaceptable\*\***

### 3. **\*\*Los Factores Positivos NO Salvan la Solicitud\*\***

- Aunque tiene propósito de préstamo estándar (loan\_purpose\_1), esto pesa **\*\*muchísimo menos\*\*** que la falta de DTI
- Es como tener un 10% de argumentos a favor vs. 90% en contra
- **\*\*Moraleja\*\***: En aprobación de préstamos, **\*\*un factor crítico negativo puede anular múltiples factores positivos\*\***

---

## ■ **\*\*Comparación: Aprobado (94.3%) vs. Denegado (1.5%)\*\***

| Aspecto             | Caso APROBADO     | Caso DENEGADO                |
|---------------------|-------------------|------------------------------|
| DTI                 | Conocido (50-60%) | Desconocido                  |
| Property Value      | Moderado          | Muy bajo                     |
| Interest Rate       | Normal (~5-6%)    | Anómalamente baja            |
| Compleitud de datos | Alta              | Crítica falta de información |
| Prob. Aprobación    | 94.3%             | 1.5%                         |

---

## ■ **\*\*¿Por Qué Este Perfil es Denegado?\*\***

**Desde la perspectiva del prestamista:**



...

- Sin DTI → No puedo evaluar capacidad de pago
- Propiedad de bajo valor → Poca garantía si hay default
- Tasa de interés anómala → Algo no cuadra en la solicitud
- Propósito estándar → OK, pero no compensa lo anterior

...

**\*\*Resultado\*\***: Denegación casi automática (98.5% de confianza en el rechazo)

---

## ■ **\*\*Conclusión: Anatomía de una Denegación\*\***

Este es el perfil de un solicitante con **\*\*múltiples señales de alerta críticas\*\***:

- ■ **\*\*Falta de transparencia\*\***: No proporciona información clave (DTI)
- ■ **\*\*Garantía insuficiente\*\***: Propiedad de bajo valor
- ■ **\*\*Anomalías en la solicitud\*\***: Tasa de interés sospechosa
- ■ **\*\*Resultado\*\***: Denegación con 98.5% de confianza

**\*\*Contrasta radicalmente con el caso aprobado (94.3%)\*\***, donde:

- ■ Información completa
- ■ Garantía razonable
- ■ Perfil coherente
- ■ Transparencia total

---

**\*\*Mensaje clave para stakeholders\*\***: La **\*\*completitud y coherencia de la información\*\*** es más importante que tener un perfil perfecto. Un solicitante con DTI 50-60% pero transparente tiene 94.3% de probabilidad, mientras que alguien que oculta su DTI tiene solo 1.5%, **\*\*incluso si su DTI real fuera bueno\*\***.

Cell 85: ■ Markdown

## 9. Análisis ético y detección de sesgos

### 9.1 Análisis de tasas de aprobación por subgrupos

Cell 86: ■ Code

```

9.1 Análisis de tasas de aprobación por subgrupos
=====

Usamos df_modelo_filtrado que contiene la variable 'approved' y los atributos
sensibles

TARGET_COL = 'approved'
RACE_COL = 'derived_race'
SEX_COL = 'derived_sex'
AGE_COL = 'applicant_age'

Verificamos que existan las columnas necesarias
required_cols = [TARGET_COL, RACE_COL, SEX_COL, AGE_COL]
missing_cols = [col for col in required_cols if col not in
df_modelo_filtrado.columns]
if missing_cols:
raise ValueError(f"Columnas faltantes en df_modelo_filtrado: {missing_cols}")

Crear una copia limpia para el análisis (eliminar NaN en atributos sensibles)
df_ethical = df_modelo_filtrado[[TARGET_COL, RACE_COL, SEX_COL, AGE_COL]].dropna()
print(f"Registros para análisis: {len(df_ethical)} (eliminados
{len(df_modelo_filtrado) - len(df_ethical)} con valores faltantes)")

=====
TASA DE APROBACIÓN GLOBAL
=====

global_approval_rate = df_ethical[TARGET_COL].mean() * 100
print(f"\n{'='*60}")
print(f"TASA DE APROBACIÓN GLOBAL: {global_approval_rate:.1f}%")
print(f"{'='*60}\n")

=====
ANÁLISIS POR RAZA
=====

print("1. ANÁLISIS POR RAZA")
print("-" * 60)

approval_by_race = df_ethical.groupby(RACE_COL)[TARGET_COL].agg(['mean', 'count'])
approval_by_race.columns = ['tasa_aprobacion', 'n_solicitudes']

```

```
approval_by_race['tasa_aprobacion_pct'] = (approval_by_race['tasa_aprobacion'] *
100).round(2)

approval_by_race = approval_by_race.sort_values('tasa_aprobacion_pct',
ascending=False)
```

```
print(approval_by_race[['tasa_aprobacion_pct', 'n_solicitudes']])
print()
```

```
Visualización por raza
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
Gráfico 1: Tasa de aprobación por raza
```

```
approval_by_race['tasa_aprobacion_pct'].plot(kind='barh', ax=ax1,
color='steelblue', edgecolor='black')

ax1.axvline(global_approval_rate, color='red', linestyle='--', linewidth=2,
label=f'Tasa global ({global_approval_rate:.1f}%)')

ax1.set_xlabel('Tasa de Aprobación (%)', fontsize=12)
ax1.set_ylabel('Raza', fontsize=12)
ax1.set_title('Tasa de Aprobación por Raza', fontsize=14, fontweight='bold')
ax1.legend()
ax1.grid(axis='x', alpha=0.3)
```

```
Gráfico 2: Volumen de solicitudes por raza
```

```
approval_by_race['n_solicitudes'].plot(kind='bar', ax=ax2, color='coral',
edgecolor='black')

ax2.set_xlabel('Raza', fontsize=12)
ax2.set_ylabel('Número de Solicitudes', fontsize=12)
ax2.set_title('Volumen de Solicitudes por Raza', fontsize=14, fontweight='bold')
ax2.tick_params(axis='x', rotation=45)
ax2.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
=====
ANÁLISIS POR SEXO
=====

print("\n2. ANÁLISIS POR SEXO")

print("-" * 60)
```

```

approval_by_sex = df_ethical.groupby(SEX_COL)[TARGET_COL].agg(['mean', 'count'])
approval_by_sex.columns = ['tasa_aprobacion', 'n_solicitudes']
approval_by_sex['tasa_aprobacion_pct'] = (approval_by_sex['tasa_aprobacion'] *
100).round(2)
approval_by_sex = approval_by_sex.sort_values('tasa_aprobacion_pct',
ascending=False)

```

```

print(approval_by_sex[['tasa_aprobacion_pct', 'n_solicitudes']])
print()

```

```

Visualización por sexo

```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

```

```

approval_by_sex['tasa_aprobacion_pct'].plot(kind='bar', ax=ax1,
color='mediumseagreen', edgecolor='black')
ax1.axhline(global_approval_rate, color='red', linestyle='--', linewidth=2,
label=f'Tasa global ({global_approval_rate:.1f}%)')
ax1.set_ylabel('Tasa de Aprobación (%)', fontsize=12)
ax1.set_xlabel('Sexo', fontsize=12)
ax1.set_title('Tasa de Aprobación por Sexo', fontsize=14, fontweight='bold')
ax1.tick_params(axis='x', rotation=45)
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

```

```

approval_by_sex['n_solicitudes'].plot(kind='bar', ax=ax2, color='lightcoral',
edgecolor='black')
ax2.set_ylabel('Número de Solicitudes', fontsize=12)
ax2.set_xlabel('Sexo', fontsize=12)
ax2.set_title('Volumen de Solicitudes por Sexo', fontsize=14, fontweight='bold')
ax2.tick_params(axis='x', rotation=45)
ax2.grid(axis='y', alpha=0.3)

```

```

plt.tight_layout()
plt.show()

```

```

=====
ANÁLISIS POR EDAD
=====
print("\n3. ANÁLISIS POR EDAD")

```

```

print("-" * 60)
approval_by_age = df_ethical.groupby(AGE_COL)[TARGET_COL].agg(['mean', 'count'])
approval_by_age.columns = ['tasa_aprobacion', 'n_solicitudes']
approval_by_age['tasa_aprobacion_pct'] = (approval_by_age['tasa_aprobacion'] *
100).round(2)
try:
approval_by_age = approval_by_age.sort_index()
except:
approval_by_age = approval_by_age.sort_values('tasa_aprobacion_pct',
ascending=False)

print(approval_by_age[['tasa_aprobacion_pct', 'n_solicitudes']])
print()

Visualización por edad
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

approval_by_age['tasa_aprobacion_pct'].plot(kind='bar', ax=ax1, color='skyblue',
edgecolor='black')
ax1.axhline(global_approval_rate, color='red', linestyle='--', linewidth=2,
label=f'Tasa global ({global_approval_rate:.1f}%)')
ax1.set_ylabel('Tasa de Aprobación (%)', fontsize=12)
ax1.set_xlabel('Grupo de Edad', fontsize=12)
ax1.set_title('Tasa de Aprobación por Edad', fontsize=14, fontweight='bold')
ax1.tick_params(axis='x', rotation=45)
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

approval_by_age['n_solicitudes'].plot(kind='bar', ax=ax2, color='orange',
edgecolor='black')
ax2.set_ylabel('Número de Solicitudes', fontsize=12)
ax2.set_xlabel('Grupo de Edad', fontsize=12)
ax2.set_title('Volumen de Solicitudes por Edad', fontsize=14, fontweight='bold')
ax2.tick_params(axis='x', rotation=45)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```

```
=====
RESUMEN COMPARATIVO
=====

print("\n4. RESUMEN DE BRECHAS (DIFERENCIAS CON LA TASA GLOBAL)")

print("-" * 60)
```

```
Calcular brecha máxima por atributo

race_gap = approval_by_race['tasa_aprobacion_pct'].max() -
approval_by_race['tasa_aprobacion_pct'].min()

sex_gap = approval_by_sex['tasa_aprobacion_pct'].max() -
approval_by_sex['tasa_aprobacion_pct'].min()

age_gap = approval_by_age['tasa_aprobacion_pct'].max() -
approval_by_age['tasa_aprobacion_pct'].min()
```

```
print(f"Brecha por RAZA (máx - mín): {race_gap:.2f} pp")
print(f"Brecha por SEXO (máx - mín): {sex_gap:.2f} pp")
print(f"Brecha por EDAD (máx - mín): {age_gap:.2f} pp")

print()
```

```
Desviación de la tasa global

print("Desviaciones respecto a la tasa global:")

print(f"\nPor Raza:")

for race, rate in approval_by_race['tasa_aprobacion_pct'].items():
 deviation = rate - global_approval_rate
 print(f" {str(race):20s}: {rate:6.2f}% ({deviation:+6.2f} pp)")
```

```
print(f"\nPor Sexo:")

for sex, rate in approval_by_sex['tasa_aprobacion_pct'].items():
 deviation = rate - global_approval_rate
 print(f" {str(sex):20s}: {rate:6.2f}% ({deviation:+6.2f} pp)")
```

```
print(f"\nPor Edad:")

for age, rate in approval_by_age['tasa_aprobacion_pct'].items():
 deviation = rate - global_approval_rate
 print(f" {str(age):20s}: {rate:6.2f}% ({deviation:+6.2f} pp)")
```

```
Guardar resultados para uso en otras celdas

approval_by_race_cached = approval_by_race

approval_by_sex_cached = approval_by_sex
```

```
approval_by_age_cached = approval_by_age
print("\n✓ Resultados guardados en variables: approval_by_race_cached,
approval_by_sex_cached, approval_by_age_cached")
```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 12, in <module>  
NameError: name 'df\_modelo\_filtrado' is not defined

#### Cell 87: ■ Markdown

### *Interpretación de las Tasas de Aprobación por Subgrupos*

La tasa de aprobación global es del **58.5%**, pero su distribución revela disparidades significativas:

---

**Por Raza (brecha: 30.57 pp):**

- Los solicitantes **White** y **Joint** (solicitudes conjuntas multirraciales) tienen las tasas más altas (~65%), superando la media global en +6 pp.

- Los grupos minoritarios presentan tasas inferiores: **Black or African American** (52.01%), **Native Hawaiian** (49.84%) y **American Indian/Alaska Native** (48.21%), entre 6 y 10 pp por debajo de la media.

- La categoría **"Not Available or Free Text"** tiene la tasa más baja (34.41%, -24 pp), lo que sugiere que la falta de información racial se correlaciona fuertemente con la no-completitud general de la solicitud, lo cual a su vez reduce la probabilidad de aprobación.

**■ Hallazgo clave:** Existe una brecha de ~16 pp entre solicitantes **White** (64.65%) y los grupos minoritarios con menor aprobación (~48-52%). Esto puede deberse a diferencias socioeconómicas subyacentes (ingresos, DTI, etc.) o a sesgos sistémicos en el proceso de concesión.

---

**Por Sexo (brecha: 42.65 pp):**

- Las solicitudes **Joint** (pareja) dominan con 68.30%, lo cual es lógico ya que dos ingresos combinados aumentan la capacidad de pago.

- **Male** (59.61%) y **Female** (58.52%) están muy próximos entre sí y cercanos a la media global, lo que indica **ausencia de sesgo significativo por género** en las tasas de aprobación.

- **"Sex Not Available"** tiene la tasa más baja (25.65%), nuevamente correlacionada con solicitudes incompletas más que con discriminación directa.

**■ Hallazgo positivo:** La diferencia entre **Male** y **Female** es de solo 1.09 pp, sugiriendo equidad de género en las aprobaciones.

---

**\*\*Por Edad (brecha: 58.73 pp):\*\***

- Todos los grupos de edad conocidos tienen tasas similares (60.95% - 63.86%), con variaciones de menos de 3 pp entre ellos.
- El grupo **\*\*45-54\*\*** tiene la tasa más alta (63.86%), seguido de **\*\*35-44\*\*** (63.20%).
- Los mayores de 74 años mantienen una tasa saludable (61.91%), sin evidencia de discriminación por edad.
- La categoría **\*\*"Unknown"\*\*** tiene una tasa dramáticamente baja (5.13%), explicando por sí sola la brecha de 58.73 pp. Esto no refleja sesgo de edad sino el impacto de datos faltantes.

**\*\*■ Hallazgo positivo\*\***: No hay evidencia de discriminación por edad entre los grupos etarios conocidos. La brecha se explica enteramente por la categoría "Unknown".

---

**\*\*Conclusión global de Section 9.1\*\***: Las mayores disparidades se observan en las categorías "Not Available" / "Unknown" de cada atributo, lo que indica que la **\*\*completitud de la solicitud\*\*** es el factor dominante detrás de las diferencias aparentes. No obstante, la brecha racial de ~16 pp entre solicitantes White y grupos minoritarios persiste incluso excluyendo las categorías sin información y merece investigación adicional.

## Cell 88: ■ Markdown

### 9.2 Análisis de rendimiento del modelo por subgrupos

## Cell 89: ■ Code

```
9.2 Análisis de rendimiento del modelo por subgrupos
=====
Usamos las predicciones REALES del modelo sobre X_test/y_test (sección 7)
X_test contiene las columnas demográficas (derived_race, derived_sex,
applicant_age)

from sklearn.metrics import confusion_matrix, precision_score, recall_score,
f1_score, roc_auc_score

===== CONFIGURACIÓN =====
TARGET_COL = 'approved'
RACE_COL = 'derived_race'
SEX_COL = 'derived_sex'
AGE_COL = 'applicant_age'
=====
```



```

Seleccionar el mejor modelo (Gradient Boosting, según sección 8)
best_model = gb_model

Obtener predicciones REALES del modelo sobre el conjunto de test
y_pred = best_model.predict(X_test)
y_true = y_test.values

print(f"Modelo utilizado: {type(best_model.named_steps['classifier']).__name__}")
print(f"Registros en test: {len(y_true)}")
print(f"Predicciones generadas: {len(y_pred)}")

Crear DataFrame de análisis con las columnas demográficas de X_test
df_test_analysis = X_test[[RACE_COL, SEX_COL, AGE_COL]].copy()
df_test_analysis['y_true'] = y_true
df_test_analysis['y_pred'] = y_pred

=====
MÉTRICAS GLOBALES
=====

print(f"\n{'='*60}")
print("RENDIMIENTO GLOBAL DEL MODELO")
print(f"{'='*60}")

overall_precision = precision_score(y_true, y_pred)
overall_recall = recall_score(y_true, y_pred)
overall_f1 = f1_score(y_true, y_pred)

print(f"Precision: {overall_precision:.4f}")
print(f"Recall: {overall_recall:.4f}")
print(f"F1-Score: {overall_f1:.4f}")
print()

=====
RENDIMIENTO POR RAZA
=====

print("1. RENDIMIENTO POR RAZA")
print(f"{'-' * 60}")

metrics_by_race = []

```

```

for race in df_test_analysis[RACE_COL].unique():
 mask = df_test_analysis[RACE_COL] == race
 y_true_sub = df_test_analysis.loc[mask, 'y_true'].values
 y_pred_sub = df_test_analysis.loc[mask, 'y_pred'].values

 if len(y_true_sub) > 0:
 try:
 precision = precision_score(y_true_sub, y_pred_sub, zero_division=0)
 recall = recall_score(y_true_sub, y_pred_sub, zero_division=0)
 f1 = f1_score(y_true_sub, y_pred_sub, zero_division=0)

 metrics_by_race.append({
 'Raza': race,
 'N': len(y_true_sub),
 'Precision': precision,
 'Recall': recall,
 'F1': f1
 })
 except Exception as e:
 print(f" [Advertencia] No se pudo calcular métricas para {race}: {e}")

df_metrics_race = pd.DataFrame(metrics_by_race).sort_values('F1', ascending=False)
print(df_metrics_race.to_string(index=False))
print()

Visualización: Comparativa de F1-Score por raza
if len(df_metrics_race) > 0:
 fig, ax = plt.subplots(figsize=(12, 6))
 bars = ax.bar(range(len(df_metrics_race)), df_metrics_race['F1'].values,
 color='steelblue', edgecolor='black')
 ax.set_xticks(range(len(df_metrics_race)))
 ax.set_xticklabels(df_metrics_race['Raza'].values, rotation=45, ha='right')
 ax.axhline(overall_f1, color='red', linestyle='--', linewidth=2, label=f'F1 Global ({overall_f1:.4f})')
 ax.set_ylabel('F1-Score', fontsize=12)
 ax.set_xlabel('Raza', fontsize=12)

```

```

ax.set_title('F1-Score por Raza (Rendimiento del Modelo)', fontsize=14,
fontweight='bold')
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

```

=====
RENDIMIENTO POR SEXO
=====

print("\n2. RENDIMIENTO POR SEXO")
print("-" * 60)

```

```

metrics_by_sex = []
for sex in df_test_analysis[SEX_COL].unique():
 mask = df_test_analysis[SEX_COL] == sex
 y_true_sub = df_test_analysis.loc[mask, 'y_true'].values
 y_pred_sub = df_test_analysis.loc[mask, 'y_pred'].values

```

```

if len(y_true_sub) > 0:
 try:
 precision = precision_score(y_true_sub, y_pred_sub, zero_division=0)
 recall = recall_score(y_true_sub, y_pred_sub, zero_division=0)
 f1 = f1_score(y_true_sub, y_pred_sub, zero_division=0)

```

```

metrics_by_sex.append({
 'Sexo': sex,
 'N': len(y_true_sub),
 'Precision': precision,
 'Recall': recall,
 'F1': f1
})
except Exception as e:
 print(f" [Advertencia] No se pudo calcular métricas para {sex}: {e}")

```

```

df_metrics_sex = pd.DataFrame(metrics_by_sex).sort_values('F1', ascending=False)
print(df_metrics_sex.to_string(index=False))
print()

```

```
Visualización: Comparativa de F1-Score por sexo
if len(df_metrics_sex) > 0:
 fig, ax = plt.subplots(figsize=(12, 6))
 bars = ax.bar(range(len(df_metrics_sex)), df_metrics_sex['F1'].values,
 color='mediumseagreen', edgecolor='black')
 ax.set_xticks(range(len(df_metrics_sex)))
 ax.set_xticklabels(df_metrics_sex['Sexo'].values, rotation=45, ha='right')
 ax.axhline(overall_f1, color='red', linestyle='--', linewidth=2, label=f'F1 Global
({overall_f1:.4f})')
 ax.set_ylabel('F1-Score', fontsize=12)
 ax.set_xlabel('Sexo', fontsize=12)
 ax.set_title('F1-Score por Sexo (Rendimiento del Modelo)', fontsize=14,
 fontweight='bold')
 ax.legend()
 ax.tick_params(axis='x', rotation=45)
 ax.grid(axis='y', alpha=0.3)
 plt.tight_layout()
 plt.show()
```

```
=====
RENDIMIENTO POR EDAD
=====

print("\n3. RENDIMIENTO POR EDAD")
print("-" * 60)
```

```
metrics_by_age = []
for age in df_test_analysis[AGE_COL].unique():
 mask = df_test_analysis[AGE_COL] == age
 y_true_sub = df_test_analysis.loc[mask, 'y_true'].values
 y_pred_sub = df_test_analysis.loc[mask, 'y_pred'].values
```

```
if len(y_true_sub) > 0:
 try:
 precision = precision_score(y_true_sub, y_pred_sub, zero_division=0)
 recall = recall_score(y_true_sub, y_pred_sub, zero_division=0)
 f1 = f1_score(y_true_sub, y_pred_sub, zero_division=0)
```

```

metrics_by_age.append({
 'Edad': age,
 'N': len(y_true_sub),
 'Precision': precision,
 'Recall': recall,
 'F1': f1
})
except Exception as e:
 print(f" [Advertencia] No se pudo calcular métricas para {age}: {e}")

df_metrics_age = pd.DataFrame(metrics_by_age).sort_values('F1', ascending=False)
print(df_metrics_age.to_string(index=False))
print()

Visualización: Comparativa de F1-Score por edad
if len(df_metrics_age) > 0:
 fig, ax = plt.subplots(figsize=(12, 6))
 bars = ax.bar(range(len(df_metrics_age)), df_metrics_age['F1'].values,
 color='skyblue', edgecolor='black')
 ax.set_xticks(range(len(df_metrics_age)))
 ax.set_xticklabels(df_metrics_age['Edad'].values, rotation=45, ha='right')
 ax.axhline(overall_f1, color='red', linestyle='--', linewidth=2, label=f'F1 Global ({overall_f1:.4f})')
 ax.set_ylabel('F1-Score', fontsize=12)
 ax.set_xlabel('Grupo de Edad', fontsize=12)
 ax.set_title('F1-Score por Edad (Rendimiento del Modelo)', fontsize=14,
 fontweight='bold')
 ax.legend()
 ax.tick_params(axis='x', rotation=45)
 ax.grid(axis='y', alpha=0.3)
 plt.tight_layout()
 plt.show()

=====
MATRICES DE CONFUSIÓN POR SUBGRUPOS (POR RAZA)
=====

print("\n4. MATRICES DE CONFUSIÓN POR RAZA")

```

```

print("-" * 60)

races_sorted = sorted(df_test_analysis[RACE_COL].unique())
n_races = len(races_sorted)
n_cols = min(3, n_races)
n_rows = (n_races + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 4 * n_rows))
axes = axes.flatten() if n_races > 1 else [axes]

for idx, race in enumerate(races_sorted):
 mask = df_test_analysis[RACE_COL] == race
 y_true_sub = df_test_analysis.loc[mask, 'y_true'].values
 y_pred_sub = df_test_analysis.loc[mask, 'y_pred'].values

 cm = confusion_matrix(y_true_sub, y_pred_sub)

 im = axes[idx].imshow(cm, cmap='Blues', aspect='auto')
 axes[idx].set_title(f'Raza: {race}\n(N={len(y_true_sub)})', fontweight='bold')
 axes[idx].set_xlabel('Predicho')
 axes[idx].set_ylabel('Real')

 for i in range(cm.shape[0]):
 for j in range(cm.shape[1]):
 axes[idx].text(j, i, cm[i, j], ha="center", va="center",
 color="w" if cm[i, j] > cm.max()/2 else "black",
 fontsize=14, fontweight='bold')

 axes[idx].set_xticks([0, 1])
 axes[idx].set_yticks([0, 1])
 axes[idx].set_xticklabels(['Denegado', 'Aprobado'])
 axes[idx].set_yticklabels(['Denegado', 'Aprobado'])

Ocultar subplots no usados
for idx in range(n_races, len(axes)):
 axes[idx].axis('off')

plt.tight_layout()
plt.show()

```

```
Guardar resultados para uso en otras celdas
df_metrics_race_cached = df_metrics_race
df_metrics_sex_cached = df_metrics_sex
df_metrics_age_cached = df_metrics_age

print("\n✓ Resultados guardados en variables: df_metrics_race_cached,
df_metrics_sex_cached, df_metrics_age_cached")
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 16, in <module>
NameError: name 'gb_model' is not defined. Did you mean: 'n_models'?
```

### Cell 90: ■ Markdown

#### *Interpretación del Rendimiento del Modelo por Subgrupos*

El análisis de rendimiento por subgrupos evalúa si el modelo Gradient Boosting funciona de manera **equitativa** para todos los grupos demográficos. Los resultados globales del modelo (Precision: 0.9549, Recall: 0.9787, F1: 0.9667) se desglosan a continuación:

---

##### **\*\*Por Raza:\*\***

- El rendimiento del modelo es **consistentemente alto** para todos los grupos raciales, con F1-scores entre 0.949 y 0.984.
- El grupo con **mejor rendimiento** es "American Indian or Alaska Native" (F1 = 0.984, Recall = 1.0), aunque su tamaño muestral es reducido (N=259).
- La categoría **"Not Available or Free Text"** tiene el F1 más bajo (0.949), lo cual es coherente con los hallazgos de la Sección 9.1: la falta de información dificulta tanto la aprobación real como la predicción.
- Las diferencias de F1 entre grupos raciales son **menores a 3.5 pp**, lo cual indica que el modelo no discrimina de forma severa entre grupos.

**■ Matiz importante:** Aunque el modelo no muestra sesgo fuerte en sus métricas por raza, hay que recordar que el modelo fue entrenado con datos que ya contienen las decisiones históricas. Si esas decisiones históricas contenían sesgos, el modelo los ha aprendido y reproducido implícitamente.

---

##### **\*\*Por Sexo:\*\***

- Las métricas son muy uniformes: F1 varía solo entre 0.948 y 0.970.
- Las solicitudes **Joint** tienen el mejor rendimiento (F1 = 0.970), seguidas de **Female** (0.968) y **Male** (0.965).

- **"Sex Not Available"** tiene el F1 más bajo (0.948), con un Recall notablemente inferior (0.946 vs. ~0.980 en los demás grupos), lo que indica mayor dificultad del modelo para predecir correctamente las aprobaciones de solicitudes sin información de sexo.

**■ Hallazgo positivo**: Male y Female tienen métricas prácticamente idénticas, confirmando la ausencia de sesgo de género en el modelo.

---

**Por Edad**:

- El modelo funciona bien para todos los grupos etarios conocidos, con F1 entre 0.958 y 0.971.

- El grupo **"Unknown"** es la excepción notable: su F1 cae a **0.828**, impulsado por un Recall de solo **0.729**. Esto significa que el modelo falla en detectar ~27% de las aprobaciones reales en este grupo.

- Los mayores de 74 años tienen el Recall más alto (0.986) pero la Precision más baja (0.932), lo que sugiere que el modelo tiende a sobre-predecir aprobaciones en este grupo.

---

**Conclusiones del análisis ético**:

1. **El modelo no muestra sesgos severos**: Las diferencias de rendimiento entre subgrupos son generalmente pequeñas (<3 pp en F1).

2. **La calidad de datos es la mayor fuente de inequidad**: Las categorías "Not Available" y "Unknown" consistentemente muestran peor rendimiento, tanto en tasas de aprobación reales como en métricas del modelo.

3. **Recomendación**: Mejorar la captura de datos demográficos reduciría las disparidades aparentes. Para los casos con información faltante, considerar un pipeline especializado o flag de revisión manual.

4. **La brecha racial requiere atención**: Aunque las métricas del modelo son uniformes, la brecha en las tasas de aprobación reales (~16 pp entre White y grupos minoritarios) podría indicar sesgos históricamente incorporados en los datos de entrenamiento.

## Cell 91: ■ Markdown

# 10. Análisis de Clustering: Segmentación de Solicitantes de Hipotecas

## ¿Por qué incluir clustering en un proyecto de clasificación supervisada?

Hasta este punto del proyecto, hemos trabajado exclusivamente con **aprendizaje supervisado**: teníamos una variable objetivo (`action_taken` → aprobado/denegado) y hemos entrenado modelos para predecirla. Sin embargo, el **aprendizaje no supervisado** —y en particular el **clustering**— aporta una perspectiva complementaria enormemente valiosa.

El clustering nos permite **descubrir patrones y agrupaciones naturales** en los datos de solicitantes **sin usar la etiqueta de aprobación**. Esto es útil por varias razones:

1. **Perfilado de solicitantes**: ¿Existen grupos naturales de solicitantes con características similares (ingresos, montos de préstamo, ratios de deuda)? Identificar estos perfiles ayuda a entender la



composición del mercado hipotecario de Utah.

2. **\*\*Detección de anomalías\*\***: Algoritmos como DBSCAN pueden identificar solicitudes atípicas que no encajan en ningún perfil habitual. Estas solicitudes podrían representar casos de fraude, errores de datos o simplemente situaciones inusuales que merecen revisión manual.

3. **\*\*Complemento al análisis ético\*\***: Al cruzar los clusters descubiertos con atributos sensibles (raza, sexo, edad), podemos detectar si ciertos grupos demográficos se concentran desproporcionadamente en perfiles de mayor riesgo, lo que podría revelar **\*\*inequidades estructurales\*\*** en el acceso a la vivienda.

4. **\*\*Validación del modelo supervisado\*\***: Comparar las agrupaciones naturales con las decisiones de aprobación/denegación nos permite evaluar si el modelo supervisado está capturando la estructura real de los datos o si hay discrepancias importantes.

## Algoritmos que utilizaremos

| Algoritmo | Ventaja principal | Limitación principal |

|-----|-----|-----|

| **\*\*K-Means\*\*** | Simple, rápido, escalable | Requiere especificar K; asume clusters esféricos |

| **\*\*DBSCAN\*\*** | Detecta formas arbitrarias y outliers; no requiere K | Sensible a la elección de `eps` y `min\_samples` |

| **\*\*Clustering Jerárquico\*\*** | Proporciona dendrograma para visualizar la jerarquía | No escala bien a datasets muy grandes |

## Cell 92: ■ Markdown

### 10.1 Preparación de Datos para Clustering

Antes de aplicar cualquier algoritmo de clustering, necesitamos:

**\*\*Seleccionar las variables numéricas relevantes\*\*** para la segmentación. Usaremos variables que describan el perfil financiero del solicitante y las características del préstamo:

- `loan\_amount`: monto del préstamo solicitado
- `interest\_rate`: tasa de intereses del préstamo
- `loan\_term`: plazo del préstamo
- `income`: ingresos anuales del solicitante
- `property\_value`: valor de la propiedad

> ■■ **\*\*Importante\*\***: No incluimos la variable objetivo (`approved`) ni los atributos sensibles (`derived\_race`, `derived\_sex`, `applicant\_age`) como features de clustering para evitar sesgos en nuestro análisis.

No es necesario ningún tratamiento de otro tipo ya que anterioremente hemos escalado y codificado los datos.

### Cell 93: ■ Code

```
df_modelo_filtrado.info()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
File "<string>", line 1, in <module>
NameError: name 'df_modelo_filtrado' is not defined
```

### Cell 94: ■ Code

```
# 10.1 Preparación de datos para clustering
# =====

import gc

# Liberar variables pesadas que ya no se necesitan para clustering
# (se pueden regenerar si hace falta después)
for var in ['df', 'df_original', 'df_limpio', 'df_modelo',
            'X', 'X_train', 'X_test', 'X_train_proc', 'X_test_proc']:
    if var in dir():
        exec(f'del {var}')
gc.collect()

# Seleccionar las variables numéricas relevantes para clustering
clustering_features = ['loan_amount', 'interest_rate', 'loan_term', 'income',
                       'property_value']

X_clustering = df_modelo_filtrado[clustering_features].values # .values → numpy, no
copia extra

# Escalar los datos con StandardScaler
scaler_clustering = StandardScaler()
X_clustering = scaler_clustering.fit_transform(X_clustering)
print(f"\nDatos escalados - shape: {X_clustering.shape}")
print(f"Memoria liberada. RAM usada por X_clustering: {X_clustering.nbytes /
1024**2:.1f} MB")
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
```

```

exec('\n'.join(lines[:-1]), glb)
~~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAA~
File "<string>", line 15, in <module>
NameError: name 'df_modelo_filtrado' is not defined

```

## Cell 95: ■ Markdown

### 10.2 K-Means: Segmentación de Perfiles de Solicitantes

#### #### 10.2.1 Selección del número óptimo de clusters (K)

Para determinar el número adecuado de clusters, utilizamos dos métodos complementarios:

1. **Método del Codo (Elbow Method)**: ejecutamos K-Means para  $K = 2$  hasta  $K = 10$  y graficamos la **inercia** (suma de distancias al cuadrado de cada punto a su centroide). Buscamos el punto de inflexión donde añadir más clusters deja de reducir significativamente la inercia.
2. **Silhouette Score**: mide qué tan bien definidos están los clusters. Un valor cercano a 1 indica clusters bien separados; cercano a 0 indica solapamiento; negativo indica asignaciones incorrectas. Elegimos el  $K$  que maximice este score.

## Cell 96: ■ Code

```

10.2.1 Método del Codo y Silhouette Score (Optimizado)
=====

from sklearn.cluster import MiniBatchKMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import numpy as np
import gc

inertias = []
sil_scores = []
K_range = range(2, 8)

Submuestreo agresivo: 2,000 muestras (suficiente estadísticamente, evita OOM)
n_samples = X_clustering.shape[0]
sample_size = min(2_000, n_samples)
rng = np.random.RandomState(42)
sample_idx = rng.choice(n_samples, size=sample_size, replace=False)
X_sample = X_clustering[sample_idx]

for k in K_range:

```

```

kmeans_temp = MiniBatchKMeans(n_clusters=k, random_state=42, n_init=3,
max_iter=100, batch_size=1024)
kmeans_temp.fit(X_clustering)
inertias.append(kmeans_temp.inertia_)
labels_sample = kmeans_temp.predict(X_sample)
sil_scores.append(silhouette_score(X_sample, labels_sample))
del kmeans_temp, labels_sample
gc.collect()

```

```

del X_sample, sample_idx
gc.collect()

```

```

Gráfico combinado: Método del Codo + Silhouette Score
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

```

```

ax1.plot(K_range, inertias, 'bo-', linewidth=2, markersize=8)
ax1.set_xlabel('Número de Clusters (K)')
ax1.set_ylabel('Inercia')
ax1.set_title('Método del Codo')
ax1.set_xticks(list(K_range))
ax1.grid(True, alpha=0.3)

```

```

ax2.plot(K_range, sil_scores, 'ro-', linewidth=2, markersize=8)
ax2.set_xlabel('Número de Clusters (K)')
ax2.set_ylabel('Silhouette Score')
ax2.set_title('Silhouette Score por K')
ax2.set_xticks(list(K_range))
ax2.grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.show()

```

```

print("K | Inercia | Silhouette Score")
print("-" * 40)
for k, inertia, sil in zip(K_range, inertias, sil_scores):
 print(f"{k} | {inertia:>12.2f} | {sil:.4f}")

```

#### Error:

```
Traceback (most recent call last):
```

```

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
File "<string>", line 56
    for k, inertia, sil in zip(K_range, inertias, sil_scores):
                                   ^
IndentationError: expected an indented block after 'for' statement on line 56
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^
File "<string>", line 15, in <module>
NameError: name 'X_clustering' is not defined

```

## Cell 97: ■ Markdown

#### Interpretación de los Resultados

**\*\*Tabla de métricas por K\*\***

K	Inercia	Silhouette Score
2	955,706	<b>**0.4758**</b>
3	886,610	0.3915
4	846,329	0.3963
5	805,659	0.4062
6	774,833	0.4464
7	758,045	0.4325

**\*\*Silhouette Score\*\***

- El **\*\*mejor Silhouette Score lo obtiene K=2\*\*** (0.4758), lo que indica que la separación más natural de los datos es en **\*\*dos grandes grupos\*\***. Sin embargo, un valor de ~0.48 refleja una estructura de clusters **\*\*moderada\*\*** — no débil, pero tampoco fuertemente definida.

- A partir de K=3, los scores oscilan entre 0.39 y 0.45. Aunque son inferiores a K=2, las diferencias entre ellos son pequeñas, lo que indica que **\*\*no hay un K claramente superior\*\*** más allá de 2.

- En general, scores en el rango 0.39–0.48 indican **\*\*solapamiento considerable\*\*** entre los grupos, algo esperable en datos hipotecarios donde los perfiles de solicitantes forman un continuo más que clusters discretos.

**\*\*Método del Codo (Inercia)\*\***

- La inercia decrece de forma continua sin un "codo" claramente marcado, lo que confirma que los datos **\*\*no tienen una segmentación natural muy definida\*\*** en múltiples clusters.

#### Decisión: K = 4

Aunque K=2 maximiza el Silhouette Score, **elegimos K=4** por las siguientes razones:

1. **Mayor granularidad interpretativa**: con solo 2 clusters, la segmentación es demasiado gruesa para extraer perfiles de negocio útiles. Con 4 clusters podemos identificar perfiles diferenciados (alto valor, largo plazo, bajo monto, etc.) que aportan valor a los stakeholders.
2. **Silhouette aceptable** (0.3963): aunque K=5 (0.4062) y K=6 (0.4464) tienen silhouettes ligeramente mejores, K=4 ofrece el mejor equilibrio entre **granularidad interpretativa** y **separación de clusters** sin fragmentar excesivamente los datos.
3. **Consistencia con la práctica**: en estudios de segmentación hipotecaria, 3–5 perfiles es el rango habitual para generar insights accionables.

A continuación, exploramos también **DBSCAN** y **clustering jerárquico** para validar estos hallazgos y detectar outliers.

## Cell 98: ■ Markdown

#### 10.2.2 Entrenamiento de K-Means con K óptimo

Una vez seleccionado el valor de K según los métodos anteriores, entrenamos el modelo definitivo y asignamos cada solicitante a un cluster.

**Tareas:**

- Entrenar `KMeans` con el K elegido
- Añadir la columna `cluster` al DataFrame original
- Verificar la distribución de solicitantes por cluster

## Cell 99: ■ Code

```
10.2.2 Entrenamiento de K-Means con K óptimo
=====
```

```
from sklearn.cluster import MiniBatchKMeans
from sklearn.metrics import silhouette_score
import gc
```

```
K_OPTIMO = 4
```

```
kmeans_final = MiniBatchKMeans(n_clusters=K_OPTIMO, random_state=42, n_init=8,
batch_size=1024)
clusters = kmeans_final.fit_predict(X_clustering)
```

```
Añadir etiqueta de cluster al DataFrame original
```

```
df_modelo_filtrado['cluster'] = clusters

Distribución de solicitantes por cluster
print("Distribución de solicitantes por cluster:")
print(df_modelo_filtrado['cluster'].value_counts().sort_index())

Silhouette sobre muestra (el dataset completo requiere ~400GB de RAM)
sample_idx = np.random.RandomState(42).choice(len(X_clustering), size=min(2_000,
len(X_clustering)), replace=False)
sil_final = silhouette_score(X_clustering[sample_idx], clusters[sample_idx])
print(f"\nSilhouette Score (estimado sobre muestra de 2,000): {sil_final:.4f}")
del sample_idx; gc.collect()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 11, in <module>
NameError: name 'X_clustering' is not defined
```

#### Cell 100: ■ Markdown

#### Resultado del Entrenamiento K-Means (K=4)

La distribución de los **226,790** solicitantes en los 4 clusters es la siguiente:

Cluster	Solicitudes	% del Total
0	19,203	8.5%
1	124,962	55.1%
2	36,707	16.2%
3	45,918	20.2%

**Observaciones:**

- **El Cluster 1 concentra más de la mitad de los solicitantes** (55.1%), lo que indica que el perfil financiero más común en el mercado hipotecario de Utah es altamente homogéneo.
- Los clusters 0 (8.5%) y 2 (16.2%) representan segmentos más pequeños pero diferenciados.
- El Cluster 3 (20.2%) agrupa al segundo contingente más grande.
- El **Silhouette Score** estimado es 0.4411, consistente con la evaluación de K=4 en la fase de selección (0.3963) y confirmando una separación moderada pero estable.

> ■ Esta distribución desbalanceada es **coherente con la realidad del mercado hipotecario**: la mayoría de los solicitantes tienen perfiles estándar (préstamos a 30 años con tasas de mercado), mientras que las hipotecas a corto plazo, de alto valor o con tasas elevadas constituyen nichos más especializados.

## Cell 101: ■ Markdown

### #### 10.2.3 Visualización de Clusters con PCA

Como nuestros datos tienen más de 2 dimensiones, necesitamos una técnica de **reducción de dimensionalidad** para poder visualizar los clusters en un plano 2D. Utilizamos **PCA (Principal Component Analysis)**, que proyecta los datos sobre los 2 componentes que mejor capturan la varianza.

## Cell 102: ■ Code

```
# 10.2.3 Visualización de Clusters con PCA
# =====

from sklearn.decomposition import PCA
import gc

# PCA sobre muestra para evitar OOM (227K × 5 features es manejable,
# pero guardamos solo las coordenadas necesarias para graficar)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_clustering)

# Graficar con subsample si hay demasiados puntos (más de 20K puntos satura el
# gráfico)
plot_size = min(20_000, len(X_pca))
plot_idx = np.random.RandomState(42).choice(len(X_pca), size=plot_size,
replace=False)

plt.figure(figsize=(12, 8))
scatter = plt.scatter(X_pca[plot_idx, 0], X_pca[plot_idx, 1], c=clusters[plot_idx],
cmap='Paired', alpha=0.6, s=30)

# Proyectar centroides al espacio PCA
centroids_pca = pca.transform(kmeans_final.cluster_centers_)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], c='red', marker='x', s=200,
edgecolors='black', linewidths=2)

plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} varianza)')
```



```
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} varianza)')
plt.title(f'Clusters K-Means: Proyección PCA 2D (muestra de {plot_size:,} puntos)')
plt.colorbar(scatter, label='Cluster')
plt.grid(True, alpha=0.3)
plt.show()

print(f"Varianza total explicada por 2 componentes:
{sum(pca.explained_variance_ratio_[:2]):.1%}")

del plot_idx; gc.collect()
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 10, in <module>
NameError: name 'X_clustering' is not defined
```

#### Cell 103: ■ Markdown

#### Interpretación de la Visualización PCA

- **Varianza explicada: 41.3%** — Las 2 componentes principales capturan solo el 41% de la variabilidad total. Esto implica que la proyección 2D muestra **menos de la mitad de la información**, por lo que la superposición visual entre clusters no significa que sean idénticos en el espacio original de 5 dimensiones.
- **Concentración central con cola de outliers**: la gran mayoría de los solicitantes se agrupa en una zona densa central. Los puntos alejados corresponden a solicitudes con valores de propiedad o montos de préstamo extremos.
- **Los centroides (X rojo)** se diferencian principalmente en la dirección del PC1, lo que indica que las variables con mayor peso en la separación son probablemente `loan\_amount`, `property\_value` y `loan\_term`.

#### Cell 104: ■ Markdown

### 10.3 DBSCAN: Detección de Solicitudes Atípicas

A diferencia de K-Means, **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) no requiere que especifiquemos el número de clusters. En su lugar, agrupa puntos que están densamente conectados y marca como **ruido (outliers)** aquellos que quedan aislados.

#### ¿Por qué DBSCAN es especialmente útil aquí?

En el contexto de préstamos hipotecarios, los **outliers** que DBSCAN detecta podrían ser:

- **Solicitudes con combinaciones inusuales**: ingresos muy bajos pero préstamos muy altos, o viceversa
- **Perfiles atípicos**: solicitantes que no encajan en ningún segmento habitual del mercado
- **Posibles errores de datos**: registros con valores inconsistentes que merecen revisión

#### Parámetros clave de DBSCAN

- **eps** (epsilon): radio máximo de vecindad. Un punto se considera vecino de otro si está a distancia  $\leq \text{eps}$ . Valores típicos: 0.3 - 1.0 (en datos escalados).
- **min\_samples**: número mínimo de puntos dentro del radio eps para que un punto se considere "core point" (núcleo de un cluster). Regla general: al menos  $\text{n\_features} + 1$  o mayor..

## Cell 105: ■ Code

```
10.3.1 Selección de eps mediante gráfico de distancias k-NN
=====

from sklearn.neighbors import NearestNeighbors
import gc

Muestra pequeña (3K) – k-NN crea matrices internas que consumen mucha RAM
KNN_SAMPLE = min(3_000, len(X_clustering))
knn_idx = np.random.RandomState(42).choice(len(X_clustering), size=KNN_SAMPLE,
replace=False)
X_knn_sample = X_clustering[knn_idx]

k = 4
nn = NearestNeighbors(n_neighbors=k, algorithm='kd_tree')
nn.fit(X_knn_sample)
distances, _ = nn.kneighbors(X_knn_sample)

k_distances = np.sort(distances[:, -1])

plt.figure(figsize=(12, 6))
plt.plot(k_distances, linewidth=2)
plt.ylim([0, np.percentile(k_distances, 99.5)])
plt.xlabel('Puntos (ordenados por distancia)')
plt.ylabel(f'Distancia al {k}-ésimo vecino más cercano')
plt.title(f'Gráfico de Distancias k-NN (k={k}) para Selección de eps (muestra de {KNN_SAMPLE:,})')
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

```
del X_knn_sample, knn_idx, nn, distances, k_distances
gc.collect()
```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 8, in <module>

NameError: name 'X\_clustering' is not defined

#### Cell 106: ■ Markdown

#### Selección del parámetro `eps`

El gráfico de distancias k-NN muestra un **codo muy pronunciado**: la gran mayoría de los puntos tienen vecinos cercanos (distancias bajas), pero un pequeño porcentaje se dispara hacia distancias altas. Esto confirma la estructura observada en K-Means: **un gran núcleo denso de solicitudes similares con una cola de outliers dispersos**.

Seleccionamos **`eps = 0.35`** como punto de corte, situado justo después de la zona de alta densidad y antes de la dispersión de outliers. Con `min_samples = 5` ( $\geq n\_features$ ), tenemos un criterio conservador que marcará como ruido solo los puntos genuinamente aislados.

#### Cell 107: ■ Code

```
10.3.2 Aplicación de DBSCAN
```

```
=====
```

```
from sklearn.cluster import DBSCAN
```

```
import gc
```

```
EPS_ELEGIDO = 0.35
```

```
MIN_SAMPLES = 5
```

```
DBSCAN sobre muestra para evitar OOM (227K puntos consume toda la RAM)
```

```
DBSCAN_SAMPLE = min(20_000, len(X_clustering))
```

```
dbscan_idx = np.random.RandomState(42).choice(len(X_clustering),
size=DBSCAN_SAMPLE, replace=False)
```

```
X_dbscan_sample = X_clustering[dbscan_idx]
```

```
dbscan = DBSCAN(eps=EPS_ELEGIDO, min_samples=MIN_SAMPLES)
```

```
dbscan_labels_sample = dbscan.fit_predict(X_dbscan_sample)
```

```

Asignar etiquetas a todo el dataset: puntos no muestreados → predicción por
vecino más cercano

Para mantener simplicidad, asignamos -2 (no evaluado) a los no muestreados
dbscan_labels = np.full(len(X_clustering), -2, dtype=int)
dbscan_labels[dbscan_idx] = dbscan_labels_sample

Resumen sobre la muestra
n_clusters_dbscan = len(set(dbscan_labels_sample)) - (1 if -1 in
dbscan_labels_sample else 0)
n_noise = (dbscan_labels_sample == -1).sum()

print(f"DBSCAN (sobre muestra de {DBSCAN_SAMPLE:,} puntos):")
print(f"Clusters encontrados: {n_clusters_dbscan}")
print(f"Puntos clasificados como ruido (outliers): {n_noise}
({n_noise/DBSCAN_SAMPLE:.1%})")
print("\nDistribución por cluster:")
for label in sorted(set(dbscan_labels_sample)):
count = (dbscan_labels_sample == label).sum()
name = "RUIDO" if label == -1 else f"Cluster {label}"
print(f" {name}: {count} solicitudes ({count/DBSCAN_SAMPLE:.1%})")

del X_dbscan_sample, dbscan
gc.collect()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 11, in <module>
NameError: name 'X_clustering' is not defined

```

#### Cell 108: ■ Markdown

#### Resultados de DBSCAN

Sobre una muestra de **20,000 solicitudes** con `eps=0.35` y `min_samples=5`:

| Categoría                           | Solicitudes | %            |
|-------------------------------------|-------------|--------------|
| ----- ----- ---                     |             |              |
| <b>Cluster 0</b> (núcleo principal) | 19,540      | <b>97.7%</b> |
| Cluster 1 (segundo grupo)           | 155         | 0.8%         |

| Clusters 2–13 (microgrupos) | 108 | 0.5% |

| **Ruido / Outliers** | **197** | **1.0%** |

#### #### Interpretación

- **Un único cluster dominante (97.7%)**: DBSCAN confirma que la inmensa mayoría de las solicitudes forman **un solo grupo denso y homogéneo**. En la práctica, esto significa que no existen múltiples segmentos naturales claramente separados por densidad en el espacio de variables financieras.

- **14 clusters en total**, pero 13 de ellos son **microgrupos** con menos de 160 solicitudes cada uno, formados por pequeñas concentraciones locales en la periferia del cluster principal.

- **197 outliers (1.0%)**: solicitudes que no pertenecen a ningún grupo denso. Son los candidatos más claros para revisión detallada.

#### #### Conclusión práctica

- DBSCAN **valida lo observado con K-Means**: los datos no poseen múltiples segmentos densos y claramente separados. La estructura natural del mercado hipotecario de Utah es un **gran núcleo central con dispersión periférica gradual**.

- El principal valor añadido de DBSCAN en este análisis es su función como **detector de anomalías**: identifica ese 1.0% de solicitudes atípicas cuyo perfil analizaremos a continuación.

- Para segmentación en perfiles de negocio, **K-Means sigue siendo más adecuado**, ya que permite forzar una partición interpretable del cluster principal que DBSCAN no puede proporcionar.

## Cell 109: ■ Code

```
# 10.3.3 Visualización de DBSCAN (PCA 2D)
# =====

# Usar solo los puntos de la muestra DBSCAN (los únicos con etiquetas válidas)
mask_evaluated = dbscan_labels != -2 # -2 = no evaluado
pidx = np.where(mask_evaluated)[0]

plt.figure(figsize=(12, 8))

mask_normal = dbscan_labels[pidx] != -1
scatter = plt.scatter(X_pca[pidx[mask_normal], 0], X_pca[pidx[mask_normal], 1],
c=dbscan_labels[pidx[mask_normal]], cmap='viridis', alpha=0.6, s=30,
label='Clusters')

mask_noise = dbscan_labels[pidx] == -1
n_noise_plot = mask_noise.sum()
plt.scatter(X_pca[pidx[mask_noise], 0], X_pca[pidx[mask_noise], 1],
c='red', marker='x', s=50, alpha=0.8, label=f'Outliers ({n_noise_plot})')
```

```
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} varianza)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} varianza)')
plt.title(f'DBSCAN: Clusters y Outliers (muestra de {len(pidx):,})')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 5, in <module>
NameError: name 'dbscan_labels' is not defined
```

#### Cell 110: ■ Code

```
plt.figure(figsize=(12, 8))

Solo puntos evaluados por DBSCAN, sin outliers
mask_evaluated = dbscan_labels != -2
pidx = np.where(mask_evaluated)[0]
mask_normal = dbscan_labels[pidx] != -1

scatter = plt.scatter(X_pca[pidx[mask_normal], 0], X_pca[pidx[mask_normal], 1],
c=dbscan_labels[pidx[mask_normal]], cmap='viridis', alpha=0.6, s=30,
label='Clusters')

plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} varianza)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} varianza)')
plt.title('DBSCAN: Solo Clusters (Proyección PCA 2D)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 4, in <module>
NameError: name 'dbscan_labels' is not defined
```

## Cell 111: ■ Markdown

#### Visualización DBSCAN en PCA 2D

Estas gráficas ilustran la estructura detectada por DBSCAN:

- **Primera gráfica (con outliers)**: los puntos rojos (X) representan las 197 solicitudes atípicas, ubicadas en la periferia del cluster principal. Se dispersan especialmente en las zonas de valores extremos de PC1 y PC2, correspondiendo a solicitudes con valores de propiedad inusualmente altos y/o tasas de interés fuera del rango habitual.
  - **Segunda gráfica (solo clusters)**: al eliminar los outliers se aprecia la extrema concentración del 97.7% de los datos en una misma región compacta, con el Cluster 1 (155 puntos) ligeramente desplazado respecto al núcleo principal.
- > ■ **Hallazgo clave**: la estructura de los datos hipotecarios de Utah es fundamentalmente **unimodal** — un gran mercado principal con características financieras similares, rodeado de una cola de solicitudes atípicas de alto valor. No existen subpoblaciones separadas por densidad, sino un continuo con dispersión gradual.

## Cell 112: ■ Markdown

#### 10.3.4 Análisis de las Solicitudes Atípicas (Outliers)

Los puntos marcados como ruido por DBSCAN son solicitudes que no encajan en ningún grupo denso. Es interesante examinar sus características para entender **por qué** son atípicas.

**Tareas:**

- Extraer las solicitudes marcadas como outliers
- Calcular estadísticas descriptivas de los outliers vs. el resto
- Identificar qué variables hacen que esas solicitudes sean atípicas
- Reflexionar: ¿son errores de datos, fraude potencial o simplemente casos legítimos pero inusuales?

## Cell 113: ■ Code

```
# 10.3.4 Análisis de solicitudes atípicas
# =====

# Solo usar puntos evaluados por DBSCAN (la muestra)
mask_evaluated = dbscan_labels != -2
X_eval = X_clustering[mask_evaluated]
labels_eval = dbscan_labels[mask_evaluated]

df_clustering = pd.DataFrame(X_eval, columns=clustering_features)
```

```

df_clustering['dbscan_label'] = labels_eval

outliers = df_clustering[df_clustering['dbscan_label'] == -1]
normales = df_clustering[df_clustering['dbscan_label'] != -1]

print("=" * 60)
print("COMPARATIVA: OUTLIERS vs. SOLICITUDES NORMALES")
print("=" * 60)

print(f"\n--- Outliers ({len(outliers)} solicitudes) ---")
print(outliers[clustering_features].describe().round(2))

print(f"\n--- Solicitudes normales ({len(normales)} solicitudes) ---")
print(normales[clustering_features].describe().round(2))

print("\n¿Qué variables presentan mayor diferencia entre outliers y normales?")
for col in clustering_features:
    diff = outliers[col].mean() - normales[col].mean()
    print(f" {col}: diferencia media = {diff:+.2f}")

del df_clustering, X_eval, labels_eval
gc.collect()

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 5, in <module>
NameError: name 'dbscan_labels' is not defined

```

#### Cell 114: ■ Markdown

#### Perfil de los Outliers

DBSCAN identificó **197 solicitudes atípicas** en la muestra de 20,000 (1.0%). Comparando las características medias (en **escala estandarizada**) de outliers vs. solicitudes normales:

| Variable | Media outliers | Media normales | Diferencia | Interpretación |
|----------|----------------|----------------|------------|----------------|
|----------|----------------|----------------|------------|----------------|

|                  |       |       |              |                                                                                            |
|------------------|-------|-------|--------------|--------------------------------------------------------------------------------------------|
| `property_value` | +3.78 | -0.03 | <b>+3.81</b> | Propiedades de valor <b>muy superior</b> al promedio (~4 desviaciones estándar por encima) |
| `interest_rate`  | +0.79 | -0.01 | <b>+0.80</b> | Tasas de interés notablemente <b>más altas</b> que la media                                |



| `loan\_term` | -0.48 | -0.01 | **-0.47** | Plazos ligeramente más cortos |

| `income` | +0.13 | -0.00 | **+0.13** | Ingresos marginalmente superiores |

| `loan\_amount` | +0.08 | -0.00 | **+0.08** | Montos de préstamo similares al promedio |

> ■■ **\*\*Nota\*\***: estos valores están en escala estandarizada (media=0, std=1). Una diferencia de +3.81 en `property\_value` significa que los outliers tienen propiedades valoradas casi 4 desviaciones estándar por encima del promedio del mercado.

#### #### Conclusión

Los outliers se caracterizan principalmente por dos rasgos:

1. **\*\*Propiedades de valor extremadamente alto\*\*** (+3.81 $\sigma$ ): son solicitudes del segmento de **\*\*lujo\*\*** del mercado inmobiliario de Utah, con propiedades que superan ampliamente los valores típicos.

2. **\*\*Tasas de interés más elevadas\*\*** (+0.80 $\sigma$ ): sorprendentemente, estas solicitudes de alto valor también presentan tasas superiores a la media, lo que podría reflejar préstamos jumbo o financiación no convencional con condiciones distintas.

A diferencia de lo que podría esperarse, **\*\*el monto del préstamo no es el factor diferenciador\*\*** (+0.08 $\sigma$ ): los outliers solicitan montos similares al mercado general, pero sobre propiedades de valor muy superior. Esto implica un **\*\*ratio préstamo/valor (LTV) muy bajo\*\***, indicativo de compradores con alta capacidad patrimonial que financian solo una pequeña fracción de la propiedad.

Estas solicitudes no son errores de datos, sino un **\*\*segmento legítimo de mercado premium\*\*** que, por su baja frecuencia y perfil financiero diferenciado, podría merecer una evaluación de riesgo específica.

## Cell 115: ■ Markdown

### 10.4 Clustering Jerárquico y Comparativa de Algoritmos

El **\*\*Clustering Jerárquico Aglomerativo\*\*** comienza tratando cada punto como un cluster individual y va fusionando los más cercanos hasta formar un número deseado de grupos. Su principal ventaja es el **\*\*dendrograma\*\***: una representación visual en forma de árbol que muestra cómo se van agrupando los datos a diferentes niveles.

**\*\*¿Qué aporta respecto a K-Means?\*\***

- El dendrograma nos permite **\*\*visualizar la jerarquía\*\*** de agrupaciones y decidir el corte óptimo
- No asume clusters esféricos
- Permite evaluar si la estructura de los datos es realmente jerárquica

**\*\*Tareas:\*\***

1. Generar el dendrograma (con una muestra si el dataset es muy grande)
2. Aplicar `AgglomerativeClustering` con el número de clusters elegido
3. Comparar visualmente los resultados de los 3 algoritmos (K-Means, DBSCAN, Jerárquico)

> ■■ **\*\*Nota sobre rendimiento\*\***: El clustering jerárquico con `linkage` completo tiene complejidad  $O(n^3)$ , por lo que si el dataset es muy grande (>10.000 registros), conviene usar una muestra para el

dendrograma.

#### Cell 116: ■ Code

```
10.4.1 Dendrograma
=====

from scipy.cluster.hierarchy import dendrogram, linkage
import gc

Muestra pequeña para dendrograma (linkage es $O(n^3)$ en memoria)
SAMPLE_SIZE = min(3_000, len(X_clustering))
np.random.seed(42)
sample_idx = np.random.choice(len(X_clustering), SAMPLE_SIZE, replace=False)
X_sample = X_clustering[sample_idx]

Calcular la matriz de enlace (linkage)
Z = linkage(X_sample, method='ward')

Visualizar el dendrograma
plt.figure(figsize=(16, 8))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=90, leaf_font_size=10)
plt.title('Dendrograma - Clustering Jerárquico (Ward)')
plt.xlabel('Índice de la muestra (o tamaño del cluster)')
plt.ylabel('Distancia')
plt.axhline(y=100, color='r', linestyle='--', label='Corte')
plt.legend()
plt.tight_layout()
plt.show()

del X_sample, sample_idx, Z
gc.collect()
```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 8, in <module>

NameError: name 'X\_clustering' is not defined

## Cell 117: ■ Code

```
10.4.2 Comparativa visual de los 3 algoritmos
=====

from sklearn.cluster import AgglomerativeClustering
import gc

Muestra pequeña para AgglomerativeClustering ($O(n^2)$ en memoria)
AGGLO_SAMPLE = min(5_000, len(X_clustering))
agglo_idx = np.random.RandomState(42).choice(len(X_clustering), size=AGGLO_SAMPLE,
replace=False)

agglo = AgglomerativeClustering(n_clusters=K_OPTIMO)
agglo_labels_sample = agglo.fit_predict(X_clustering[agglo_idx])
del agglo; gc.collect()

Usar muestra DBSCAN evaluada para el panel central
dbscan_eval_mask = dbscan_labels != -2
dbscan_eval_idx = np.where(dbscan_eval_mask)[0]

Muestra para K-Means (panel izquierdo)
plot_size = min(5_000, len(X_pca))
plot_idx = np.random.RandomState(42).choice(len(X_pca), size=plot_size,
replace=False)

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
fig.suptitle('Comparativa de Algoritmos de Clustering (Proyección PCA 2D)',
fontsize=16)

K-Means
axes[0].scatter(X_pca[plot_idx, 0], X_pca[plot_idx, 1], c=clusters[plot_idx],
cmap='viridis', alpha=0.6, s=20)
axes[0].set_title(f'K-Means (K={K_OPTIMO})')
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')

DBSCAN (solo puntos evaluados)
dbscan_plot = dbscan_eval_idx[:5000] # limitar puntos a graficar
axes[1].scatter(X_pca[dbscan_plot, 0], X_pca[dbscan_plot, 1],
c=dbscan_labels[dbscan_plot],
```

```

cmap='viridis', alpha=0.6, s=20)
axes[1].set_title(f'DBSCAN (eps={EPS_ELEGIDO}, clusters={n_clusters_dbscan})')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')

Jerárquico (sobre su muestra)
axes[2].scatter(X_pca[agglo_idx, 0], X_pca[agglo_idx, 1], c=agglo_labels_sample,
cmap='viridis', alpha=0.6, s=20)
axes[2].set_title(f'Jerárquico (K={K_OPTIMO}, muestra)')
axes[2].set_xlabel('PC1')
axes[2].set_ylabel('PC2')

plt.tight_layout()
plt.show()

del agglo_labels_sample, agglo_idx, plot_idx, dbscan_plot
gc.collect()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 8, in <module>
NameError: name 'X_clustering' is not defined

```

#### Cell 118: ■ Markdown

#### Comparativa Visual de los 3 Algoritmos

La gráfica muestra los tres enfoques de clustering proyectados en 2D mediante PCA:

Algoritmo	Comportamiento observado	Utilidad principal
<b>K-Means</b>	Divide el espacio en 4 regiones, asignando <b>todos</b> los puntos a un cluster	Segmentación de negocio: perfiles interpretables
<b>DBSCAN</b>	Identifica un único cluster denso (97.7%) y marca outliers periféricos	Detección de anomalías: solicitudes atípicas
<b>Jerárquico</b>	Resultados muy similares a K-Means, con la misma partición en 4 grupos	Validación: confirma la consistencia de la segmentación

> ■ **Conclusión de la comparativa**: los tres algoritmos coinciden en la misma estructura fundamental — un mercado hipotecario dominado por un perfil mayoritario, con subgrupos diferenciados

principalmente por **plazo del préstamo** y **tasa de interés**. La consistencia entre algoritmos refuerza la confianza en la segmentación obtenida.

---

## 10.5 Interpretación y Perfilado de Clusters

Una vez asignados los clusters, el paso más importante es **interpretar qué representa cada grupo**. Para ello:

1. **Estadísticas descriptivas por cluster**: media, mediana y desviación estándar de cada variable por cluster
2. **Boxplots comparativos**: distribución de cada variable financiera por cluster
3. **Heatmap de centroides**: mapa de calor que muestra las características promedio (normalizadas) de cada cluster
4. **Descripción cualitativa**: nombre descriptivo y perfil de cada segmento

> ■ **Objetivo**: que cualquier persona no técnica pueda entender los perfiles descubiertos y su relevancia para el negocio hipotecario.

### Cell 119: ■ Code

```
# 10.5.1 Estadísticas descriptivas por cluster
# =====

# Media de cada variable por cluster (K-Means)
cluster_profiles =
df_modelo_filtrado.groupby('cluster')[clustering_features].agg(['mean', 'median',
'std'])
print("Perfil promedio de cada cluster (K-Means):")
print(cluster_profiles.round(2))
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~^~~~~~
 File "<string>", line 5, in <module>
NameError: name 'df_modelo_filtrado' is not defined
```

### Cell 120: ■ Code

```
10.5.2 Boxplots comparativos por cluster
=====

n_features = len(clustering_features)
```

```

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Distribución de Variables Financieras por Cluster', fontsize=16)

for i, col in enumerate(clustering_features):
 ax = axes[i // 3, i % 3]
 df_modelo_filtrado.boxplot(column=col, by='cluster', ax=ax)
 ax.set_title(col)
 ax.set_xlabel('Cluster')
 ax.set_ylabel(col)

Ocultar el subplot vacío (posición 2,2)
if n_features < 6:
 axes[1, 2].set_visible(False)

plt.tight_layout()
plt.show()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 10, in <module>
NameError: name 'df_modelo_filtrado' is not defined

```

#### Cell 121: ■ Code

```

# 10.5.3 Heatmap de centroides normalizados
# =====

import seaborn as sns

# Crear DataFrame con los centroides en escala original
centroids_original =
scaler_clustering.inverse_transform(kmeans_final.cluster_centers_)
df_centroids = pd.DataFrame(centroids_original, columns=clustering_features)
df_centroids.index = [f'Cluster {i}' for i in range(K_OPTIMO)]

# Normalizar para el heatmap (min-max por columna)
df_centroids_norm = (df_centroids - df_centroids.min()) / (df_centroids.max() -
df_centroids.min())

```

```
plt.figure(figsize=(12, 6))
sns.heatmap(df_centroids_norm, annot=df_centroids.round(0), fmt='.0f',
            cmap='YlOrRd',
            linewidths=0.5, cbar_kws={'label': 'Valor normalizado (0-1)'})
plt.title('Heatmap de Centroides: Perfil de Cada Cluster')
plt.ylabel('Cluster')
plt.xlabel('Variable')
plt.tight_layout()
plt.show()

print("\nValores reales de los centroides (sin normalizar):")
print(df_centroids.round(2))
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 7, in <module>
NameError: name 'scaler_clustering' is not defined
```

#### Cell 122: ■ Markdown

##### #### 10.5.4 Descripción Cualitativa de los Perfiles

A partir de los centroides, los boxplots y el heatmap, definimos los **4 perfiles** descubiertos:

Cluster	Nombre del Perfil	Solicitudes	Tasa Aprob.	Características principales
0	Propiedades Premium con LTV Bajo	19,203 (8.5%)	92.9%	Préstamos de ~293K sobre propiedades de alto valor (~880K), resultando en un LTV muy bajo (~33%). Tasa de interés más baja del mercado (2.98%). Plazo corto (~177 meses ≈ 15 años). Ingresos altos (~151K).
1	Hipoteca Estándar a 30 Años	124,962 (55.1%)	48.5%	El perfil mayoritario del mercado. Préstamos de ~424K, plazo de 30 años (361 meses), tasa moderada (4.30%), propiedades de ~633K (LTV ~67%). Ingresos de ~143K.
2	Préstamos a Largo Plazo con Tasa Elevada	36,707 (16.2%)	82.7%	Préstamos de ~328K con la tasa de interés más alta (6.57%), plazo largo (~334 meses), propiedades de ~576K (LTV ~57%). Ingresos de ~151K.
3	Hipotecas Modestas a Corto Plazo	45,918 (20.2%)	51.8%	Préstamos más bajos (~177K), tasa moderada-alta (5.05%), plazo corto (~167 meses ≈ 14 años), propiedades de ~630K (LTV ~28%). Ingresos más bajos (~135K).

Cluster	Nombre del Perfil	Solicitudes	Tasa Aprob.	Características principales
0	Propiedades Premium con LTV Bajo	19,203 (8.5%)	92.9%	Préstamos de ~293K sobre propiedades de alto valor (~880K), resultando en un LTV muy bajo (~33%). Tasa de interés más baja del mercado (2.98%). Plazo corto (~177 meses ≈ 15 años). Ingresos altos (~151K).
1	Hipoteca Estándar a 30 Años	124,962 (55.1%)	48.5%	El perfil mayoritario del mercado. Préstamos de ~424K, plazo de 30 años (361 meses), tasa moderada (4.30%), propiedades de ~633K (LTV ~67%). Ingresos de ~143K.
2	Préstamos a Largo Plazo con Tasa Elevada	36,707 (16.2%)	82.7%	Préstamos de ~328K con la tasa de interés más alta (6.57%), plazo largo (~334 meses), propiedades de ~576K (LTV ~57%). Ingresos de ~151K.
3	Hipotecas Modestas a Corto Plazo	45,918 (20.2%)	51.8%	Préstamos más bajos (~177K), tasa moderada-alta (5.05%), plazo corto (~167 meses ≈ 14 años), propiedades de ~630K (LTV ~28%). Ingresos más bajos (~135K).

0 | **Propiedades Premium con LTV Bajo** | 19,203 (8.5%) | 92.9% | Préstamos de ~293K sobre propiedades de **alto valor** (~880K), resultando en un LTV muy bajo (~33%). **Tasa de interés más baja** del mercado (2.98%). Plazo corto (~177 meses ≈ 15 años). Ingresos altos (~151K). |

1 | **Hipoteca Estándar a 30 Años** | 124,962 (55.1%) | 48.5% | El perfil **mayoritario** del mercado. Préstamos de ~424K, plazo de **30 años** (361 meses), tasa moderada (4.30%), propiedades de ~633K (LTV ~67%). Ingresos de ~143K. |

2 | **Préstamos a Largo Plazo con Tasa Elevada** | 36,707 (16.2%) | 82.7% | Préstamos de ~328K con la **tasa de interés más alta** (6.57%), plazo largo (~334 meses), propiedades de ~576K (LTV ~57%). Ingresos de ~151K. |

3 | **Hipotecas Modestas a Corto Plazo** | 45,918 (20.2%) | 51.8% | **Préstamos más bajos** (~177K), tasa moderada-alta (5.05%), plazo corto (~167 meses ≈ 14 años), propiedades de ~630K (LTV ~28%). **Ingresos más bajos** (~135K). |

> ■ **Nota metodológica**: los valores mostrados corresponden a los **centroides** de K-Means, que son más representativos que las medias simples por cluster. Las medias aritméticas presentan

desviaciones estándar muy elevadas (especialmente en `loan\_amount` e `income`), indicando presencia de valores extremos dentro de cada cluster.

#### Hallazgos clave del perfilado

**\*\*1. El plazo del préstamo es el principal diferenciador\*\***

Los clusters se separan principalmente entre hipotecas a **\*\*corto plazo\*\*** (~15 años, Clusters 0 y 3) vs. **\*\*largo plazo\*\*** (~30 años, Clusters 1 y 2). Este es el factor más determinante en la segmentación natural del mercado.

**\*\*2. Las tasas de aprobación varían drásticamente entre perfiles\*\***

- Cluster 0 (LTV bajo, plazo corto): **\*\*92.9%\*\*** de aprobación

- Cluster 1 (estándar 30 años): solo **\*\*48.5%\*\***

Esta diferencia de 44 puntos porcentuales es uno de los hallazgos más relevantes del análisis.

**\*\*3. El ratio LTV parece ser un factor clave en la aprobación\*\***

Los clusters con menor LTV (0: ~33%, 3: ~28%) o con tasas elevadas que compensan el riesgo (2: tasa 6.57%) tienen aprobaciones muy superiores al Cluster 1, que presenta el LTV más alto (~67%).

**\*\*4. La aparente paradoja del Cluster 2\*\***

El Cluster 2 combina la **\*\*tasa de interés más alta\*\*** (6.57%) con la **\*\*segunda mejor tasa de aprobación\*\*** (82.7%). Esto no es contradictorio: una tasa elevada indica que el prestamista cobra una prima por mayor riesgo percibido, pero precisamente esa prima hace que la operación sea **\*\*rentable para la entidad\*\***, facilitando la aprobación. En otras palabras, estos solicitantes acceden al crédito pagando un precio más alto por el riesgo.

**\*\*5. El Cluster 1 (mayoritario) tiene la menor tasa de aprobación\*\***

El 55% del mercado solicita hipotecas estándar a 30 años con un LTV relativamente alto (~67%). Este es el segmento más competitivo y donde las entidades son más selectivas, lo que explica la tasa de aprobación más baja (48.5%).

---

## Cell 123: ■ Markdown

### 10.6 Relación entre Clusters y Decisiones de Aprobación

Esta subsección conecta el análisis no supervisado (clustering) con el problema supervisado (clasificación aprobado/denegado). El objetivo es responder:

- **\*\*¿Coinciden las agrupaciones naturales con los patrones de aprobación/denegación?\*\*** Si un cluster tiene una tasa de denegación mucho mayor que otro, eso nos dice que el perfil financiero de ese grupo es más riesgoso o que las políticas de concesión son más estrictas con ellos.

- **\*\*¿Aporta información adicional al modelo?\*\*** Si los clusters capturan estructura que el modelo supervisado no está usando, podríamos incluir la etiqueta de cluster como feature adicional.

**\*\*Tareas:\*\***



1. Calcular la tasa de aprobación por cluster
2. Visualizar con un gráfico de barras
3. Reflexionar sobre los resultados

#### Cell 124: ■ Code

```
10.6.1 Tasa de aprobación por cluster
=====

df_modelo_filtrado ya tiene 'cluster' y 'approved'
approval_by_cluster =
df_modelo_filtrado.groupby('cluster')['approved'].agg(['mean', 'count'])
approval_by_cluster.columns = ['tasa_aprobacion', 'n_solicitudes']
approval_by_cluster['tasa_aprobacion_pct'] =
(approval_by_cluster['tasa_aprobacion'] * 100).round(1)

print("Tasa de aprobación por cluster:")
print(approval_by_cluster)

Gráfico de barras
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

Tasa de aprobación
ax1.bar(approval_by_cluster.index, approval_by_cluster['tasa_aprobacion_pct'],
color='steelblue', edgecolor='black')
ax1.set_xlabel('Cluster')
ax1.set_ylabel('Tasa de Aprobación (%)')
ax1.set_title('Tasa de Aprobación de Préstamos por Cluster')
ax1.set_xticks(approval_by_cluster.index)
for i, v in enumerate(approval_by_cluster['tasa_aprobacion_pct']):
ax1.text(i, v + 1, f'{v}%', ha='center', fontweight='bold')

Número de solicitudes
ax2.bar(approval_by_cluster.index, approval_by_cluster['n_solicitudes'],
color='coral', edgecolor='black')
ax2.set_xlabel('Cluster')
ax2.set_ylabel('Número de Solicitudes')
ax2.set_title('Volumen de Solicitudes por Cluster')
ax2.set_xticks(approval_by_cluster.index)
```

```
plt.tight_layout()
plt.show()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'df_modelo_filtrado' is not defined
```

## Cell 125: ■ Markdown

### Interpretación: Conexión entre Clusters y Modelo Supervisado

La diferencia de **44.4 puntos porcentuales** entre el Cluster 0 (92.9% de aprobación) y el Cluster 1 (48.5%) es el hallazgo más significativo del análisis no supervisado. Esta brecha se explica a través de las mismas variables que el modelo supervisado (Gradient Boosting) identificó como las más importantes:

---

#### Conexión con las variables del modelo SHAP/Feature Importance

| Variable clave (SHAP) | Cluster 0 (92.9%) | Cluster 1 (48.5%) | Efecto en aprobación |

|-----|-----|-----|-----|

| **interest\_rate** (Top 1 RF) | 2.98% (muy baja) | 4.30% (moderada) | Tasa baja → mayor aprobación |

| **LTV implícito** (property\_value / loan\_amount) | ~33% (muy bajo) | ~67% (alto) | LTV bajo → menor riesgo → mayor aprobación |

| **loan\_term** | ~15 años | 30 años | Plazo corto → menor exposición al riesgo |

| **property\_value** (Top 3 SHAP) | ~951K (alto) | ~624K (moderado) | Mayor colateral → mayor seguridad |

---

#### ¿Por qué esta conexión es importante?

1. **Validación cruzada aprendizaje supervisado ↔ no supervisado**: El clustering, que **no utilizó la variable de aprobación**, identificó agrupaciones donde las variables financieras que el modelo supervisado considera más importantes (`interest_rate`, `property_value`, `income`) varían de forma coherente con las tasas de aprobación. Esto **valida la coherencia** de los hallazgos del modelo.

2. **El Cluster 1 es el "campo de batalla"**: Con 124,962 solicitudes (55.1% del mercado) y una tasa de aprobación de solo 48.5%, este cluster contiene la mayor concentración de decisiones difíciles. Su perfil —préstamos grandes (424K), plazo de 30 años, LTV del 67%— corresponde exactamente al segmento donde el modelo supervisado muestra mayor incertidumbre (probabilidades cercanas a 0.5).

3. **\*\*El `debt\_to\_income\_ratio` no aparece en el clustering pero domina el modelo\*\***: La variable más influyente según SHAP (`DTI\_Unknown`, `DTI\_>60%`) no se usó como feature de clustering. Esto sugiere que **\*\*dentro del Cluster 1\*\***, las diferencias de aprobación se deben principalmente al DTI y a la calidad de información de la solicitud — factores que el clustering financiero no captura pero el modelo supervisado sí.

4. **\*\*El Cluster 0 es "autoaprobado"\*\*: Su perfil de bajo riesgo (LTV 33%, tasa 2.98%, plazo corto) explica por qué el 92.9% se aprueba. Estas solicitudes probablemente serían aprobadas por cualquier modelo, lo que confirma que las variables financieras básicas son suficientes para predecir la aprobación en perfiles de bajo riesgo.**

---

**\*\*Conclusión\*\***: El clustering revela que la estructura del mercado hipotecario de Utah tiene una dicotomía clara entre perfiles de bajo riesgo (aprobación casi garantizada) y el grueso del mercado estándar (aprobación ~50%), cuya decisión depende de variables de calidad de crédito (DTI) que solo el modelo supervisado captura. **\*\*Ambas aproximaciones son complementarias\*\***: el clustering explica *\*qué tipo\** de solicitante tiene mayor probabilidad, y el modelo supervisado explica *\*por qué\** dentro de cada tipo.

## Cell 126: ■ Markdown

### 10.7 Dimensión Ética del Clustering: Distribución Demográfica

Este apartado conecta los clusters financieros con los **\*\*atributos sensibles\*\*** (raza, sexo, edad) para responder una pregunta crítica:

> **\*\*¿Se concentran ciertos grupos demográficos en clusters con peores condiciones financieras o menores tasas de aprobación?\***

Si un cluster con alta tasa de denegación muestra una sobrerrepresentación de solicitantes de un grupo demográfico específico, esto podría revelar **\*\*inequidades estructurales\*\*** en el acceso al crédito hipotecario — no necesariamente sesgo algorítmico, sino desigualdades socioeconómicas preexistentes reflejadas en los datos.

Este análisis complementa directamente la **\*\*Sección 9 (Análisis Ético)\*\***, aportando la perspectiva del aprendizaje no supervisado: dado que los clusters se construyeron **\*\*sin utilizar la variable de aprobación ni los atributos demográficos\*\***, cualquier correlación entre clusters y demografía es especialmente reveladora.

## Cell 127: ■ Code

```
# 10.7.1 Composición demográfica por cluster
# =====

# df_modelo_filtrado ya tiene derived_race, derived_sex, applicant_age
print("=" * 60)

print("COMPOSICIÓN POR RAZA EN CADA CLUSTER")

print("=" * 60)
```

```

race_by_cluster = pd.crosstab(df_modelo_filtrado['cluster'],
df_modelo_filtrado['derived_race'], normalize='index') * 100
print(race_by_cluster.round(1))

```

#### Output:

```

=====
COMPOSICIÓN POR RAZA EN CADA CLUSTER
=====

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 8, in <module>
NameError: name 'df_modelo_filtrado' is not defined

```

#### Cell 128: ■ Code

```

10.7.2 Visualización de distribución demográfica por cluster
=====

```

```

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle('Distribución Demográfica por Cluster', fontsize=16)

```

```

Raza
race_by_cluster = pd.crosstab(df_modelo_filtrado['cluster'],
df_modelo_filtrado['derived_race'], normalize='index')
race_by_cluster.plot(kind='bar', stacked=True, ax=axes[0], colormap='Set3')
axes[0].set_title('Composición por Raza')
axes[0].set_xlabel('Cluster')
axes[0].set_ylabel('Proporción')
axes[0].legend(title='Raza', bbox_to_anchor=(1.05, 1), loc='upper left',
fontsize=8)
axes[0].tick_params(axis='x', rotation=0)

```

```

Sexo
sex_by_cluster = pd.crosstab(df_modelo_filtrado['cluster'],
df_modelo_filtrado['derived_sex'], normalize='index')
sex_by_cluster.plot(kind='bar', stacked=True, ax=axes[1], colormap='Set2')
axes[1].set_title('Composición por Sexo')

```

```

axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('Proporción')
axes[1].legend(title='Sexo', bbox_to_anchor=(1.05, 1), loc='upper left',
fontsize=8)
axes[1].tick_params(axis='x', rotation=0)

Edad
age_by_cluster = pd.crosstab(df_modelo_filtrado['cluster'],
df_modelo_filtrado['applicant_age'], normalize='index')
age_by_cluster.plot(kind='bar', stacked=True, ax=axes[2], colormap='Paired')
axes[2].set_title('Composición por Edad')
axes[2].set_xlabel('Cluster')
axes[2].set_ylabel('Proporción')
axes[2].legend(title='Edad', bbox_to_anchor=(1.05, 1), loc='upper left',
fontsize=8)
axes[2].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 8, in <module>
NameError: name 'df_modelo_filtrado' is not defined

```

#### Cell 129: ■ Code

```

# 10.7.3 Tabla cruzada: Cluster × Raza × Tasa de Aprobación
# =====

# Tasa de aprobación por cluster y raza
cross_analysis = df_modelo_filtrado.groupby(['cluster',
'derived_race'])['approved'].agg(['mean', 'count'])
cross_analysis.columns = ['tasa_aprobacion', 'n_solicitudes']
cross_analysis['tasa_aprobacion_pct'] = (cross_analysis['tasa_aprobacion'] *
100).round(1)

# Mostrar solo combinaciones con al menos N solicitudes para evitar ruido

```

```

MIN_SOLICITUDES = 20

cross_filtered = cross_analysis[cross_analysis['n_solicitudes'] >=
MIN_SOLICITUDES]

print(f"Tasa de aprobación por Cluster × Raza (mín. {MIN_SOLICITUDES}
solicitudes):")

print(cross_filtered[['tasa_aprobacion_pct', 'n_solicitudes']])

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 5, in <module>
NameError: name 'df_modelo_filtrado' is not defined

```

### Cell 130: ■ Markdown

## 10.8 Conclusiones del Análisis de Clustering

---

### #### 1. Segmentación del mercado hipotecario de Utah

Se identificaron **4** perfiles naturales de solicitantes, coherentes con la realidad del mercado:

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Cluster	Perfil	% Mercado	Tasa Aprob.	Rasgo diferenciador
0	Propiedades Premium, LTV Bajo	8.5%	92.9%	Préstamos pequeños sobre propiedades caras (LTV ~33%), tasa 2.98%, plazo 15 años
1	Hipoteca Estándar 30 Años	55.1%	48.5%	El grueso del mercado: préstamos de ~424K, tasa 4.30%, LTV ~67%
2	Largo Plazo, Tasa Elevada	16.2%	82.7%	Tasa más alta (6.57%) compensa el riesgo; la prima facilita la aprobación
3	Hipoteca Modesta, Corto Plazo	20.2%	51.8%	Montos más bajos (~177K), plazo ~14 años, ingresos más bajos (~135K)

Los tres algoritmos aplicados (K-Means, DBSCAN y Jerárquico) coinciden en que los datos hipotecarios de Utah **no** presentan clusters fuertemente separados, sino un continuo dominado por un perfil central. **K-Means** resultó ser el más útil para segmentación interpretable, mientras que **DBSCAN** aportó su valor como detector de anomalías.

---

### #### 2. Hallazgos clave sobre aprobación

Las **diferencias en tasas de aprobación entre clusters** son el hallazgo más significativo de esta sección:

- La diferencia entre el cluster con mayor aprobación (92.9%, Cluster 0) y el de menor (48.5%, Cluster 1) es de **44 puntos porcentuales**.
- El factor más determinante no es el ingreso del solicitante (similar entre clusters), sino el **ratio préstamo/valor de propiedad (LTV)**: los clusters con LTV más bajo tienen tasas de aprobación significativamente superiores.
- El **plazo del préstamo** es el segundo factor diferenciador: los plazos cortos (~15 años) se asocian a mayor aprobación que los de 30 años.
- La **tasa de interés elevada del Cluster 2** no es un obstáculo para la aprobación, sino un mecanismo de compensación de riesgo que facilita que las entidades concedan el crédito.

---

### #### 3. Detección de anomalías

DBSCAN identificó un **1.0%** de solicitudes atípicas caracterizadas por:

- **Propiedades de valor extremo** (+3.81 desviaciones estándar sobre la media)
- **Tasas de interés superiores** (+0.80σ), posiblemente por financiación no convencional (préstamos jumbo)
- Montos de préstamo similares al mercado general, lo que implica un **LTV muy bajo**

Estas solicitudes corresponden al **segmento de lujo** — legítimas pero con un perfil de riesgo diferenciado que justifica evaluación especializada.

---

### #### 4. Dimensión ética

La composición demográfica de los clusters muestra diferencias **modestas pero relevantes**:

- Los Clusters 0 y 3 (plazos cortos, mejores condiciones) tienen una proporción ligeramente mayor de solicitantes blancos (~82%) frente a los Clusters 1 y 2 (~71%).
- La categoría "Not Available" varía entre 10% y 22% según el cluster, lo que limita la certidumbre de las conclusiones demográficas.
- No se observa una concentración desproporcionada extrema de grupos minoritarios en los clusters con peores condiciones, aunque las diferencias de ~11 puntos en la proporción de solicitantes blancos entre clusters merecen investigación adicional.

---

### #### 5. Valor añadido del clustering al proyecto

El aprendizaje no supervisado reveló insights que la clasificación supervisada por sí sola no habría descubierto:

1. **La estructura unimodal del mercado**: los datos hipotecarios de Utah no forman grupos discretos, sino un continuo dominante. Esto tiene implicaciones para la modelización (no existen subpoblaciones ocultas que requieran modelos separados).
2. **La importancia del LTV**: los clusters muestran que el ratio préstamo/propiedad es un predictor implícito clave de aprobación, más determinante que el ingreso o la tasa de interés por sí solos.

3. **El segmento de lujo como categoría diferenciada**: el 1% de solicitudes extremas detectado por DBSCAN pertenece a un mercado con dinámicas propias.

**¿Incluir el cluster como feature del modelo supervisado?** No sería recomendable: los clusters se construyeron con las mismas variables financieras ya utilizadas por el modelo, por lo que añadirían **información redundante** con riesgo de sobreajuste. El valor del clustering es **explicativo y estratégico**, no predictivo.

Cell 131: ■ Markdown

11. Conclusiones Generales del Proyecto

---

11.1 Resumen Ejecutivo

Este proyecto ha analizado **226,790** solicitudes de préstamos hipotecarios del estado de Utah (año 2022) utilizando técnicas de Machine Learning supervisado y no supervisado. A continuación se sintetizan los hallazgos principales de cada componente del análisis.

---

11.2 Rendimiento de los Modelos de Clasificación

Se entrenaron tres modelos para predecir la aprobación/denegación de solicitudes:

Modelo	Accuracy	F1-Score	AUC-ROC	Fortaleza principal
	-----	-----	-----	-----
<b>Regresión Logística</b>	~90%	88.67%	~0.94	Máxima interpretabilidad
<b>Random Forest</b>	~96%	96.80%	~0.99	Mejor recall (98.59%)
<b>Gradient Boosting</b> ■	~96%	96.67%	~0.99	Mejor precision (95.49%) y balance global

**Mejor modelo**: Gradient Boosting, seleccionado por su equilibrio óptimo entre precision y recall, minimizando tanto el riesgo financiero (falsos positivos) como la pérdida de oportunidades de negocio (falsos negativos).

---

11.3 Variables más Influyentes

El análisis de interpretabilidad (SHAP + Feature Importance) reveló las variables clave que determinan la aprobación:

- debt\_to\_income\_ratio** — El factor más crítico. La categoría "Unknown" tiene el efecto negativo más fuerte, y el rango 30-42% es el óptimo para aprobación.
- interest\_rate** — Dominante en los modelos basados en árboles. Tasas bajas se asocian fuertemente con aprobación.



3. **`property\_value` e `income`** — Relación directa con la aprobación: mayor valor de propiedad y mayores ingresos aumentan la probabilidad.

4. **`loan\_term`** — Plazos más cortos (15 años) se asocian con menor riesgo y mayor aprobación.

5. **Calidad de datos** — La completitud de la solicitud (DTI, edad, raza) es un predictor implícito de aprobación: la información faltante reduce significativamente la probabilidad.

---

## 11.4 Análisis Ético y Detección de Sesgos

**Hallazgos por raza:**

- Existe una brecha de ~16 pp en tasas de aprobación entre solicitantes White (~65%) y grupos minoritarios (~48-52%).

- Sin embargo, las métricas del modelo (F1, Precision, Recall) son consistentes entre grupos raciales (diferencias < 3.5 pp), lo que indica que el modelo no introduce sesgo adicional más allá del ya presente en los datos históricos.

**Hallazgos por sexo:**

- Male y Female tienen tasas de aprobación prácticamente idénticas (diferencia de ~1 pp), confirmando equidad de género.

- Las solicitudes Joint (pareja) tienen la tasa más alta (68.3%), reflejando la ventaja de dos ingresos.

**Hallazgos por edad:**

- No hay evidencia de discriminación por edad entre grupos conocidos (variación < 3 pp).

- La categoría "Unknown" distorsiona las estadísticas con una tasa de aprobación del 5.13%.

**Conclusión ética:** El mayor factor de inequidad es la **calidad de los datos** (categorías "Not Available" y "Unknown"), no la discriminación directa. No obstante, la brecha racial en tasas de aprobación merece investigación adicional para determinar si refleja diferencias socioeconómicas legítimas o sesgos sistémicos.

---

## 11.5 Segmentación del Mercado (Clustering)

El análisis no supervisado identificó **4 perfiles naturales** de solicitantes:

- **Cluster 0 (8.5%)** — Propiedades Premium, LTV bajo → **92.9% aprobación**

- **Cluster 1 (55.1%)** — Hipoteca estándar 30 años → **48.5% aprobación**

- **Cluster 2 (16.2%)** — Largo plazo, tasa elevada → **82.7% aprobación**

- **Cluster 3 (20.2%)** — Hipoteca modesta, corto plazo → **51.8% aprobación**

La diferencia de 44 pp entre Cluster 0 y Cluster 1 se explica por las mismas variables identificadas por el modelo supervisado (LTV, interest\_rate, loan\_term), validando la coherencia entre ambos enfoques. DBSCAN identificó un 1% de solicitudes atípicas correspondientes al segmento de lujo.

---

## 11.6 Recomendaciones Accionables

1. **\*\*Mejorar la captura de DTI\*\***: Implementar validación obligatoria en formularios para reducir "Unknown DTI", la mayor fuente de denegaciones y errores del modelo.
2. **\*\*Educar a solicitantes sobre DTI óptimo\*\***: Crear herramientas que comuniquen que el rango 30-42% maximiza la aprobación.
3. **\*\*Auditar sesgo racial\*\***: Realizar un análisis causal para determinar si las brechas de aprobación por raza se deben a diferencias de DTI/ingresos o a factores discriminatorios.
4. **\*\*Implementar Gradient Boosting en producción\*\***: Con SHAP values para explicaciones individuales y monitoreo de drift en las distribuciones de DTI e interest\_rate.
5. **\*\*Pipeline diferenciado para datos incompletos\*\***: Las solicitudes con información faltante deberían marcarse para revisión manual en lugar de procesarse automáticamente.

---

## 11.7 Limitaciones del Estudio

- **\*\*Alcance geográfico\*\***: Los resultados son específicos de Utah 2022 y pueden no generalizarse a otros estados o años.
- **\*\*Variables no incluidas\*\***: El dataset HMDA no contiene score crediticio (FICO), historial de pagos u otros factores importantes en la decisión real de préstamo.
- **\*\*Sesgo histórico\*\***: El modelo aprende de decisiones humanas pasadas que pueden contener sesgos implícitos.
- **\*\*Categorías "Unknown"\*\*\***: La elevada proporción de datos faltantes en atributos demográficos limita la robustez de las conclusiones éticas.